

RHEL 9 RHCSA / RHCE 合订讲义

RHEL 9 实操考试备考讲义

目录

[完成一场红帽实操考试的方法](#)

[命令行、Shell 与本地帮助](#)

[文件、目录、文本处理与归档传输](#)

[用户、组、权限、ACL 与 sudo](#)

[进程、作业、服务、日志与计划任务](#)

[软件包、仓库与基础系统维护](#)

[网络、名称解析、SSH 与 firewalld](#)

[分区、文件系统、Swap 与持久挂载](#)

[LVM 逻辑存储](#)

[网络文件系统与自动挂载](#)

[SELinux](#)

[启动过程、Target 与系统恢复](#)

[Podman 容器与持久运行](#)

[RHCSA 综合任务](#)

[Ansible 架构、配置与 Inventory](#)

[YAML、Playbook、Task 与模块](#)

[变量、Facts、注册结果与优先级](#)

[循环、条件、Handler、Block 与错误控制](#)

[文件、模板与 Jinja2](#)

[外部数据、敏感变量与 Vault](#)

[Include、Import、Role 与 Collection](#)

[自动化用户、软件包、仓库、服务与计划任务](#)

[自动化网络、防火墙与 SELinux](#)

[自动化存储、文件系统与挂载](#)

[Ansible 故障排除、验证与幂等性](#)

[RHCE 综合任务](#)

通用章 完成一场红帽实操考试的方法

红帽实操考试要求把系统从给定初态配置到可验证的目标终态。真正困难的不是记住某一条命令，而是在有限时间内持续回答三个问题：题目要求改变哪个对象，当前证据支持哪一步，怎样证明修改既生效又能够保持。本章建立 RHCSA 与 RHCE 共用的读题、操作和验收方法，并分别给出一页检查清单。

[概念] 目标终态（desired end state）是题目真正评分的系统状态；当前状态是操作前或某一步之后系统实际呈现的状态；持久配置决定状态能否在重启、服务重载或下一次自动化执行后继续成立。三者不能由同一条查询自动证明。

[概念] 对象、约束与证据构成实操题的最小模型。对象可能是用户、文件、服务、块设备、端口、Inventory 主机或受管节点；约束说明必须保留什么、不能改变什么；证据用于决定下一步并证明终态。

[操作语义] 调查命令读取对象而不改变目标状态；配置命令或 Playbook 改变对象；验证命令从配置、当前状态、功能、外部访问、持久性和幂等性中证明一个明确层次。

[操作语义] RHCSA 的主线是“手工修改单机终态”，RHCE 的主线是“用 Inventory、变量、模块和 Playbook 表达多机终态”。RHCE 的 `ok` 或 `changed` 只描述任务执行结果，不能替代受管节点上的功能验收。

[知识专题] 把题目改写成对象、状态、约束与证据

① [知识点] 先找评分对象，不从命令名开始

一道题可能写着“配置服务开机可用”，真正的评分对象至少包含软件包、配置文件、服务当前状态、启动配置、监听端口以及必要的访问链路。若只看到“服务”就立即执行 `systemctl start`，得到的只是一个局部当前状态。

读题时把每一项要求压缩成四列：对象是什么，当前状态未知在哪里，目标终态是什么，限制条件是什么。例如“保留现有数据扩展卷”中的“保留现有数据”是硬约束，它会排除重新格式化这条看似快速的路径。

② [知识点] 把模糊动词改写成可判定状态

“配置”“启用”“部署”“保证可访问”都不是单一动作。`enabled` 描述开机启动配置，`active` 描述当前运行；防火墙已放行不代表服务正在监听；目录存在不代表文件系统已经挂载；Playbook 没有 `failed` 不代表模板内容正确。

每个目标都应有能够判定对错的状态表达，例如：用户 UID 为指定值且属于给定附加组；挂载源为目标 UUID、当前已挂载且 `fstab` 可验证；服务处于 `active` 与 `enabled`，端口正在监听且本机或远端访问成功。

③ [边界] 已知条件不是永远可信的当前证据

题目提供的初始状态用于缩小调查范围，但操作过程中对象会互相影响。修改网络可能使远程连接中断；重新加载防火墙可能暴露遗漏；一个错误的 `fstab` 条目会影响启动。完成相关操作前仍应通过最小查询确认关键前提，尤其是设备身份、已有数据、目标主机范围和变量来源。

[Cheatsheet] 读题四问：对象是什么？当前缺什么证据？目标如何判定？哪些状态必须保留？把“配置成功”拆成配置、当前状态、功能与持久性。

[操作专题] 用最小调查建立可恢复的操作基线

① [操作点] 只查询会改变决策的字段

调查不是把所有命令运行一遍，而是为下一步选择证据。存储题先建立设备、文件系统、挂载和 LVM 映射；服务题先看配置语法、状态、日志与监听；RHCE 先确认项目目录、配置来源、Inventory 匹配、SSH 与提权。

可把调查结果保留在考试草稿中：记录题号、对象、当前证据、待验证项。不要在系统中创建会干扰评分的复杂跟踪平台，也不要把口令或 Vault 明文复制到草稿。

② [操作点] 优先选择最小且可重复的修改

最小修改只改变题目要求的对象。编辑配置前保留系统原有结构；已有记录时先查询再选择新增或修改；能够使用专用模块表达目标状态时，不用无条件执行的 `shell`。破坏性命令只能在设备身份、数据要求和题意都明确时使用。

一次完成一个可验证层次。服务配置可按“软件包 → 配置语法 → 服务当前与启动状态 → 监听 → 防火墙 → 功能”推进；存储可按“设备 → 容量对象 → 文件系统 → 当前挂载 → 持久配置 → 数据”推进。

③ [验证点] 每次修改立即留下局部证据

局部验证的价值是缩小故障范围。配置文件修改后先做语法或结构检查，再启动服务；写入 `fstab` 后在重启前运行静态验证和实际挂载；Playbook 先做语法检查、列出主机范围并限制到小组执行。

局部证据必须注明边界：`systemctl is-active` 证明当前运行，不证明开机启动；`findmnt` 证明当前挂载，不证明 `fstab`；`ansible-playbook --syntax-check` 只证明控制端能够解析结构，不证明远端任务可执行。

[Cheatsheet] 查询只看影响决策的字段；修改只推进一个目标层；每步立即验证并写清证明边界；破坏性操作前重新确认对象与保留要求。

[诊断专题] 卡题时保留现场，让下一条证据推进判断

① [诊断点] 先给失败定位层次

失败可以先粗分为输入或语法、对象不存在、权限或连接、容量或依赖、当前状态、持久配置、最终功能。这个分类不等于根因，却能决定下一条查询。命令不存在时先查帮助和软件包；服务失败看状态与日志；远端 unreachable 先查 Inventory、SSH 和提权，不应先改业务任务。

② [诊断点] 不用重复执行代替调查

重复运行相同命令只在幂等验证或确认瞬态问题时有意义。lvextend 报空间不足，应查询 VG 余量；端口新增提示已存在，应查询现有类型；模板任务失败，应查看变量、模板路径和渲染语法。错误信息是证据入口，不是要求“换几个参数试试”。

③ [诊断点] 暂时跳过时保存缺失证据

若一道题耗时过长，记录已经成立的层次、最后错误、尚未确认的对象和可能影响的其他题。尽量保留现场，不使用删除、重建或全局放宽权限来换取短暂成功。完成其他题后再回来，新的系统状态和剩余时间可能让判断更清晰。

[Cheatsheet] 现象 → 所在层 → 当前证据 → 一个假设 → 下一条查询；不要重复失败命令；跳题时记录“已证实、未证实、最后错误、影响范围”。

[操作专题] RHCSA：把单台主机验收到可重启终态

① [操作点] 按风险和依赖安排顺序

先确认身份、主机名、网络和仓库等基础条件，再处理用户、软件、服务、存储、SELinux、容器与恢复任务。高风险的磁盘、启动和网络修改应在证据充分时执行，并在修改后立即验证。题目之间共享对象时，记录后一次修改是否会覆盖前一题。

② [验证点] 一页 RHCSA 检查清单

- 网络地址、路由、DNS、主机名与名称解析符合题意；
- 仓库可用，指定软件包存在，服务配置语法正确；
- 用户 UID、主组、附加组、密码策略、权限、ACL 与 sudo 分别核对；
- 服务同时检查 active、enabled、监听、日志、firewalld 与最终访问；
- 分区、PV/VG/LV、文件系统、Swap、当前挂载与 fstab 分层检查；
- SELinux 保持 Enforcing，文件规则、当前标签、端口类型与 Boolean 使用最小入口；
- cron/at、时间同步、日志持久性和容器用户服务按题意验证；
- findmnt --verify、mount -a 等重启前检查没有未处理错误；
- 不依赖临时 chmod、临时标签、runtime-only 防火墙或手工启动状态；

- 最后从题目清单逐项复核，不以“命令都执行过”代替验收。

③ [边界] 重启是高价值但有前提的验证

重启能够暴露启动配置、挂载和服务依赖问题，但错误的网络或 `fstab` 也可能让系统难以恢复。应先完成静态验证、当前功能检查并保留恢复路径，再根据考试环境和题意决定是否重启。不能把未经检查的重启当作排错第一步。

[Cheatsheet] RHCSA：调查 → 手工修改 → 当前状态 → 功能 → 持久配置 → 交叉影响 → 最终清单；重启前先验证网络、`fstab` 与关键服务。

[操作专题] RHCE：从控制端语法推进到多节点幂等终态

① [操作点] 先证明执行边界

确认当前工作目录中的 `ansible.cfg` 是否生效，Inventory 是否解析到预期主机与组，连接用户、Python 与 become 是否可用。执行前使用 `--syntax-check`、`--list-hosts`、`--list-tasks`，必要时以 `--limit` 缩小第一轮范围。

② [验证点] 一页 RHCE 检查清单

- 项目目录、`ansible.cfg`、Inventory 与 requirements 文件位于题目要求路径；
- 主机组、嵌套关系、变量目录和 Vault 密码来源能够被当前项目读取；
- YAML 语法、模块 FQCN、字段类型、模板与变量名经过静态检查；
- 第一轮限制范围执行，区分 `failed`、`unreachable`、`changed`、`ok` 与 `skipped`；
- Handler 只由真实变更通知，错误控制没有掩盖必须失败的任务；
- 文件内容、所有者、模式、服务、端口、防火墙、SELinux、存储和挂载在受管节点验证；
- 不把控制节点文件存在误认为远端文件已经正确；
- 扩展到全部目标主机后再次核对 recap 与每组终态；
- 第二次执行不应出现无法解释的 `changed`，并再次检查远端功能；
- Vault 内容保持加密，输出和提交文件中没有明文秘密。

③ [验证点] 第二次执行只证明幂等性的一个维度

第二次执行没有 `changed`，说明模块认为目标状态已满足；若模板本身写错、Inventory 漏掉主机或验证对象选错，错误终态同样可能稳定不变。因此幂等检查必须和受管节点上的真实文件、服务、端口、挂载与业务访问组合使用。

[Cheatsheet] RHCE：配置/Inventory → 语法与主机范围 → 限制执行 → 远端终态 → 扩展执行 → 第二次执行 → 再验收；recap 不是最终功能证明。

[经典任务] 在混合任务清单中建立可验证的执行顺序

你接手一组尚未开始的考试任务：其中包含静态网络、软件仓库、共享目录、持久挂载、非标准 Web 服务，以及一个面向多组主机的自动化部署。部分任务共享服务、路径和端口；存储设备已有状态未知；自动化项目已提供若干变量文件。

目标终态：在不删除未知数据、不降低 SELinux 保护、不泄露敏感变量的前提下，为这些任务建立一条可执行顺序。每个任务都要说明操作前需要的证据、完成后最少验证层次、可能影响的其他任务，以及最终综合验收方法。

限制条件：不能用重建对象代替排错；不能以服务 active 代替外部访问；不能以 Playbook recap 成功代替远端终态；暂时跳过的任务必须能够恢复上下文。

验收要求：执行计划能够区分 RHCSA 与 RHCE 路径，包含重启前检查、幂等性检查、交叉影响检查和两份最终清单。

请先独立完成。参考分析与推荐路径从下一页开始。

[参考解答] 用证据门把高风险任务和依赖任务串起来

① [操作点] 读题拆解与依赖排序

先建立对象表：网络与名称解析影响仓库和远程连接；仓库影响软件包；存储设备身份决定能否写入；Web 访问依赖服务配置、监听、防火墙与 SELinux；自动化依赖控制端配置、Inventory、SSH、提权和变量。共享对象旁标记题号，避免后续操作覆盖已有终态。

推荐顺序是先做低风险基础调查，再完成网络、名称解析和仓库；随后处理身份、文件与服务；设备证据充分后处理存储；最后完成 SELinux 与外部功能闭环。RHCE 项目先做控制端静态检查和小范围执行，再扩展到所有受管节点。

② [验证点] 为每类任务设置证据门

网络任务至少验证连接配置、活动地址、路由、DNS 与必要连通；共享目录验证身份、组继承、权限与新文件行为；存储验证设备归属、容量对象、文件系统、当前挂载、持久条目与数据；Web 服务验证配置语法、active/enabled、监听、firewalld、SELinux 和最终请求。

自动化在执行前证明配置来源、主机范围和变量可解析；执行后在远端检查文件、服务与功能；第二次执行检查无法解释的变更。任何证据门失败时停留在该层调查，不删除已经正确的下层对象。

③ [诊断点] 卡题与最终收束

卡题记录“目标对象、已成立状态、最后错误、缺失证据、共享影响”。转去其他题时不留下临时 Permissive、全局权限放宽或半写入的关键配置。返回后从缺失证据继续，而不是重新执行全部命令。

最终按 RHCSA 与 RHCE 检查清单逐项验收。重启前先检查持久网络、`fstab` 和关键服务；是否重启取决于恢复条件。RHCE 在全量执行和第二次执行后仍要检查受管节点终态。

[Cheatsheet] 建对象表与共享影响 → 基础证据 → 低风险依赖 → 高风险存储/网络 → 服务完整链 → RHCE 小范围到全量 → 持久与幂等 → 最终清单。

[本章收束] 让每一步都能回答“改变了什么，证明了什么”

实操考试的稳定方法不是背一条固定流水线，而是持续维护对象、状态和证据之间的对应关系。调查只读取会影响决策的信息；修改只推进一个目标层；验证说明自己能证明什么、不能证明什么；卡题时保留现场并记录缺失证据。

RHCSA 最终把单机配置验收到当前、功能和持久状态；RHCE 还要证明主机范围、变量化、多节点终态与第二次执行。只要不把局部成功扩大成整体结论，就能把复杂任务拆成可恢复、可验证的小闭环。

第一章 命令行、Shell 与本地帮助

命令行题首先考察的不是输入速度，而是 Shell 如何把字符转换为命令、参数和数据流。引号、展开、重定向或退出状态理解错误，会让一条看似正确的命令作用于错误对象。本章把命令解析、输入输出、本地帮助和安全组合命令组织成可验证的操作方法。

[概念] Shell 是读取命令行、完成展开和重定向、再启动命令的解释环境。命令名、选项和位置参数只是常见约定；真正支持哪些参数，应以本机帮助和手册为准。

[概念] 标准输入（stdin，文件描述符 0）、标准输出（stdout，1）和标准错误（stderr，2）是进程默认的数据通道。管道只把前一命令的 stdout 连接到后一命令的 stdin，stderr 不会自动进入管道。

[操作语义] `type` 判断名称如何被 Shell 解析；`man`、`--help`、`apropos` 和 `info` 从不同入口查询本机用法；`dnf provides` 从文件路径反查软件包。

[操作语义] `>` 重建输出文件，`>>` 追加，`2>` 单独重定向错误，`2>&1` 让 stderr 指向 stdout 当前目标；`tee` 在管道中同时写文件并继续输出。

[知识专题] Shell 解析、引用与展开顺序

① [知识点] 一条命令先由 Shell 解释

Shell 先识别引号、变量、命令替换、通配符和重定向，再把结果作为参数交给程序。空格通常分隔参数，因此含空格的路径必须引用；通配符由 Shell 展开为现有文件名，程序通常看不到原始的 `*`。

```
printf '%s\n' "$HOME"      # 双引号内展开变量
printf '%s\n' '$HOME'      # 单引号保留字面文本
printf '%s\n' './*.conf'   # Shell 展开匹配文件
```

双引号保留变量和命令替换，但抑制分词与文件名展开；单引号几乎逐字保留内容。需要在单引号文本中出现单引号时，应结束引用、转义该字符、再重新开始，而不是不断增加反斜线猜测结果。

② [知识点] 命令替换与退出状态回答不同问题

`$(command)` 把命令 stdout 去掉末尾换行后嵌入当前命令；`$?` 是上一条前台命令的退出状态。状态 `0` 通常表示该命令按自身定义成功，非零表示不同类型失败，但 `0` 不能证明整个业务目标完成。

```
today=$(date +%F)          # 捕获标准输出
tar -cjf "backup-{$today}.tar.bz2" /srv # 使用替换结果
printf 'exit=%s\n' "$?"    # 紧接着读取退出状态
```

读取 `$?` 前若先运行了其他命令，得到的就是后一命令状态。需要基于成功与否继续执行时，`cmd1 && cmd2` 只在前者成功后执行后者；`cmd1 || cmd2` 只在前者失败后执行后者。分号无条件继续，不能表达

验证门。

③ [边界] 变量必须明确是否允许分词

路径、用户输入和命令替换结果通常应使用 "\$var"。未引用变量会经历分词和文件名展开，空格或 * 可能改变参数数量。不要把命令及参数整体塞进字符串再依赖未引用展开；复杂命令应直接书写，脚本需要动态参数时使用数组。

[Cheatsheet] 字面文本用单引号；需要变量展开用双引号；命令输出用 \$(...)；上一命令状态用 \$?；成功链用 &&，失败分支用 ||。

[操作专题] 精确控制 stdout、stderr 与管道

① [参数点] 覆盖、追加和错误通道不能混淆

> 在命令启动前打开并截断目标文件，即使命令随后失败，旧内容也可能已经丢失。>> 追加到末尾。数字 2 明确选择 stderr；没有数字默认处理 stdout。

```
find /etc -name '*.conf' > /tmp/files.txt          # 覆盖 stdout
find /etc -name '*.conf' 2> /tmp/find-errors.txt    # 单收 stderr
date >> /tmp/run.log                                # 追加 stdout
```

合并两个通道时，顺序决定含义：>file 2>&1 先让 stdout 指向文件，再让 stderr 复制该目标；2>&1 >file 先让 stderr 指向当时的终端，随后只改变 stdout，因此不会把两者都写入文件。

② [操作点] 管道只传递可消费的数据

管道适合把稳定文本交给筛选或统计工具。前一命令的错误默认仍显示在终端。如果题意要求错误也进入后续命令，可显式使用 2>&1 |；但排错时把错误混入数据会污染结果，通常应分别保存。

```
journalctl -u sshd --since today | grep -i 'fail' | wc -l # 日志筛选与计数
printf '%s\n' alpha beta | tee /tmp/items | sort           # 写文件并继续传递
```

tee -a 追加而不是覆盖。需要 root 权限写受保护文件时，sudo command > /root/file 的重定向仍由当前 Shell 执行；可让具有权限的 tee 接收管道数据，但必须确认覆盖或追加语义。

③ [验证点] 检查文件内容、通道和状态

用 wc -l 或 sed -n 检查结果规模与代表性内容，用 stat 确认目标文件，用单独错误文件确认是否遗漏权限失败。管道默认的 \$? 是最后一个命令状态；交互排错可查看 Bash 的 PIPESTATUS 数组，脚本需要整条管道任一环节失败时可考虑 set -o pipefail。

[Cheatsheet] stdout: > / >> ; stderr: 2> / 2>> ; 合并到同一目标: >file 2>&1 ; 管道只接 stdout; 边看边存: tee , 追加用 tee -a 。

[操作专题] 从目标反查命令、参数与软件包

① [操作点] 先识别当前名称是什么

`type` 是 Shell 语义入口，能区分 alias、关键字、函数、内建命令和外部可执行文件。`which` 主要按 PATH 查可执行文件，无法完整解释别名或 Shell 内建；`command -V` 也能给出解析结果。

```
type cd           # Shell 内建
type -a ls        # 列出所有可解析入口
command -V useradd # 描述命令解析结果
```

② [操作点] 按由近到远的帮助入口查询

已知命令时先看 `command --help` 的短语法，再用 `man command` 查完整说明。Shell 内建可用 `help <builtin>`；配置文件常有独立手册，如 `man 5 passwd`。`man` 的 section 区分普通命令、系统调用、库、设备、文件格式和管理命令。

```
useradd --help    # 短选项入口
man useradd        # 完整命令手册
man 5 crontab      # 文件格式手册
help test          # Bash 内建帮助
```

不知道命令名时，先确保 `mandb` 索引可用，再用 `apropos 'password aging'` 或 `man -k` 搜索描述。`info` 对部分 GNU 工具有更完整的主题式说明。

③ [操作点] 命令缺失时反查包

先确认不是 PATH、拼写或别名问题。若确实缺少文件，用路径模式查询仓库：

```
dnf provides '*/semanage'    # 按可执行文件名反查包
dnf provides /usr/bin/rsync   # 已知完整路径时精确查询
```

查询结果只证明仓库元数据中哪个包提供文件；还要确认仓库可用、安装正确包，并重新用 `type` 或 `rpm -qf` 验证。

④ [验证点] 用帮助得到最小可执行答案

从手册中提取本题需要的命令骨架、关键参数、对象位置和验证入口，不抄参数大全。运行前可用 `--help` 复核大小写和增量符号；运行后仍按对象状态验收，帮助页不会证明配置已生效。

[Cheatsheet] 名称解析：`type -a`；短帮助：`--help / help`；完整手册：`man`；按描述找命令：`apropos`；按文件找包：`dnf provides '*/name'`。

[诊断专题] 从空结果、错误文件和退出状态推进证据

① [诊断点] 空输出不一定是成功答案

`grep` 没有输出可能表示无匹配，也可能上游文件不可读；`find` 结果不全可能伴随 `stderr` 的 `Permission denied`；通配符未匹配时在默认 `Bash` 配置下可能原样传给程序。先查看退出状态和错误通道，再解释空结果。

② [诊断点] “command not found” 先解析与安装

检查拼写、`type -a`、`echo "$PATH"` 和绝对路径。只有确认命令文件不存在后，才使用 `dnf provides`。不要为了一个命令无条件安装猜测的软件包，也不要当前目录加入 `PATH` 作为通用修复。

③ [诊断点] 文件被清空先检查重定向时机

`sort file > file` 会在 `sort` 读取前截断同一文件。应写入临时文件，验证后原子替换，或使用工具支持的原地模式。看到零字节文件时，不要重复相同命令，应先保留现场和确认是否有备份。

[Cheatsheet] 空结果查退出状态与 `stderr`；找不到命令查 `type /PATH` 再查包；输入输出是同一文件时禁止直接 `> 原文件`；重复执行前先判断 `Shell` 已经改变了什么。

[经典任务] 在陌生系统中找到归档命令并保存可审计结果

系统需要从 `/etc` 中找出名称以 `.conf` 结尾的普通文件，将可读取的路径排序后保存到 `/root/conf-files.txt`，把权限错误单独保存到 `/root/conf-errors.txt`，并创建日期为当天的 bzip2 归档 `/root/conf-YYYY-MM-DD.tar.bz2`。

你不确定归档工具的压缩参数，也不确定相关命令是否已安装。不得把错误信息混入路径列表，不得因为某些文件不可读而覆盖已有归档；最终要证明路径列表、错误文件、归档格式和归档内容分别符合要求。

卡住时再想一想：哪一通道进入排序？日期怎样安全嵌入文件名？帮助和软件包查询分别解决什么问题？

请先独立完成。参考解答从下一页开始。

[参考解答] 分离数据与错误，再按帮助构造归档

① [操作点] 调查命令与目标文件

先用 `type -a find sort tar date` 确认解析入口，用 `tar --help` 或 `man tar` 查 `-c`、`-j`、`-f` 的语义。若工具缺失，再执行 `dnf provides '*/tar'`，不要从记忆猜包。

② [操作点] 分离输出并建立日期变量

`find` 的 stdout 进入 `sort`，stderr 单独写入错误文件。先写临时路径列表，验证后再替换正式文件，避免失败时覆盖已有结果。

```
today=$(date +%F) # 生成 ISO 日期
find /etc -type f -name '*.conf' \
  2> /root/conf-errors.txt | sort > /root/conf-files.txt.tmp
test -s /root/conf-files.txt.tmp && \
  mv /root/conf-files.txt.tmp /root/conf-files.txt # 有结果才替换
```

③ [操作点] 依据列表创建归档

`tar -T` 从文件读取成员列表；`-c` 创建，`-j` 使用 bzip2，`-f` 后紧跟归档名。

```
tar -cjf "/root/conf-${today}.tar.bz2" \
  -T /root/conf-files.txt # 按路径清单归档
```

④ [验证点] 分别证明四个对象

```
sed -n '1,10p' /root/conf-files.txt # 抽查排序路径
wc -l /root/conf-files.txt /root/conf-errors.txt # 结果与错误规模
file "/root/conf-${today}.tar.bz2" # 归档格式
tar -tjf "/root/conf-${today}.tar.bz2" | sed -n '1,10p' # 成员抽查
```

退出状态只说明对应命令结果；路径列表、错误通道、压缩格式和成员内容需要分别检查。

[Cheatsheet] `type` / 帮助确认语法 → stdout 排序、stderr 分离 → `date +%F` → `tar -cjf ... -T list` → `wc + file + tar -tjf` 分层验收。

[本章收束] 把字符写法还原为参数和数据流

Shell 题的核心是知道谁在解释字符。引号和展开决定程序实际收到哪些参数；重定向和管道决定数据流向哪里；退出状态只描述一条命令或管道的局部结果；本地帮助提供准确语法，却不能替代操作后的对象验证。

面对陌生命令，先用 `type` 确认解析，再从短帮助、手册、描述搜索和包反查逐步靠近答案。面对空输出或错误，先分离 `stdout` 与 `stderr`，并保留证据。这样命令行就不是符号记忆，而是一套可推演、可验证的接口。

第二章 文件、目录、文本处理与归档传输

文件题把路径、对象类型、内容筛选和传输连接在一起。结果文件存在并不代表找到的是正确对象，归档可打开也不代表成员路径符合要求。本章从目录树和链接语义出发，建立“选择对象 → 处理内容 → 归档或传输 → 验证成员与目标”的闭环。

[概念] 绝对路径从 `/` 开始，含义不依赖当前目录；相对路径从当前工作目录解释。inode 保存文件元数据并指向数据，目录项把名称映射到 inode；硬链接共享 inode，符号链接保存另一个路径文本。

[概念] 文件类型、文件名和内容是三个不同筛选维度。扩展名不决定 Linux 文件类型；`find` 依据目录树与元数据选择对象，`grep` 依据文本内容选择行。

[操作语义] `cp` 复制对象，`mv` 改名或移动目录项，`rm` 删除目录项，`install` 可在复制时设置目录、所有者和模式；`stat`、`file`、`du` 与 `df` 分别观察元数据、内容类型、目录占用与文件系统容量。

[操作语义] `tar` 把成员及元数据组织为归档；`scp` 通过 SSH 复制，`sftp` 提供交互传输，`rsync` 比较源和目标并同步差异。传输成功后仍要在目标端核对路径、大小和内容。

[知识专题] 路径、inode 与链接的判断边界

① [知识点] 目录的执行权限控制路径穿越

文件的 `r` 允许读取内容，`w` 允许修改内容，`x` 允许作为程序执行；目录的 `r` 允许列出名称，`w` 允许创建和删除目录项，`x` 允许穿越并访问已知名称。某个文件本身可读，但任一父目录缺少 `x`，访问仍会失败。

`pwd` 确认当前目录，`readlink -f` 解析可解析的最终路径，`namei -l` 逐层显示路径权限。排错时不要只检查最后一个文件的 `ls -l`。

② [知识点] 硬链接与符号链接失效方式不同

硬链接是同一文件 inode 的另一个名称，只能在同一文件系统内创建，通常不能指向目录。删除一个硬链接只减少链接计数，其他名称仍可访问数据。符号链接拥有自己的 inode，内容是目标路径，可跨文件系统并可指向目录；目标移动或删除后它会悬空。

```
ln /srv/report /srv/report.hard      # 同一 inode 的新名称
ln -s /srv/report /opt/report.current # 保存目标路径
stat -c '%i %h %n' /srv/report /srv/report.hard # inode 与链接数
readlink /opt/report.current          # 查看链接保存的路径
```

相对符号链接从链接所在目录解释，不是从创建命令时的工作目录解释。迁移一组目录时，相对链接可能更便携，但必须推演目标位置。

③ [验证点] 区分对象、目标与数据

用 `stat` 比较 inode 和链接计数，用 `test -L` 识别符号链接，用 `readlink -f` 查看最终目标。内容校验可用 `sha256sum`，但相同校验值只证明字节相同，不证明权限、所有者、时间戳或链接关系相同。

[Cheatsheet] 路径逐层权限：`namei -l`；元数据：`stat`；硬链接同 inode、不能跨文件系统；符号链接保存路径、可悬空；字节验证用 `sha256sum`。

[操作专题] 安全创建、复制、移动和删除对象

① [操作点] 创建目标前确认父目录和覆盖边界

`mkdir -p` 创建缺失父目录且对已存在目录不报错。`cp -a` 尽量保留递归结构和元数据，普通 `cp -r` 只表达递归；考试题是否需要保留所有者、权限和时间，应由目标终态决定。`install -D -m` 适合一次创建父目录、复制文件并设置模式。

```
mkdir -p /srv/project/archive          # 建立目录树
install -D -m 0640 source.conf /etc/app/app.conf # 复制并设模式
cp -a /srv/project/. /backup/project/    # 包含隐藏成员
```

源目录末尾的 `/.` 表示复制其内容；复制目录本身还是内容会改变目标层级。执行前用 `ls -ld` 和 `find <DIR> -maxdepth 1` 检查两端。

② [边界] 删除与移动作用于目录项

`rm -r` 和覆盖式 `mv` 可能不可恢复。通配符结果必须先用非破坏性命令预览，变量必须引用。不要用 `rm -rf` 作为权限或挂载排错工具；挂载点未挂载时删除目录内容，可能删除根文件系统中的数据。

③ [验证点] 元数据与内容分别验收

```
stat -c '%U %G %a %F %n' /etc/app/app.conf    # 所有者、组、模式、类型
sha256sum source.conf /etc/app/app.conf        # 字节内容
find /backup/project -maxdepth 2 -printf '%y %p\n' # 结构抽查
```

`du -sh` 统计目录实际占用，`df -hT <PATH>` 查询该路径所在文件系统的容量，两者对象不同，不能用 `du` 解释文件系统预留空间，也不能用 `df` 证明某目录内容大小。

[Cheatsheet] 建树：`mkdir -p`；复制并设模式：`install -D -m`；保留目录结构：`cp -a`；覆盖/删除前预览展开；验收：`stat + sha256sum + find`。

[操作专题] 用 find 选择对象，再批量处理

① [参数点] 起点、条件和动作必须分开读

`find <START> <TESTS> <ACTIONS>` 从起点递归。 `-type f` 选择普通文件， `-name` 区分大小写， `-iname` 忽略大小写， `-user / -group` 按身份， `-size` 按大小， `-mtime` 按 24 小时时段。

```
find /var/log -type f -name '*.log' -size +10M -print # 大于 10 MiB
find /home -user student -type f -mtime -7 -print    # 近 7 个 24h 周期
```

`+10M` 是严格大于， `-10M` 是严格小于， `10M` 是按 `find` 的单位向上取整后匹配。时间边界容易受取整影响，核心任务应结合 `stat` 抽查，不把一个条件机械解释为精确时间戳区间。

② [操作点] 批量动作避免空格和特殊字符破坏

优先使用 `-exec command {} +`，它把多个路径作为独立参数批量传递；需要逐文件动作时用 `\;`。向支持 NUL 的工具传递列表，可用 `-print0` 与 `xargs -0`。不要用 `for f in $(find ...)`，它会按空白分词。

```
find /srv/inbox -type f -name '*.txt' \
    -exec cp -t /srv/selected -- {} +          # 安全传递路径
```

复制前先把动作改为 `-print` 检查集合。 `--` 结束选项解析，避免以 `-` 开头的文件名被当成参数。

③ [验证点] 验证集合而不只看目标目录存在

保存源清单和目标清单，比较计数与代表性路径；需要字节一致时再计算校验值。目标目录中可能已有同名文件，单纯比较数量不能证明一一对应。

[Cheatsheet] `find` 起点 条件 动作；先 `-print` 预览；批量安全动作：`-exec ... {} +`；NUL 管道：`-print0 | xargs -0`；时间/大小边界用 `stat` 抽查。

[操作专题] 从文本中提取稳定证据

① [操作点] grep 选择行，字段工具再塑形

`grep -E` 使用扩展正则， `-i` 忽略大小写， `-n` 显示行号， `-v` 反选， `-r` 递归。模式以 `-` 开头时使用 `-e` 或 `--`。锚点 `^ / $` 匹配行首/行尾，字符类 `[[:space:]]` 比在命令中输入不易辨认的空白更可靠。

```
grep -nE '^[[:space:]]*#[^[:space:]]' /etc/ssh/sshd_config # 非空非注释行
cut -d: -f1,3 /etc/passwd | sort -t: -k2,2n             # 用户名与 UID 排序
```

`sort` 排序, `uniq` 只折叠相邻重复行, 因此通常先排序; `wc -l` 统计换行记录; `tr` 做字符级替换。
`sed` 和 `awk` 在考试中应服务于明确的行或字段变换, 不写难以现场验证的长程序。

② [边界] 输出格式是接口

命令的面向人类默认输出可能随语言、终端宽度或版本变化。脚本或严格题目优先选择明确字段选项、稳定分隔符或专用查询格式。解析 `/etc/passwd` 可使用冒号字段, 但不要把带空格的普通输出硬拆成列。

③ [验证点] 对结果做规模、样本和反例检查

查看前后若干行、统计数量, 并用一个应匹配与一个不应匹配的样本验证模式。只有输出文件非空不足以证明正则没有过宽。

[Cheatsheet] 行筛选 `grep -nE`; 字段 `cut / awk`; 排序去重 `sort | uniq`; 计数 `wc -l`; 验证同时看数量、样本和反例。

[操作专题] 创建归档并通过 SSH 传输

① [参数点] 归档成员名称由工作目录和源路径决定

`tar -c` 创建, `-t` 列表, `-x` 提取, `-f` 指定归档; `-z` `gzip`, `-j` `bzip2`, `-J` `xz`。用 `-C` 先切换目录, 可避免归档中出现不需要的长前缀。

```
tar -C /srv -cJf /root/project.tar.xz project      # 成员以 project/ 开始
tar -tJf /root/project.tar.xz | sed -n '1,20p'    # 不解包检查成员
```

② [操作点] 根据传输语义选择工具

`scp source user@host:/path/` 适合直接复制; `sftp` 适合交互; `rsync -a` 递归并保留常用元数据, `-n` `dry-run`, `--delete` 会删除目标多余文件, 必须由题意明确授权。`rsync` 源目录末尾斜线表示复制目录内容, 不带斜线通常复制目录本身。

```
rsync -an /srv/project/ serverb:/srv/project/    # 先预览差异
rsync -a /srv/project/ serverb:/srv/project/     # 执行同步
```

③ [验证点] 本地归档与远端落点分别检查

先用 `tar -t` 检查成员与路径前缀, 再在目标端运行 `stat`、`sha256sum` 或第二次 `rsync -an`。SSH 命令退出 0 只说明传输工具没有报告失败, 不证明目标路径是题目要求的位置。

[Cheatsheet] 归档: `tar -C <BASE> -c[zjJ]f <ARCHIVE> <MEMBER>`; 查看: `tar -t...`; `rsync` 先 `-n`; 源末尾 `/` 决定复制内容还是目录; 目标端复核路径和校验值。

[诊断专题] 结果存在但层级、集合或内容不正确

① [诊断点] 多出一层目录

复制或 rsync 后目标出现 `/dest/source/source...`，先比较源末尾斜线和目标是否已存在，不要继续重复同步。用 `find -maxdepth 2` 看真实层级，再选择移动或重新以正确语义执行。

② [诊断点] find 漏掉含空格文件

若使用了命令替换或未引用变量，路径可能被拆分。回到 `-exec ... {} +` 或 NUL 分隔，不要通过禁止空格文件名掩盖管道错误。

③ [诊断点] 归档可打开但成员路径错误

`tar -t` 直接显示归档内名称。问题在创建时的工作目录和源参数，不应通过解包后手工移动来声称原归档合格；重新使用 `-C` 创建并验证。

[Cheatsheet] 层级错查源/目标末尾 `/`；集合漏项查分词与错误通道；归档路径错查 `tar -t` 和 `-C`；结果存在不等于对象关系正确。

[经典任务] 筛选日志、生成归档并传送到指定主机

从 `/var/log/app` 中找出所有者为 `appsvc`、大小超过 1 MiB、七个 24 小时周期内修改的普通 `.log` 文件。路径可能包含空格。将文件复制到 `/srv/submission/logs`，保留其相对于起点的目录结构；生成 `/srv/submission/app-logs.tar.xz`，归档成员必须从 `logs/` 开始；传送到 `serverb:/srv/archive/`。

不得删除或改写源文件，不得把查找错误当作成员路径。验收时证明筛选集合、复制层级、归档成员、远端落点和归档校验值。

请先独立完成。参考解答从下一页开始。

[参考解答] 用 NUL 路径流保留层级并分层验收

① [操作点] 预览集合与边界

```
find /var/log/app -type f -user appsvc -name '*.log' \
    -size +1M -mtime -7 -print                # 先人工抽查
```

确认条件后创建目标。GNU `cp --parents` 能保留给定路径结构；先切到起点，避免把 `/var/log/app` 整段带入目标。

② [操作点] 安全复制路径集合

```
mkdir -p /srv/submission/logs
cd /var/log/app
find . -type f -user appsvc -name '*.log' -size +1M -mtime -7 \
    -exec cp --parents -- {} /srv/submission/logs/ \; # 保留相对层级
```

③ [操作点] 创建并传输归档

```
tar -C /srv/submission -cJf /srv/submission/app-logs.tar.xz logs
scp /srv/submission/app-logs.tar.xz serverb:/srv/archive/
```

④ [验证点] 五层验收

```
find /srv/submission/logs -type f -printf '%P\n' | sort
tar -tJf /srv/submission/app-logs.tar.xz | sed -n '1,20p'
sha256sum /srv/submission/app-logs.tar.xz
ssh serverb 'stat /srv/archive/app-logs.tar.xz; sha256sum /srv/archive/app-logs.tar.xz'
```

比较源集合与目标相对路径，确认所有归档成员以 `logs/` 开始，并让本地与远端校验值一致。若 `find` 有权限错误，应单独保留并处理，不能静默当作完整集合。

[Cheatsheet] 预览条件 → 从起点生成相对路径 → `cp --parents` 保层级 → `tar -C` 控制成员前缀 → `scp` → 路径集合 + `tar -t` + 双端校验值。

[本章收束] 先证明选中了谁，再证明对它做了什么

文件操作的稳定顺序是先确认路径与对象类型，再建立筛选集合，最后执行复制、归档或传输。链接关系由 `inode` 与目标路径证明，文本筛选要检查规模、样本与反例，批量路径必须避免空白分词。

归档和远端文件都是新对象：归档成员名称、压缩格式、远端落点和字节内容需要分别验证。只要把“对象集合”和“处理结果”分成两层，文件题就不会因为一个存在的输出文件而提前结束。

第三章 用户、组、权限、ACL 与 sudo

身份与权限题要求把账号属性、组成员关系、目录访问和新建对象的继承行为同时配置正确。已有文件可写，不代表新文件会继承相同权限；用户属于组，也不代表当前登录会话已经获得新组。本章用“身份 → 基础权限 → 特殊位与 ACL → 账号策略 → sudo → 实际身份验证”建立闭环。

[概念] 用户由 UID 标识，组由 GID 标识。用户有一个主组和零个或多个附加组；文件只记录一个所有者 UID、一个所属组 GID 和权限模式。名称只是系统文件对数字身份的映射。

[概念] 权限模式按 owner、group、other 三类主体分配 `rwX`。ACL 可为额外用户或组增加条目，并由 mask 限制 named user、named group 与文件 group class 的有效权限；默认 ACL 只影响目录中新创建的后代对象。

[操作语义] `useradd` / `usermod` / `userdel` 管理账号属性，`groupadd` / `groupmod` 管理组；`id` 与 `getent` 查询解析后的身份；`passwd` 与 `chage` 管理口令和期限状态。

[操作语义] `chmod` 修改模式位，`chown` / `chgrp` 修改所有者和组；`getfacl` / `setfacl` 读取和修改 ACL；`visudo` 校验 sudoers 语法，`sudo -l` 显示当前身份可执行的授权。

[知识专题] 账号、主组、附加组与会话身份

① [知识点] 三个账号文件承担不同职责

`/etc/passwd` 保存用户名、UID、主 GID、注释、家目录和登录 Shell；口令哈希及期限字段在 `/etc/shadow`；组名、GID 和显式成员在 `/etc/group`。不要把 `/etc/group` 成员列表为空解释为没人属于该组，因为用户可通过 `passwd` 中的主 GID 归属。

```
getent passwd alice          # 使用系统名称服务查询账号
getent group project         # 查询组及显式成员
id alice                     # 汇总 UID、主组和附加组
```

`getent` 比直接 `grep` 本地文件更符合名称服务语义；考试环境若只用本地账号，结果仍来自相同文件。

② [参数点] 创建与修改的选项必须区分

`useradd -u` 指定 UID，`-g` 指定主组，`-G` 指定附加组，`-d` 指定家目录路径，`-m` 创建家目录，`-s` 指定登录 Shell。`usermod -aG` 追加附加组；漏掉 `-a` 的 `-G` 会把附加组列表替换为给定集合。

```
groupadd -g 3000 project      # 创建指定 GID 的组
useradd -u 2001 -g project -G wheel \
    -m -s /bin/bash alice     # 创建指定身份账号
usermod -aG developers alice   # 追加附加组
```

账号创建后用 `id`、`getent passwd` 和 `getent group` 分别核对。不要仅以 `useradd` 退出 0 证明所有 UID、Shell 和组关系都符合题意。

③ [边界] 现有会话不会自动刷新组列表

进程在启动时获得 UID/GID 和附加组。管理员把已登录用户加入新组后，该用户现有 Shell 通常仍使用旧组列表；需要重新登录或启动具有新组身份的会话。排错时分别运行 `id <USER>` 查询账号数据库和在用户会话中运行 `id` 查询进程实际身份。

[Cheatsheet] 账号解析：`getent passwd/group`；最终身份：`id`；创建主组 `-g`、附加组 `-G`；修改时追加必须 `usermod -aG`；组变更后重新建立会话。

[知识专题] 文件与目录上的 `rw`、`umask` 和特殊位

① [知识点] 同一字母在文件与目录上含义不同

普通文件的 `r` 是读取内容，`w` 是修改内容，`x` 是执行；目录的 `r` 是列出目录项，`w` 是创建、删除和改名目录项，`x` 是穿越与访问已知名称。删除文件主要由父目录权限决定，即使文件本身只读，拥有父目录 `wx` 的用户仍可能删除目录项。

权限检查先选择 owner、group 或 other 中唯一一类，不会把多类权限相加。若用户是文件所有者，就只使用 owner 位，即使所属组位更宽。

② [知识点] `umask` 从创建请求中屏蔽权限

普通文件常以最大请求 `0666` 创建，目录以 `0777` 创建，再由 `umask` 清除位。`umask 0027` 常得到文件 `0640`、目录 `0750`；程序可主动请求更窄权限，因此结果不一定只由 `umask` 决定。

```
umask                                # 查看当前 mask
umask 0027                          # 仅改变当前 Shell 及其后代
```

持久 `umask` 应放在题目指定的 Shell 配置范围，并通过新登录会话创建样本验证；仅运行一次 `umask` 不是持久配置。

③ [知识点] `setgid` 与 `sticky` 解决不同目录问题

目录 `setgid` (`2`) 让新建对象继承目录所属组，适合协作目录；`sticky` (`1`) 限制可写目录中的删除/改名，通常只允许文件所有者、目录所有者或 `root` 操作。`setuid` (`4`) 主要作用于可执行文件，使进程获得文件所有者的有效 UID，不应作为普通共享目录方案。

```
chown root:project /srv/project
chmod 2770 /srv/project          # setgid + owner/group rw
chmod 1777 /srv/ Dropbox         # 公共写入但限制互删
```

④ [验证点] 用实际身份创建对象

`stat -c '%A %a %U %G %n'` 验证目录模式与身份，再用 `sudo -u alice` 和 `sudo -u bob` 创建、读取、修改及删除样本。只看目录位不能证明用户的当前组或父路径权限正确。

[Cheatsheet] 目录访问需逐层 `x`；删除由父目录 `wx` 主导；`umask` 是清除位；协作组继承用 `setgid 2`；公共目录防互删用 `sticky 1`；最终用实际用户验证。

[操作专题] 建立 `setgid` 协作目录并控制新文件权限

① [操作点] 先建立身份与目录所有权

```
groupadd project          # 创建协作组
usermod -aG project alice # 追加成员
usermod -aG project bob
mkdir -p /srv/project
chown root:project /srv/project # 设置目录所属组
chmod 2770 /srv/project      # 组继承与组内完全访问
```

`setgid` 只保证新对象组继承，不保证新文件一定组可写。若用户 `umask` 为 `0022`，新文件通常是 `0644`，组仍不可写。可以为协作用户配置合适 `umask`，或使用默认 `ACL` 明确新对象的组权限。

② [验证点] 分别验证账号数据库和新对象行为

```
id alice
stat -c '%A %a %U %G %n' /srv/project
sudo -u alice touch /srv/project/alice.txt
stat -c '%A %a %U %G %n' /srv/project/alice.txt
sudo -u bob sh -c 'printf "%s\n" team >> /srv/project/alice.txt'
```

最后一条功能测试能证明 `bob` 当前实际身份可以修改 `alice` 新建的文件；若失败，依次检查 `bob` 的会话组、目录逐层权限、文件 `group` 和有效权限。

[Cheatsheet] 组 → `usermod -aG` → `chown root:<GROUP>` → `chmod 2770`；`setgid` 管组继承，`umask`/默认 `ACL` 管新对象可写性；用两个真实身份交叉验证。

[操作专题] 使用访问 `ACL` 与默认 `ACL` 表达额外授权

① [知识点] `access ACL` 与 `default ACL` 作用时点不同

访问 `ACL` 描述当前对象谁能访问；目录的默认 `ACL` 是新建子对象的继承模板，不会自动回溯修改已有文件。`setfacl -m u:alice:rw` 修改访问条目，`-m d:u:alice:rw` 修改默认条目；`-x` 删除指定条目，`-b` 清除扩展访问 `ACL`，属于可能扩大或收缩权限的动作，不能作为通用排错。

② [操作点] 同时配置当前目录和后续对象

```
setfacl -m u:auditor:rx /srv/project          # 当前目录访问
setfacl -m d:u:auditor:rx /srv/project        # 后代默认条目
setfacl -m d:g:project:rx,d:m:rx /srv/project # 组默认权限与 mask
```

ACL mask 限制 named user、named group 和所属组条目的有效权限。getfacl 中某条权限后出现 #effective:，说明条目写得更宽但被 mask 截断。修改模式位中的 group class 也可能改变 ACL mask。

③ [验证点] 旧对象与新对象分别检查

```
getfacl -p /srv/project
sudo -u alice touch /srv/project/new.txt
getfacl -p /srv/project/new.txt
sudo -u auditor test -r /srv/project/new.txt
```

目录默认 ACL 存在只证明模板已配置；必须创建新对象检查继承，并用目标身份测试功能。已有对象若也需授权，要单独修改其访问 ACL。

[Cheatsheet] 当前访问：setfacl -m u:<USER>:<PERMS>；默认继承加 d:；查询：getfacl；有效权限受 mask 限制；默认 ACL 只在新对象创建时生效。

[操作专题] 配置密码期限、账号锁定与最小 sudo 授权

① [操作点] 区分口令锁定、账号过期和 Shell 禁止登录

passwd -l 锁定口令认证，passwd -u 解锁；它不一定终止 SSH 公钥等其他认证方式。chage -E 设置账号过期日期，-M 最大口令天数，-m 最小天数，-W 到期前警告天数，-d 0 强制下次登录修改口令。

```
chage -M 90 -m 1 -W 7 alice          # 配置口令期限
chage -l alice                        # 查看可读状态
passwd -S alice                       # 查看口令状态摘要
```

/sbin/nologin 阻止交互登录 Shell，但不能被笼统描述为“完全禁用账号”。选择控制入口前先明确题目要求禁止哪种访问。

② [操作点] sudoers 只授予必要命令

主文件是 /etc/sudoers，也可在 /etc/sudoers.d/ 放片段。使用 visudo 或 visudo -cf <FILE> 校验语法和包含关系；文件权限通常为 0440。命令规范应使用绝对路径，并明确是否允许参数；ALL= (ALL) ALL 是广泛授权，不应在只要求管理单个服务时使用。

```
%webops ALL=(root) /usr/bin/systemctl restart httpd, /usr/bin/systemctl status httpd
```

NOPASSWD: 只控制 `sudo` 是否再次询问口令，不改变允许命令范围。通配符和可编辑脚本可能扩大授权，应避免把用户可修改的程序交给 `root` 执行。

③ [验证点] 语法、可见授权与实际执行三层验证

```
visudo -cf /etc/sudoers.d/webops          # 语法
sudo -l -U alice                          # 账号可见规则
sudo -u alice sudo -n /usr/bin/systemctl status httpd # 按授权实际测试
```

若规则不使用 `NOPASSWD`，`-n` 会因需要口令而失败，此时应在合适交互环境验证。`sudo -l` 结果正确仍不能证明命令目标或服务功能正确。

[Cheatsheet] 期限：`chage`；口令状态：`passwd -S/-l/-u`；sudoers 用 `visudo -cf`；授权最小到绝对命令路径；验收：语法 + `sudo -l` + 实际目标身份执行。

[诊断专题] 从 Permission denied 定位身份、路径、模式或 ACL

① [诊断点] 当前进程没有新附加组

账号数据库中的 `id alice` 正确，但 `alice` 已登录 Shell 的 `id` 没有新组，应重新登录或创建新会话。反复 `chmod` 会掩盖真正的会话身份问题。

② [诊断点] ACL 条目存在但 effective 权限不足

运行 `getfacl` 查看 `mask` 和 `#effective:`。修正目标 ACL 或 `mask` 后重新用实际身份测试；不要无条件把 `mask` 设为 `rwX`，因为这会同时放宽整个 `group class`。

③ [诊断点] 文件权限正确但路径仍不可达

用 `namei -l` 检查每级父目录 `x`，再确认 SELinux 等其他安全层。只给最终文件 `chmod 777` 既可能无效又扩大权限。

[Cheatsheet] `id` 查进程身份 → `namei -l` 查父路径 → `stat` 查模式/身份 → `getfacl` 查条目与 `mask` → 目标身份功能测试；不要先全局放宽。

[经典任务] 为项目团队建立共享目录和受控提权

创建 `project` 组以及用户 `alice`、`bob`、`auditor`。`alice` 与 `bob` 属于项目组；`/srv/project` 只允许项目成员创建和修改内容，新对象自动继承 `project` 组且组成员可协作。`auditor` 只能读取和穿越当前目录及后续新对象，不得获得写权限。

`alice` 的 UID 为 2001，登录 Shell 为 `/bin/bash`，口令最大期限 90 天、提前 7 天警告。项目组成员只允许以 `root` 身份查看和重启 `httpd`，不得获得通用 `root` Shell。已有文件不得删除或重建。

验收要求：账号属性、当前会话组、目录模式、已有/新对象 ACL、跨用户写入、`auditor` 只读、`sudoers` 语法与允许/拒绝命令分别有证据。

请先独立完成。参考解答从下一页开始。

[参考解答] 分开配置身份、继承、有效权限和 sudo

① [操作点] 创建账号与期限

```
groupadd project
useradd -u 2001 -g project -m -s /bin/bash alice
useradd -g project -m -s /bin/bash bob
useradd -m -s /bin/bash auditor
chage -M 90 -W 7 alice
```

② [操作点] 配置目录模式和 ACL

```
mkdir -p /srv/project
chown root:project /srv/project
chmod 2770 /srv/project
setfacl -m u:auditor:rx /srv/project
setfacl -m d:u:auditor:r-X,d:g:project:rwX,d:m:rwx /srv/project
```

若已有文件也要求 auditor 读取，应按题意对现有树设置访问 ACL，不能依赖默认 ACL 回溯生效。执行递归修改前先预览对象范围。

③ [操作点] 写入最小 sudoers 片段

```
%project ALL=(root) /usr/bin/systemctl status httpd, /usr/bin/systemctl restart httpd
```

将其保存为 `/etc/sudoers.d/project-httpd`，设置 `0440`，并运行 `visudo -cf`。

④ [验证点] 用不同身份完成验收

```
id alice; id bob; id auditor
chage -l alice
stat -c '%A %a %U %G %n' /srv/project
getfacl -p /srv/project
sudo -u alice touch /srv/project/from-alice
sudo -u bob sh -c 'echo team >> /srv/project/from-alice'
sudo -u auditor test -r /srv/project/from-alice
sudo -u auditor test ! -w /srv/project/from-alice
sudo -l -U alice
```

另以项目成员验证允许命令，并确认未授权命令被拒绝。验证不应改变现有业务服务终态，必要时选择 `status` 作为非破坏性检查。

[Cheatsheet] 身份与期限 → `2770` setgid 目录 → 访问 ACL + 默认 ACL → 新对象交叉测试 → 最小 sudoers → `visudo` + `sudo -l` + 允许/拒绝验证。

[本章收束] 权限结果取决于身份、路径和创建时规则

用户能否访问对象，先取决于进程实际 UID/GID 和附加组，再经过父路径穿越、模式位、ACL mask 以及 SELinux 等其他层。setgid 解决新对象组继承，umask 或默认 ACL 决定创建时权限，它们不能互相替代。

账号策略同样要按作用范围验证：口令锁定不一定阻止公钥登录，nologin 主要阻止交互 Shell，sudoers 授权必须缩小到必要命令。最终使用目标身份创建、读取、修改、删除和提权，才形成完整证据。

第四章 进程、作业、服务、日志与计划任务

进程处理正在运行的实例，systemd 处理受管理的 unit，日志提供失败证据，计划任务则在另一时间和有限环境中启动命令。把它们混在一起，容易出现“手工执行成功、定时执行失败”或“服务已启动、重启后未运行”。本章按运行实例、管理声明、证据和调度四层组织操作。

[概念] 进程是程序的一次运行实例，具有 PID、父 PID、用户、状态、优先级和打开资源；Shell job 是当前 Shell 对一个或多个进程的交互管理视图；systemd unit 是服务、socket、target、mount 或 timer 等受管理对象的声明。

[概念] service 的 active 状态描述当前运行，enabled 描述启动依赖中是否建立持久链接，masked 表示禁止启动。日志记录是证据，不等于服务终态；时间和时区错误会直接影响日志关联与计划任务。

[操作语义] ps / pgrep / top 查询进程，kill / pkill 发送信号，nice / renice 调整调度优先级；jobs / fg / bg 只管理当前 Shell 的作业表。

[操作语义] systemctl 查询和改变 unit 当前及启动状态；journalctl 从 systemd journal 取证；crontab 描述周期任务，at 提交一次性任务，systemd timer 把调度与 service unit 组合。

[知识专题] 进程、作业、信号与优先级

① [知识点] 先选择稳定标识再操作

PID 唯一标识当前进程实例，但重启后可复用。ps -ef 适合父子关系，ps aux 适合资源概览，ps -eo 可选择稳定字段；pgrep -a 按名称查 PID 和命令行，pidof 适合已知程序名。

```
ps -eo pid,ppid,user,stat,ni,etimes,cmd --sort=-etimes
pgrep -a -u appsvc worker
```

操作前核对用户、完整命令行和父子关系。仅按模糊名称 pkill 可能命中不相关进程。

② [知识点] 信号是请求，不是删除进程

kill <PID> 默认发送 SIGTERM (15)，允许程序清理退出；SIGKILL (9) 由内核立即终止，进程无法处理或清理，应只在 TERM 无效且对象确认后使用。SIGHUP (1) 常被守护进程用于重新加载，但具体语义由程序定义。

```
kill -TERM 1234          # 请求有序退出
ps -p 1234 -o pid,stat,cmd # 验证是否仍存在
kill -KILL 1234          # 最后手段
```

僵尸进程已经退出，只等待父进程回收，向僵尸发送更多信号无效；应调查父进程。不可中断睡眠状态 D 常在等待内核 I/O，SIGKILL 也要等阻塞返回。

③ [知识点] nice 值越大, CPU 优先级越低

普通 nice 范围通常为 -20 到 19, 数值越小优先级越高。普通用户可降低自己进程优先级, 但提高优先级需要相应权限。nice 只影响调度倾向, 不是 CPU 配额或性能完成证明。

④ [边界] Shell job 只存在于所属 Shell

`command &` 后台启动, `jobs` 列出当前 Shell 作业, `fg %1` 拉回前台, `bg %1` 继续已停止作业。关闭终端可能向作业发送 SIGHUP; 需要长期服务时应使用 `systemd` 或合适会话工具, 不把 `nohup` 当成服务管理替代品。

[Cheatsheet] 查实例: `ps -eo / pgrep -a`; 先 TERM 后验证, 必要时 KILL; 僵尸查父进程; nice 越大优先级越低; `jobs/fg/bg` 只属于当前 Shell。

[操作专题] 管理 systemd 当前状态与启动配置

① [知识点] start、enable、mask 作用于不同状态

`systemctl start` 现在启动, `stop` 现在停止; `enable` 建立启动依赖, `disable` 移除; `enable --now` 同时启用并启动。 `mask` 把 unit 链接到 `/dev/null`, 连手工或依赖启动也会被阻止, 解除用 `unmask`。

`restart` 停止后重新启动, 可能中断服务; `reload` 请求进程重新读取配置, 前提是 unit 支持。配置是否支持 reload 应由 `systemctl show`、unit 定义或服务文档确认。

② [操作点] 修改配置后先校验, 再改变进程

```
httpd -t # 服务专用语法检查
systemctl reload httpd # 支持时平滑加载
systemctl is-active httpd
systemctl is-enabled httpd
```

unit 文件或 drop-in 变化后执行 `systemctl daemon-reload`, 它让 systemd 重新读取 unit 定义, 不会自动重启服务。应用自身配置变化通常不需要 daemon-reload, 而需要应用校验和 reload/restart。

③ [验证点] status 适合入口, 机器判定用专用查询

`systemctl status` 汇总近期日志和状态, 适合调查; `is-active`、`is-enabled` 适合明确判定。`list-dependencies` 查依赖, `cat` 显示主 unit 和 drop-in, `show -p` 获取属性。

```
systemctl status httpd --no-pager
systemctl cat httpd
systemctl show httpd -p ActiveState,SubState,UnitFileState
ss -lntp | grep ':80' # 功能链的监听层
```

服务 active 只证明 systemd 认为主进程处于活动状态; 仍需按题意检查监听、日志、外部访问和数据。

[Cheatsheet] 当前: `start/stop/reload/restart`; 持久: `enable/disable`; 禁止启动: `mask`; unit 变化才 `daemon-reload`; 验收 `is-active` + `is-enabled` + 服务功能。

[操作专题] 从 journal 与持久日志提取服务证据

① [参数点] 按 unit、启动轮次、时间和优先级收缩

`journalctl -u <UNIT>` 按 unit, `-b` 当前启动, `-b -1` 上一次启动, `--since/--until` 按时间, `-p err` 按优先级上限, `-f` 跟随新日志。组合过滤比对整份 `journal grep` 更精确。

```
journalctl -u httpd -b --since '-10 min' --no-pager
journalctl -p err..alert -b
journalctl --disk-usage
```

日志时间依赖系统时钟和时区。取证前用 `timedatectl` 确认当前时区与同步状态, 避免把错误时间范围解释为“没有日志”。

② [知识点] journal 持久性取决于存储配置

`journald` 可写入易失 `/run/log/journal` 或持久 `/var/log/journal`。RHEL 配置由 `/etc/systemd/journal.conf` 及 drop-in 控制; 建立持久目录或设置 Storage 后应重启/重载相应服务并验证跨启动记录。不要因为当前能看到日志就断言重启后保留。

`rsyslog` 可把消息按 facility/priority 规则写入文本文件或远端。修改 `/etc/rsyslog.conf` 或 `/etc/rsyslog.d/*.conf` 后使用 `rsyslogd -N1` 检查语法, 再重启服务并用 `logger` 生成受控测试消息。

③ [验证点] 受控事件比等待业务日志更可靠

```
logger -p local0.notice -t exam-check 'logging path test'
journalctl -t exam-check --since '-2 min'
grep -F 'logging path test' /var/log/custom.log
```

测试消息证明当前路由链; 若题目要求持久性, 还需检查配置、目录和启动后记录。日志到达文件不证明轮转策略或远端传输正确。

[Cheatsheet] journal 收缩: `-u` + `-b` + `--since` + `-p`; 时间先看 `timedatectl`; `rsyslog` 配置先 `rsyslogd -N1`; 用 `logger` 产生可识别事件并查两端。

[操作专题] 配置时区、chrony 与时间同步

① [操作点] 时区与同步是独立维度

`timedatectl set-timezone Asia/Shanghai` 改显示时区，不改变 UTC 时间来源；`timedatectl set-ntp true` 请求启用系统配置的网络时间机制。在 RHEL 9 常由 `chronyd` 提供 NTP 客户端服务。

```
timedatectl
chronyc sources -v          # 时间源选择与可达性
chronyc tracking             # 本机同步与偏差状态
```

`chronyc sources` 中 `^*` 常表示当前选中的服务器，`^+` 是可接受候选，`^?` 表示不可达或无有效测量。单看服务 `active` 不能证明已经选到时间源。

② [操作点] 修改时间源后分层验证

编辑 `/etc/chrony.conf` 或 drop-in 时按题意添加 `server/pool`，先检查名称解析和网络，再重启 `chronyd`。大偏差是否立即 step 受 `makestep` 配置和运行阶段影响，不应无条件手工改系统时间。

③ [验证点] 服务、源和跟踪三层

```
systemctl is-active chronyd
systemctl is-enabled chronyd
chronyc sources -v
chronyc tracking
```

这组证据分别说明进程、启动配置、可用源/选择和当前时钟跟踪，不由其中一条替代全部。

[Cheatsheet] 时区：`timedatectl set-timezone`；`chronyd` 状态：`systemctl`；源：`chronyc sources -v`；同步质量：`chronyc tracking`；`active` 不等于已同步。

[操作专题] 用 cron 与 at 运行可审计任务

① [知识点] 用户 crontab 与系统 crontab 字段不同

用户 `crontab -e` 每行是五个时间字段加命令：分钟、小时、日、月、星期；`/etc/crontab` 和 `/etc/cron.d/` 在时间字段后多一个执行用户。把两种格式混用会让命令无法按预期执行。

```
15 2 * * * /usr/local/sbin/report.sh >>/var/log/report.log 2>&1
```

日字段和星期字段同时受限时具有特殊 OR 语义，复杂日历需求应现场查 `man 5 crontab`，不要凭直觉组合。

② [操作点] 计划任务必须使用可预测环境

cron 的 PATH、工作目录和环境通常比交互 Shell 精简。使用绝对路径，脚本自身设置必要环境和安全 umask；显式重定向 stdout/stderr。不要依赖 alias、当前目录、交互提示或未导出的变量。

`at <TIME>` 提交一次性任务，`atq` 列表，`at -c <JOB>` 查看展开后的作业，`atrm <JOB>` 删除。提交成功只证明队列中存在，仍要确认 `atd` 服务和执行结果。

③ [验证点] 缩短周期只用于受控验证

先手工以目标用户和最小环境运行脚本，再临时选择较近时间验证，检查输出文件、日志和 `journalctl -u crond / -u atd`。验证后恢复题目周期，避免留下高频任务。

```
crontab -l -u appsvc
systemctl is-active crond
journalctl -u crond --since today
```

④ [边界] systemd timer 是另一套调度对象

timer unit 触发同名或指定 service unit，可用 `systemctl list-timers` 查询，支持相对启动时间和持久补跑等语义。题目明确要求 cron 时不要擅自改用 timer；遇到现有 timer 则同时检查 timer 与被触发 service 的日志。

[Cheatsheet] 用户 cron：5 时间字段 + 命令；系统 cron 多执行用户；任务用绝对路径并重定向；一次性任务 `at/atq/at -c`；验收队列/配置 + 调度服务 + 实际产物。

[诊断专题] 服务或计划任务失败时按运行层取证

① [诊断点] 服务启动失败

先 `systemctl status` 获取主错误，再 `journalctl -u ... -b` 收缩日志；随后运行应用专用语法检查，检查端口冲突、权限和 SELinux。不要反复 restart 冲掉最接近失败时刻的证据。

② [诊断点] 手工成功但 cron 失败

以目标用户在 `env -i` 的精简环境中运行，检查绝对路径、工作目录、权限、变量和错误重定向；再查 `crond` 日志。给 cron 增加完整交互 PATH 可能掩盖脚本依赖，应让脚本明确依赖。

③ [诊断点] enabled 但没有运行

`enabled` 只是启动依赖，当前可能失败、被条件跳过或启动后退出。检查 `is-active`、`status` 和本次启动 `journal`；若 `masked`，先确认为什么被 `mask`，而不是直接 `unmask`。

[Cheatsheet] `status` 定位 → `unit journal` → 应用语法 → 端口/权限/SELinux；cron 差异查用户与环境；`enabled ≠ active`；保留最近失败证据再修改。

[经典任务] 配置服务、持久日志和周期健康检查

服务器已安装 `httpd`，但服务无法启动。修复现有配置而不删除站点内容，使其当前运行并开机启动。将 `local0.notice` 消息持久写入 `/var/log/web-health.log`，每天 02:15 以用户 `monitor` 运行 `/usr/local/sbin/web-health.sh`，追加输出与错误到同一日志。

系统时区应为 `Asia/Shanghai`，`chronyd` 使用题目指定时间源并处于有效跟踪状态。不得通过清空配置、禁用 SELinux 或高频 `cron` 掩盖问题。

验收要求：服务语法、`active/enabled`、监听与本机访问；日志路由和受控消息；`cron` 字段、执行身份、环境与实际产物；时区、`chronyd` 服务、源和 `tracking` 分别有证据。

请先独立完成。参考解答从下一页开始。

[参考解答] 从日志证据修复服务并验证调度链

① [诊断点] 收集服务失败证据

```
systemctl status httpd --no-pager
journalctl -u httpd -b --since '-15 min' --no-pager
httpd -t
ss -lntp
```

根据具体语法或端口证据做最小修正，再运行 `httpd -t`，执行 `systemctl enable --now httpd`，核对 active/enabled、监听和 `curl -I http://localhost/`。

② [操作点] 配置日志路由与时间

在 `/etc/rsyslog.d/web-health.conf` 写入 `local0.notice /var/log/web-health.log` 对应规则，执行 `rsyslogd -N1` 后重启 rsyslog。用唯一消息验证 journal 与文件。

```
timedatectl set-timezone Asia/Shanghai
systemctl enable --now chronyd
chronyc sources -v
chronyc tracking
logger -p local0.notice -t web-health 'route verification'
grep -F 'route verification' /var/log/web-health.log
```

③ [操作点] 配置并验证 cron

先以 monitor 手工运行脚本并修正绝对路径/权限，再写入用户 crontab：

```
15 2 * * * /usr/local/sbin/web-health.sh >>/var/log/web-health.log 2>&1
```

确认 monitor 对日志目标具有题目允许的写入路径；更稳妥的设计可以让脚本使用 `logger`，但应服从现有题意。检查 `crontab -l -u monitor`、crond 状态和受控近时执行产物，最后恢复 02:15。

④ [验证点] 最终分层验收

```
systemctl is-active httpd rsyslog crond chronyd
systemctl is-enabled httpd chronyd
ss -lntp | grep ':80'
curl -I http://localhost/
rsyslogd -N1
crontab -l -u monitor
timedatectl; chronyc sources -v; chronyc tracking
```

[Cheatsheet] status/journal/语法查服务 → active/enabled/监听/访问 → rsyslog 语法 + logger 路由 → 目标用户与精简环境测脚本 → cron 配置 + 实际产物 → 时间四层验收。

[本章收束] 运行实例、管理状态和调度环境各有证据

进程工具观察或影响当前实例，systemd 同时管理当前 unit 与启动依赖，journal/rsyslog 保存不同持久范围的证据，cron/at 在独立时间和环境中启动命令。一个层次成功不会自动推进其他层次。

稳定排错从最近失败日志出发，先做应用语法检查，再改变进程。计划任务则先以目标身份和精简环境验证，再检查调度服务和真实产物。最终把 active、enabled、监听、功能、日志、时间和调度逐项验收。

第五章 软件包、仓库与基础系统维护

软件题不止要求“安装成功”，还要确认仓库来源、包版本、文件落点和服务入口。RPM 数据库描述已安装状态，DNF 使用仓库元数据求解依赖并执行事务；repo 文件正确也不等于远端内容可达。本章从仓库可用性开始，沿事务、包文件和服务 unit 完成验收。

[概念] RPM package 是带元数据、文件和脚本的安装单元，RPM 数据库记录本机已安装包；repository 提供包和 repodata；DNF 读取多个仓库、解析依赖、选择版本并调用 RPM 完成事务。

[概念] 包名、NEVRA (Name-Epoch-Version-Release-Architecture)、文件路径和命令名是不同查询入口。命令缺失时先从路径反查提供包；文件已存在时可从 RPM 数据库反查所属包。

[操作语义] `dnf install/remove/upgrade` 改变包事务；`dnf search/info/list/repoquery/provides` 查询仓库或包元数据；`rpm -q` 查询已安装数据库，`rpm -V` 比较已安装文件与包元数据。

[操作语义] `dnf repolist` 观察启用仓库，`dnf makecache` 实际获取元数据；repo 文件中的 `baseurl`、`enabled`、`gpgcheck` 和 `gpgkey` 分别描述位置、启用、签名校验和密钥入口。

[知识专题] RPM 已安装状态与 DNF 仓库状态

① [知识点] 查询对象决定命令入口

`rpm -q <PACKAGE>` 查询本机是否安装，`rpm -qi` 看已安装信息，`rpm -ql` 列包文件，`rpm -qf <PATH>` 从现有文件反查所属包。`dnf info` 默认可能同时显示 installed 和 available 信息；`dnf list --installed` 或 `--available` 明确范围。

```
rpm -q httpd                # 已安装包状态
rpm -ql httpd | grep -E 'systemd|conf'  # 包内文件
rpm -qf /usr/sbin/httpd      # 文件所属包
dnf repoquery -l httpd       # 仓库包文件列表
```

`repoquery` 可查询未安装包，RPM 查询只依赖已安装数据库。看到仓库中存在包不能证明本机已安装；看到二进制文件也要确认它是否受 RPM 管理。

② [知识点] 验证包文件不是自动修复

`rpm -V <PACKAGE>` 将当前文件的大小、校验值、模式、所有者、组等与包元数据比较，无输出通常表示这些可验证属性没有差异。配置文件被合法修改也会报告差异，因此结果是调查线索，不应直接重装覆盖。

③ [边界] 直接 rpm 安装不解析完整仓库依赖

`rpm -ivh file.rpm` 处理本地包但不会像 DNF 一样从仓库自动求解依赖。安装本地 RPM 时优先 `dnf install ./file.rpm`，让 DNF 处理依赖和事务。`--nodeps`、`--force` 会绕过保护，不是常规解法。

[Cheatsheet] 已安装：`rpm -q/qi/ql/qf`；仓库：`dnf info/list/repoquery`；文件完整性线索：`rpm -V`；本地 RPM 优先 `dnf install ./file.rpm`，不绕过依赖。

[操作专题] 配置并证明软件仓库可用

① [知识点] repo 文件最小字段与变量

仓库片段位于 `/etc/yum.repos.d/*.repo`，每个 section ID 必须唯一。`name` 是显示名，`baseurl` 指向包含 repodata 的根，`enabled=1` 启用，`gpgcheck=1` 校验包签名，`gpgkey` 指向可信公钥。

```
[exam-baseos]
name=Exam BaseOS
baseurl=http://content.example.com/rhel9/BaseOS
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

`$releasever`、`$basearch` 可由 DNF 展开；在 Shell 中生成文件时必须防止它们被 Shell 提前展开。题目给出固定 URL 时直接使用，不把 RHEL 10 路径代入 RHEL 9。

② [操作点] 写入后用元数据请求验证

<code>dnf repolist --all</code>	# 配置解析与启用状态
<code>dnf clean metadata</code>	# 必要时清理旧元数据
<code>dnf makecache --refresh</code>	# 实际刷新启用仓库
<code>dnf repoquery --repo exam-baseos bash</code>	# 限定仓库查询代表包

`repolist` 显示 ID 不能证明 URL 可达；`makecache --refresh` 才会请求元数据。限定 repo 的查询能证明目标仓库确实提供预期包，但不证明所有包都完整。

③ [验证点] 区分配置、网络和签名层

元数据失败先读取完整错误 URL，检查名称解析、路由、代理、HTTP 状态和 repodata 路径。GPG 错误则检查题目提供的 key、系统时间和包签名，不用 `gpgcheck=0` 作为默认修复。

[Cheatsheet] repo 字段：ID/name/baseurl/enabled/gpgcheck/gpgkey；配置可见 `repolist --all`；真实可用 `makecache --refresh`；来源验证 `repoquery --repo`；签名问题不靠关闭 `gpgcheck`。

[操作专题] 查找、安装、升级和删除软件

① [操作点] 从目标能力反查包

已知描述用 `dnf search`，已知命令或路径用 `dnf provides`。通配模式应引用，避免 Shell 展开。

```
dnf search 'web server'
dnf provides '*/semanage'
dnf provides /usr/bin/rsync
```

查询结果可能包含多个版本或兼容包，应结合 RHEL 9 仓库、架构和目标命令选择，不机械安装第一项。

② [操作点] 事务前读摘要，事务后查数据库

`dnf install <PKG>` 安装，`upgrade <PKG>` 升级指定包，`upgrade` 升级可用包，`remove` 可能连带删除依赖。执行前阅读 Transaction Summary，确认没有意外删除或跨仓库替换。

```
dnf install httpd
rpm -q httpd
rpm -ql httpd | grep -E '/httpd\.service$|/httpd\.conf$'
systemctl cat httpd
```

安装包不会自动保证服务配置、active 或 enabled。包、配置文件和 unit 都确认后，继续按服务章节完成当前与持久状态。

③ [知识点] history 提供事务线索

`dnf history` 列出事务，`dnf history info <ID>` 查看包变化。undo/rollback 可能受当前仓库中旧版本是否仍存在影响，也可能撤销后来依赖变更，不能作为无条件恢复按钮。

④ [验证点] 包、文件、unit 与功能分层

`rpm -q` 证明安装数据库，`rpm -ql/qf` 证明文件归属，`systemctl cat` 证明 unit 入口，应用版本/语法和功能检查证明使用结果。`dnf` 返回成功只覆盖事务层。

[Cheatsheet] 找能力 `search/provides` → 看事务摘要 → `dnf install/upgrade/remove` → `rpm -q/ql/qf` → unit/配置/功能；历史查 `dnf history info`，不盲目 rollback。

[知识专题] 模块流、包组和版本选择边界

① [知识点] AppStream 模块影响可见版本集合

RHEL 9 的部分软件以 AppStream 模块流组织。`dnf module list` 查看流，`module info` 看 profile；启用流会影响依赖求解，安装 profile 会安装一组包。不是所有包都属于模块，且具体可用流由当前 RHEL 9 仓库决定。

② [边界] switch/reset 是有状态操作

改变模块流前先检查已启用流和已安装包。dnf module reset 重置选择，不等于删除已安装内容；切换流可能需要专用 switch-to 流程。考试题未要求版本流时，不额外改变模块状态。

③ [知识点] 包组名称与环境组需查询

dnf group list、group info 查询组，dnf group install '<GROUP>' 安装。显示名可能受语言影响，组 ID 更稳定；不要把一组包安装成功解释为相关服务已配置。

[Cheatsheet] 模块：dnf module list/info；只按题意启用/安装指定流；reset 不删除包；包组先 group list/info，安装后仍按每个服务终态验收。

[操作专题] 配置主机名、默认 target 与 tuned 基线

① [操作点] 主机名同时验证静态值和解析

```
hostnamectl set-hostname servera.example.com
hostnamectl status
getent hosts servera.example.com
```

hostnamectl 证明系统主机名；getent 证明名称解析入口。设置主机名不会自动创建 DNS 或 /etc/hosts 记录。

② [操作点] target 的当前切换与默认值分开

systemctl isolate multi-user.target 当前切换并停止不属于目标依赖的 unit，具有会话中断风险；set-default 只设置下次启动默认 target。分别用 get-default 和当前 unit 状态验证。

③ [操作点] tuned profile 需要推荐值和当前值证据

tuned-adm recommend 给出系统推荐 profile，tuned-adm active 显示当前，tuned-adm profile <NAME> 切换。推荐值不是已经应用；active 也不证明所有工作负载性能达到目标。

④ [验证点] 基础维护不替代服务验收

主机名、target 和 tuned 都是独立系统状态。题目若要求从仓库安装服务，应在这些设置之后仍验证包、unit、监听和功能，避免修改 target 停止刚配置的服务。

[Cheatsheet] 主机名 hostnamectl + getent；默认 target set-default/get-default，当前切换 isolate；tuned recommend/active/profile；基础状态与业务服务分别验收。

[诊断专题] 从仓库错误定位配置、传输、签名或依赖层

① [诊断点] repomd.xml 404 或无法下载

查看错误中的完整 URL，确认 baseurl 是否指向包含 `repodata/repomd.xml` 的根，检查变量展开、DNS、路由和代理。不要反复 clean cache 期待错误 URL 自行修复。

② [诊断点] GPG key 或签名失败

确认 key 来源与题目一致、系统时间正确、包来自期望仓库。关闭 `gpgcheck` 会消除错误但同时移除目标安全属性，不是完成任务。

③ [诊断点] 依赖冲突或包被过滤

用完整 DNF 错误、`repoquery --requires/--whatrequires` 和 `module list` 判断依赖、架构、版本流或仓库缺失。`--allowerase` 可能删除冲突包，执行前必须理解事务摘要，不作为第一尝试。

[Cheatsheet] 404 查 baseurl/repodata；连接错查 DNS/路由/代理；GPG 错查 key/时间/来源；依赖错查 `repoquery/module`；任何放宽选项前先读事务影响。

[经典任务] 从指定仓库安装并验证 Web 服务

配置两个题目给出的 RHEL 9 仓库 `exam-baseos` 与 `exam-appstream`，保持包签名校验。只从这些仓库确认并安装 `httpd`，使服务当前运行且开机启动。主机名设为 `web1.example.com`，但不得假定 DNS 已自动配置。

不得关闭 GPG 校验、使用 `rpm --nodeps` 或无条件允许删除冲突包。验收时证明 repo 文件解析、元数据可达、包来源与安装数据库、关键配置和 unit 文件、服务 active/enabled、监听和本机 HTTP 响应。

请先独立完成。参考解答从下一页开始。

[参考解答] 从仓库元数据推进到服务功能

① [操作点] 写入并刷新仓库

按题目真实 URL 与 key 写 `/etc/yum.repos.d/exam.repo`，保留 `enabled=1`、`gpgcheck=1`。随后：

```
dnf repolist --all
dnf makecache --refresh
dnf repoquery --repo exam-appstream httpd
```

② [操作点] 安装并确认来源对象

```
dnf --disablerepo='*' --enablerepo=exam-baseos,exam-appstream install httpd
rpm -q httpd
rpm -qf /usr/sbin/httpd /usr/lib/systemd/system/httpd.service
rpm -ql httpd | grep -E 'httpd\.conf$|httpd\.service$'
```

限定 repo 只在题目明确要求时使用，并确保依赖仓库同时启用。

③ [操作点] 配置系统状态与服务

```
hostnamectl set-hostname web1.example.com
httpd -t
systemctl enable --now httpd
```

④ [验证点] 完成分层验收

```
hostnamectl status
getent hosts web1.example.com
systemctl is-active httpd
systemctl is-enabled httpd
ss -lntp | grep ':80'
curl -I http://localhost/
```

若 `getent` 没有结果，只能说明名称解析尚未成立，不能撤销已经正确的静态主机名。根据题目另配 DNS 或 hosts。

[Cheatsheet] repo 文件 → `repolist` → `makecache --refresh` → 限定来源 `repoquery` → DNF 事务 → `rpm -q/qf/ql` → 应用语法 → active/enabled/监听/HTTP。

[本章收束] 包事务只是服务终态的中间层

仓库配置决定 DNF 从哪里取得元数据与包，DNF 事务决定安装数据库，RPM 查询把包映射到文件，systemd 和应用工具再验证运行状态。每一层都能局部成功，也都可能留下下一层缺口。

排错时根据错误定位 baseurl、传输、签名、模块或依赖，不用关闭保护或强制删除换取事务通过。最终从仓库、包、文件、unit、监听和功能逐层证明，才能把“已安装”推进为“可用并持久”。

第六章 网络、名称解析、SSH 与 firewalld

网络任务包含至少四类状态：NetworkManager 的持久连接配置、内核当前地址和路由、名称解析结果，以及服务监听和防火墙策略。能 ping 网关不能证明 DNS 正确，SSH 密钥存在不能证明服务端接受，runtime 防火墙放行也不能证明重载后保留。本章用逐层证据避免把连通性结论扩大。

[概念] NetworkManager device 是网卡等设备，connection 是可持久保存并激活到设备上的配置 profile。一个设备可有多个 profile，但通常只有一个处于活动状态；修改未激活 profile 不会立即改变内核网络。

[概念] IPv4 地址由地址和前缀构成，默认网关是 `0.0.0.0/0` 路由的下一跳；DNS server 与 search domain 影响名称解析。主机名、`/etc/hosts`、DNS 与 NSS 查询顺序是不同对象。

[操作语义] `nmcli device` 查询设备状态，`nmcli connection` 创建、修改、激活和查询 profile；`ip address/route` 观察内核当前状态；`getent hosts` 通过系统名称服务解析名称。

[操作语义] `ssh` 建立远程会话，`ssh-keygen` 生成密钥，`ssh-copy-id` 安装公钥；`firewall-cmd` 管理 zone 中的 runtime 或 permanent service/port 规则，reload 用持久配置重建 runtime。

[知识专题] device、connection、地址、路由与 DNS

① [知识点] profile 名称不一定等于接口名

`nmcli device status` 显示设备、类型、状态和活动 connection；`nmcli connection show` 显示所有 profile，`--active` 只显示已激活。操作时先从设备映射到活动 profile，不从 `ens160` 猜 profile 也叫 `ens160`。

```
nmcli device status
nmcli connection show --active
nmcli -f NAME,UUID,TYPE,DEVICE connection show
```

② [知识点] 持久配置与内核当前状态需交叉查询

`nmcli connection show <CON>` 显示 profile 属性，`ip -br address` 和 `ip route` 显示当前内核状态，`resolvectl` 或 `/etc/resolv.conf` 显示当前 DNS 入口。profile 中写入地址但未重新激活时，这两组证据可能不同。

③ [知识点] 前缀决定直连网络，不是地址装饰

`192.0.2.10/24` 的 `/24` 决定哪些目的地址通过本地链路到达。网关必须能通过某条路由到达；错误前缀会造成邻居或路由判断偏差。`ip route get <DEST>` 显示内核为特定目的选择的源地址、出口和下一跳，比只看 route 表更直接。

④ [验证点] 每个网络结论选择对应证据

地址: `ip -br a`; 默认路由: `ip route`; 路径选择: `ip route get`; DNS: `getent hosts`; 监听: `ss -lntup`; 远端应用: 协议客户端。ICMP 成功只能证明对应路径允许 ICMP, 不证明 TCP 端口或应用层。

[Cheatsheet] 设备/profile 映射 `nmcli device status`; 持久属性 `nmcli con show`; 当前地址/路由 `ip -br a / ip route`; 目的路径 `ip route get`; 解析 `getent hosts`。

[操作专题] 创建并激活静态 IPv4 连接

① [操作点] 先保存远程访问与当前基线

修改前记录设备、活动 profile、地址、路由和 DNS; 远程考试环境中改变正在使用的连接可能立即断开会话。确认有控制台或恢复入口, 并避免删除仍需使用的 profile。

```
nmcli device status
nmcli connection show --active
ip -br address
ip route
```

② [参数点] 创建与修改都要表达 method

创建 profile 时 `type ethernet`、`ifname`、`con-name` 明确对象; 静态 IPv4 需要 `ipv4.method manual`、地址/前缀、gateway 和 DNS。禁用 IPv6 只有题目明确要求时做, 不把它当作解决 IPv4 问题的默认步骤。

```
nmcli connection add type ethernet ifname enp1s0 con-name exam-static \
  ipv4.method manual ipv4.addresses 192.0.2.10/24 \
  ipv4.gateway 192.0.2.1 ipv4.dns '192.0.2.53 192.0.2.54'
nmcli connection modify exam-static connection.autoconnect yes
```

修改现有 profile 使用 `nmcli con mod`。多值属性前缀 `+` 表示追加, `-` 表示删除指定值; 不带前缀通常替换整个属性, 修改 DNS 等列表前先查询当前值。

③ [操作点] 激活并处理旧 profile

```
nmcli connection up exam-static          # 把 profile 应用到设备
nmcli connection show --active
```

如果另一 profile 会自动抢占设备, 先理解优先级和 `autoconnect`, 再禁用或删除旧项。 `nmcli con down` 会立即中断该连接, 远程操作必须评估风险。

④ [验证点] 配置、当前状态与功能三层

```
nmcli -f ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.dns con show exam-static
ip -br address show enp1s0
ip route
getent hosts serverb.example.com
ping -c 2 192.0.2.1
```

ping 网关失败时先查接口、前缀和路由；DNS 失败不应通过反复修改地址掩盖。持久性由 profile 与 autoconnect 证明，必要的重启验证应在恢复路径存在时进行。

[Cheatsheet] 基线 → `nmcli con add/mod` 明确 manual/address/gateway/DNS → `con up` → profile 属性 + `ip` 当前状态 + DNS/连通功能；多值属性修改前先查。

[操作专题] 配置主机名、hosts 与名称解析

① [操作点] 主机名与地址映射分开配置

`hostnamectl set-hostname <FQDN>` 设置静态主机名。 `/etc/hosts` 条目典型顺序为地址、规范名、别名：

```
192.0.2.10 servera.example.com servera
```

修改 hosts 前检查是否已有同名或同地址冲突。它是本地解析来源，不会发布到 DNS，也不会自动改变远端客户端。

② [知识点] 查询工具走的解析路径不同

`getent hosts <NAME>` 走 NSS 配置，适合验证应用常用的系统解析；`dig / host` 直接查询 DNS，不会同样反映 `/etc/hosts`。`ping` 混合了名称解析和 ICMP，失败时不够精确。

③ [验证点] 分别证明短名、FQDN 与反向需求

```
hostnamectl status
getent hosts servera.example.com
getent hosts servera
```

正向解析正确不自动证明反向 PTR；只有题目或服务明确要求反向解析时才验证。`hostname -f` 的结果依赖主机名和解析配置，应作为组合结果而非唯一真相。

[Cheatsheet] 主机名 `hostnamectl`；本地映射 `/etc/hosts`；系统解析 `getent hosts`；DNS 专查 `dig/host`；短名、FQDN、反向解析按题意分别验收。

[操作专题] 建立 SSH 密钥认证并验证服务端边界

① [知识点] 私钥留在客户端，公钥进入服务端账号

`ssh-keygen` 生成密钥对，私钥必须受保护，`.pub` 才可分发。`ssh-copy-id user@host` 把公钥追加到远端用户的 `~/.ssh/authorized_keys`，但执行时通常仍需要一次现有认证。

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519
ssh-copy-id -i ~/.ssh/id_ed25519.pub alice@serverb
ssh -i ~/.ssh/id_ed25519 alice@serverb
```

② [知识点] 客户端与服务端配置分别影响连接

客户端 `~/.ssh/config` 可设置 `Host`、`Hostname`、`User`、`IdentityFile`；服务端 `/etc/ssh/sshd_config` 与 `drop-in` 决定监听、认证和账号限制。修改服务端前用 `sshd -t` 校验，再 `reload`；不要在尚未验证密钥时关闭口令认证，避免锁死访问。

权限通常要求 `~/.ssh` 仅用户可写、`authorized_keys` 不允许其他人写。SELinux 上下文错误可用 `restorecon -Rv ~/.ssh` 恢复默认规则，不能只靠 `chmod` 解释所有拒绝。

③ [知识点] `known_hosts` 验证服务器身份

首次连接记录主机密钥。主机密钥改变可能是系统重装或攻击，不能无条件删除条目继续；先核对可信指纹，再使用 `ssh-keygen -R <HOST>` 移除旧记录并重新接受。

④ [验证点] 用详细输出定位连接层

`ssh -v` 显示配置选择、密钥尝试和认证结果；服务端查 `journalctl -u sshd`。成功登录后运行 `id` 和 `hostname` 证明实际远端身份，不能只看到连接命令退出 0。

[Cheatsheet] 生成 `ssh-keygen` → 公钥 `ssh-copy-id` → `ssh -i`；服务端改前 `sshd -t`；失败用客户端 `-v` + 服务端 `journal`；主机密钥变化先核指纹。

[操作专题] 管理 firewalld zone、service 与端口

① [知识点] zone 通过接口或源选择规则集合

先查 active zones 和接口归属：

```
firewall-cmd --get-active-zones
firewall-cmd --get-default-zone
firewall-cmd --zone=public --list-all
```

默认 zone 只用于没有显式绑定的连接/源。向错误 zone 添加规则会显示成功却不影响实际流量，因此每次修改显式写 zone 更清晰。

② [操作点] service 与 port 表达层次不同

service 是 `/usr/lib/firewalld/services` 或 `/etc/firewalld/services` 中定义的一组端口/协议；port 是直接的 `PORT/PROTOCOL`。已存在准确 service 时优先 service；非标准端口或无 service 定义时使用 port。

```
firewall-cmd --zone=public --add-service=http
firewall-cmd --zone=public --add-service=http --permanent
firewall-cmd --zone=public --add-port=8080/tcp --permanent
```

不带 `--permanent` 修改 runtime，立即生效但 reload 后消失；带 permanent 只写持久配置，不立即改变 runtime。可以分别执行两条，或先写 permanent 后 reload，但 reload 会用全部持久配置替换 runtime，可能丢弃其他未持久变化。

③ [验证点] runtime、permanent、监听和外部访问

```
firewall-cmd --zone=public --query-service=http
firewall-cmd --zone=public --query-service=http --permanent
ss -lntp | grep ':80'
curl -I http://servera.example.com/
```

防火墙规则只允许流量，不创建监听。localhost 访问可能不经过与远端相同的 zone/路径，外部访问验证应从题目指定客户端执行。

[Cheatsheet] 先查 active zone；优先 service，非标准用 `port/protocol`；runtime 与 permanent 分别查询；reload 会重建 runtime；最终仍查监听和外部协议访问。

[诊断专题] 按链路定位地址、路由、解析、监听或过滤失败

① [诊断点] 地址正确但目标不可达

用 `ip route get <DEST>` 看出口、源地址和下一跳，检查前缀、默认路由和邻居；不要先改 DNS，因为数字 IP 路径尚未成立。

② [诊断点] IP 可达但名称失败

用 `getent hosts` 区分系统解析，查 `DNS server/search` 和 `/etc/hosts` 冲突；`dig` 可进一步检查 DNS。不要把 IP 写入应用配置绕过题目要求的名称解析。

③ [诊断点] 服务 active 但远端连接拒绝或超时

拒绝通常优先查是否监听目标地址/端口；超时再查路由、防火墙和中间路径，但两者不能绝对等同根因。依次验证 `ss`、本机协议请求、firewalld zone/runtime/permanent、远端请求和 SELinux 端口类型。

④ [诊断点] SSH 公钥被拒绝

客户端 `ssh -v` 看是否选择并提供目标 key，服务端 journal 看拒绝原因；检查远端用户、家目录、`.ssh/authorized_keys` 所有者权限和 SELinux 标签，再检查 sshd 的有效配置。

[Cheatsheet] 数字 IP 不通查地址/路由；IP 通名称不通查 NSS/DNS；服务问题查监听→本机→防火墙→远端→SELinux；SSH 用双端日志与身份/权限证据。

[经典任务] 配置静态网络、SSH 密钥与 Web 访问

将接口 `enp1s0` 配置为持久静态地址 `192.0.2.10/24`，网关 `192.0.2.1`，DNS `192.0.2.53`，profile 名为 `exam-static` 且开机自动连接。主机名为 `servera.example.com`，本机能解析 `serverb.example.com` 到题目地址。

为用户 `alice` 建立到 `serverb` 的 `ed25519` 密钥认证，不泄露私钥。使 `servera` 的 `httpd` 可从外部通过标准 HTTP service 访问，`firewalld` 规则当前和持久配置都正确。

不得删除未确认用途的连接 profile，不得在密钥验证前关闭口令认证。验收包括 profile、当前地址/路由/DNS、主机名解析、SSH 实际远端身份、`httpd` active/enabled/监听、正确 zone 的 runtime/permanent 以及外部 HTTP 请求。

请先独立完成。参考解答从下一页开始。

[参考解答] 从 profile 激活推进到外部服务链

① [操作点] 记录基线并创建连接

```
nmcli device status
nmcli connection show --active
ip -br address; ip route
nmcli con add type ethernet ifname enp1s0 con-name exam-static \
  ipv4.method manual ipv4.addresses 192.0.2.10/24 \
  ipv4.gateway 192.0.2.1 ipv4.dns 192.0.2.53 \
  connection.autoconnect yes
nmcli con up exam-static
```

② [操作点] 配置名称和密钥

```
hostnamectl set-hostname servera.example.com
ssh-keygen -t ed25519 -f /home/alice/.ssh/id_ed25519
ssh-copy-id -i /home/alice/.ssh/id_ed25519.pub alice@serverb.example.com
```

按题目真实地址维护 serverb 的解析，并确保密钥文件由 alice 所有；不要把私钥复制到远端。

③ [操作点] 开放正确 zone 的 HTTP

```
systemctl enable --now httpd
firewall-cmd --get-active-zones
firewall-cmd --zone=public --add-service=http
firewall-cmd --zone=public --add-service=http --permanent
```

若活动接口不在 public，应把命令中的 zone 换成真实归属，而不是把接口随意移动到 public。

④ [验证点] 完成多层验收

```
nmcli con show exam-static
ip -br address show enp1s0; ip route
getent hosts servera.example.com serverb.example.com
sudo -u alice ssh serverb.example.com 'hostname; id'
systemctl is-active httpd; systemctl is-enabled httpd
ss -lntp | grep ':80'
firewall-cmd --zone=public --query-service=http
firewall-cmd --zone=public --query-service=http --permanent
```

最后从外部客户端执行 HTTP 请求。任何失败都停在对应层调查，不回滚已经正确的 profile 或密钥。

[Cheatsheet] 基线 → 静态 profile + `con up` → 地址/路由/DNS → 主机名/hosts → 密钥双端验证 → 服务监听 → 正确 zone runtime/permanent → 外部 HTTP。

[本章收束] 连通性是多层证据的交集

持久 profile、内核当前网络、名称解析、服务监听、SSH 认证和 firewalld 各自回答不同问题。修改 connection 后必须激活并核对当前状态；修改防火墙时必须确认实际 zone，并分别检查 runtime 与 permanent。

排错先用数字 IP 验证路由，再进入名称解析；服务从监听、本机访问推进到过滤和外部访问；SSH 同时读取客户端详细输出和服务端日志。只要每一层都用对应工具证明，网络题就不会被一个 ping 或 active 状态过早结束。

第七章 分区、文件系统、Swap 与持久挂载

存储题的危险不在命令复杂，而在设备名相似且写操作不可逆。磁盘存在不代表空闲，分区存在不代表有文件系统，目录存在不代表已挂载，当前 Swap 可用也不代表重启后启用。本章从设备证据开始，沿分区、签名、文件系统、挂载与持久配置逐层验收。

[概念] 块设备可承载分区表、分区、LVM 元数据、文件系统或 Swap 签名。GPT 与 MBR 描述分区布局；文件系统在块设备上组织文件；挂载把文件系统连接到统一目录树。

[概念] UUID 属于文件系统或 Swap 签名，不是磁盘永久属性。重新格式化会生成新 UUID；`/etc/fstab` 使用旧 UUID 时，配置与当前对象会脱节。

[操作语义] `lsblk` 展示块设备拓扑，`blkid` 读取签名，`wipefs` 默认列出现有签名；`parted` / `fdisk` 修改分区表，`partprobe` 请求内核重新读取分区。

[操作语义] `mkfs.xfs` / `mkfs.ext4` 新建文件系统，`mount` 建立当前挂载，`findmnt` 查询挂载关系；`mkswap` 创建 Swap 签名，`swapon` / `swapoff` 改变当前启用状态。

[知识专题] 设备、分区、签名、文件系统与挂载层级

① [知识点] 六层状态不能互相替代

典型链路是“磁盘 → 分区 → 文件系统/Swap 签名 → 当前挂载或 swapon → fstab 持久条目”。每层都有独立查询：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS
blkid
findmnt
swapon --show
```

`lsblk` 可把多层信息放在一张表中，但仍需按列解释。FSTYPE 为空不充分证明安全空闲，设备可能包含未识别数据或分区表；写操作前还要核对已有挂载、LVM/RAID 归属和题意。

② [知识点] GPT 与 MBR 的考试边界

GPT 支持更多分区和大容量设备，使用 GUID 类型；MBR 具有主/扩展/逻辑分区等旧限制。RHEL 9 新磁盘通常优先 GPT，除非题目指定 MBR 兼容需求。选择分区表会重写磁盘布局，不是在已有数据盘上随意切换的显示选项。

③ [知识点] XFS 与 ext4 的增长/缩小边界

XFS 支持在线增长但不支持原地缩小；ext4 可增长，并在卸载等条件下缩小。创建文件系统的 `mkfs` 会重建结构，不是修复或扩容命令。题目只要求创建时按指定类型执行，不因个人偏好替换。

④ [验证点] 设备身份必须来自多条一致证据

用设备 PATH、容量、拓扑、型号/序列（可用时）、签名、挂载和上层归属共同判断。`/dev/sdb`、`/dev/vdb` 等枚举名可能随环境变化；操作前重新运行查询，不能依赖上一次考试练习中的设备名。

[Cheatsheet] 拓扑 `lsblk`；签名 `blkid/wipefs`；当前挂载 `findmnt`；Swap `swapon --show`；磁盘存在 ≠ 空闲；`mkfs/mkswap` 都是写签名操作。

[操作专题] 在确认空闲的磁盘上创建 GPT 分区

① [操作点] 写入前建立设备基线

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS
blkid /dev/vdb /dev/vdb1
wipefs /dev/vdb                # 只列签名，不加 -a
pvs; vgs; lvs                  # 排除 LVM 归属
```

若发现文件系统、分区、挂载或现有 PV，停止并核对题意。`wipefs -a` 会擦除签名，不是调查命令。

② [参数点] parted 的单位和边界必须明确

`parted -s` 非交互执行，`mklabel gpt` 创建 GPT，`mkpart` 指定名称/类型和起止位置。使用 MiB/GiB 避免十进制与二进制含糊，并预留开头对齐空间。

```
parted -s /dev/vdb mklabel gpt          # 新建 GPT
parted -s /dev/vdb mkpart data xfs 1MiB 2049MiB # 创建约 2 GiB 分区
parted -s /dev/vdb print
partprobe /dev/vdb                      # 请求内核重读
udevadm settle                          # 等待设备节点事件
```

`parted` 中的 `xfs` 参数主要设置预期/类型提示，不会创建 XFS；文件系统仍需独立 `mkfs.xfs`。

③ [验证点] 分区表与内核设备视图一致

```
parted /dev/vdb unit MiB print
lsblk -o NAME,START,SIZE,TYPE,FSTYPE /dev/vdb
```

如果新分区节点未出现，先检查内核是否仍在用旧布局和命令错误，不要重复 `mklabel`。生产环境可能需要重启才能安全重读被占用磁盘，但考试新盘应先确认没有使用者。

[Cheatsheet] 查询签名/归属 → `mklabel gpt` → `mkpart ... 1MiB <END>` → `partprobe / udevadm settle` → `parted print` + `lsblk`；`mkpart` 不创建文件系统。

[操作专题] 创建文件系统并用 UUID 持久挂载

① [操作点] 只在新分区上创建指定文件系统

`mkfs.xfs` 与 `mkfs.ext4` 会覆盖现有文件系统结构。确认 `/dev/vdb1` 是新建空分区后执行：

```
mkfs.xfs -L DATA /dev/vdb1          # 新建 XFS 并设置 label
mkdir -p /srv/data
blkid /dev/vdb1                      # 获取真实 UUID、TYPE、LABEL
```

label 可读但不一定全局唯一；UUID 更适合 `fstab`。UUID 必须从当前对象查询，不能复制示例。

② [知识点] `fstab` 六字段分别表达对象和策略

六字段是设备标识、挂载点、类型、选项、dump、fsck 顺序。XFS 常用末两项 `0 0`；ext4 根外文件系统常见 fsck pass 为 2，但应服从题意和系统惯例。

```
UUID=<真实 UUID> /srv/data xfs defaults 0 0
```

`nofail` 会改变启动失败处理，不应为了掩盖错误 UUID 默认添加。挂载选项如 `ro`、`noexec`、`nodev`、`nosuid` 会改变功能边界，只有目标明确时设置。

③ [操作点] 在重启前验证配置并实际挂载

```
systemctl daemon-reload          # 重读 fstab 生成的 mount units
findmnt --verify                  # 检查明显语法/引用问题
mount -a                          # 实际尝试未挂载条目
```

`findmnt --verify` 不等于实际挂载，`mount -a` 无报错也不证明挂载源就是目标分区。继续查询目标目录。

④ [验证点] 当前、容量、UUID 与写入功能

```
findmnt /srv/data
df -hT /srv/data
lsblk -f /dev/vdb
touch /srv/data/.write-test && rm /srv/data/.write-test
```

`findmnt` 证明挂载源/类型/选项，`df` 证明文件系统视角容量，`lsblk -f` 交叉检查 UUID。写入测试还受目录权限和只读选项影响。

[Cheatsheet] `mkfs.<TYPE>` → `blkid` 取 UUID → 写 `fstab` → `daemon-reload` → `findmnt --verify` → `mount -a` → `findmnt` / `df` / `lsblk` / 功能测试。

[操作专题] 创建并持久启用 Swap

① [知识点] Swap 签名、当前启用和持久条目是三层

分区创建后，`mkswap` 写入 Swap 签名并产生 UUID；`swapon` 加入当前 Swap；`fstab` 决定启动时启用。任一层成功不代表其他层完成。

② [操作点] 创建 Swap 分区并启用

假设 `/dev/vdb2` 已由分区步骤创建且确认空闲：

```
mkswap -L EXAMSWAP /dev/vdb2      # 创建 Swap 签名
blkid /dev/vdb2                   # 获取真实 UUID
swapon /dev/vdb2                  # 当前启用
```

`fstab` 条目：

```
UUID=<真实 UUID> none swap defaults 0 0
```

③ [验证点] 签名、当前状态和持久配置

```
swapon --show --output=NAME,TYPE,SIZE,USED,PRI0
cat /proc/swaps
findmnt --verify
```

`free -h` 显示 Swap 总量和使用量，但不直接说明由哪个设备提供；`swapon --show` 更适合设备验收。Swap 未被使用不表示未启用。

④ [边界] 优先级与删除顺序

多个 Swap 可设 `pri=`，数值较高者优先。删除 Swap 前先 `swapoff` 并确认内存能够容纳迁移，再移除 `fstab` 和签名/分区；这不属于创建题的通用清理流程。

[Cheatsheet] `mkswap` 写签名 → `blkid` → `swapon` 当前启用 → `fstab none swap defaults 0 0` → `swapon --show` + `findmnt --verify`；`USED=0` 仍可正常启用。

[知识专题] mount 选项、systemd units 与卸载边界

① [知识点] fstab 会被 systemd 转换为 mount unit

路径 `/srv/data` 对应 unit 名通常为 `srv-data.mount`，可用 `systemd-escape --path --suffix=mount /srv/data` 计算。`systemctl status srv-data.mount` 和 `journal` 可提供启动挂载失败证据。

② [知识点] 卸载失败说明对象仍被使用

`umount /srv/data` 报 `target is busy` 时，用 `findmnt` 确认嵌套挂载，用 `fuser -vm` 或 `lsof` 查打开文件与工作目录。不要先使用 `lazy/force` 卸载；应让进程释放对象并离开目录。

③ [边界] 挂载会遮蔽目录原有内容

挂载到非空目录后，原目录内容被挂载文件系统遮蔽但没有删除。卸载后会重新出现。题目要求保留数据时，在挂载前检查目录是否为空；不要把“挂载后文件消失”误判为数据被删除。

[Cheatsheet] `mount unit` 用 `systemd-escape` 计算；`busy` 查嵌套挂载和 `fuser -vm`；挂载遮蔽原目录内容；不以 `force/lazy` 作为第一修复。

[诊断专题] 根据症状定位分区、签名、挂载或持久层

① [诊断点] 目录存在但数据写入根文件系统

`findmnt /srv/data` 无结果表示当前未挂载；检查 `fstab`、`findmnt --verify`、`mount -a` 和 `mount unit journal`。不要重新 `mkfs`。

② [诊断点] `fstab` 报 `UUID` 不存在

用 `blkid / lsblk -f` 找当前 `UUID`，检查是否复制错误或重新格式化后 `UUID` 已变。修正条目后重新 `verify/mount`；不要再格式化以“得到新 `UUID`”。

③ [诊断点] 分区已创建但内核看不到

核对 `parted print` 是否真的写入，运行 `partprobe / udevadm settle`，检查设备是否被使用导致重读失败。不要重复创建同一分区。

④ [诊断点] `Swap` 签名存在但总量未增加

`blkid` 只证明签名；检查 `swapon --show` 和 `swapon <DEV>` 的错误，再检查 `fstab` 持久层。

[Cheatsheet] 目录无挂载查 `fstab/mount unit`；`UUID` 错查当前签名；分区节点缺失查内核重读；`Swap` 不见查 `swapon --show`；任何层失败都不重建上层对象。

[经典任务] 在新磁盘上创建 XFS 与 Swap 并持久启用

服务器提供一块确认用于本题但当前状态未知的 `/dev/vdb`。在确认没有需要保留的数据后，使用 GPT 创建约 2 GiB 的分区用于 XFS，持久挂载到 `/srv/data`；再创建约 1 GiB 的 Swap 分区并当前启用、重启后保持。

XFS 挂载必须使用真实 UUID，目录原有内容如存在必须先报告并保留。不得使用 `wipefs -a`、强制格式化或重启代替调查。验收包括分区边界、签名、UUID、当前挂载、文件系统容量、写入功能、Swap 当前设备和 `fstab` 静态检查。

请先独立完成。参考解答从下一页开始。

[参考解答] 从设备证据到两类持久状态

① [操作点] 调查并创建分区

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,UUID,MOUNTPOINTS
blkid /dev/vdb /dev/vdb1 /dev/vdb2
wipefs /dev/vdb
pvs
parted -s /dev/vdb mklabel gpt
parted -s /dev/vdb mkpart data xfs 1MiB 2049MiB
parted -s /dev/vdb mkpart swap linux-swap 2049MiB 3073MiB
partprobe /dev/vdb; udevadm settle
parted /dev/vdb unit MiB print
```

只有调查确认可写后才执行 mklabel。

② [操作点] 创建 XFS 与 Swap 签名

```
test ! -e /srv/data || find /srv/data -mindepth 1 -maxdepth 1 -print
mkfs.xfs -L DATA /dev/vdb1
mkswap -L EXAMSWAP /dev/vdb2
mkdir -p /srv/data
blkid /dev/vdb1 /dev/vdb2
```

将两个真实 UUID 写入 fstab 的 XFS 和 Swap 条目。

③ [操作点] 当前启用并验证持久配置

```
systemctl daemon-reload
findmnt --verify
mount -a
swapon -a
```

④ [验证点] 分层验收

```
lsblk -f /dev/vdb
findmnt /srv/data
df -hT /srv/data
touch /srv/data/.check && rm /srv/data/.check
swapon --show --output=NAME,TYPE,SIZE,USED,PRI0
findmnt --verify
```

[Cheatsheet] 设备/签名/归属调查 → GPT 两分区 → 内核重读 → `mkfs.xfs` / `mkswap` → 双 UUID 写 fstab → `mount -a` / `swapon -a` → 文件系统与 Swap 分层验收。

[本章收束] 每个写操作只推进一个存储层

分区工具改变布局，mkfs/mkswap 写签名，mount/swap on 改变当前状态，fstab 描述启动持久性。目录、分区或 UUID 的存在都只能证明自己的层，不能自动代表应用可用。

稳定路径始终从设备身份和已有数据开始，写入后立即用对应查询确认。错误 UUID、未重读分区表或未启用 Swap 都有局部修复入口，不需要重新格式化。把破坏性命令留在证据充分的创建阶段，存储题才能既完成目标又保留数据边界。

第八章 管理 LVM 逻辑存储

LVM 把底层磁盘容量变成可分配、可扩展的逻辑块设备，但应用最终使用的仍然是挂载到目录树中的文件系统。做题时若把“磁盘、PV、VG、LV、文件系统、挂载点”混成一个对象，就容易出现局部命令成功、最终任务却没有完成的情况。本章按对象层级、典型操作和分层验收组织内容，所有命令都要回答一个明确问题。

[概念] 物理卷 (Physical Volume, PV)、卷组 (Volume Group, VG) 与逻辑卷 (Logical Volume, LV) 构成 LVM 的三层容量对象。PV 把设备纳入 LVM；VG 汇聚一个或多个 PV，形成容量池；LV 再从 VG 中获得一段可供上层使用的逻辑块设备。

[概念] 物理区域 (Physical Extent, PE) 是 VG 的固定容量单位，逻辑区域 (Logical Extent, LE) 是 LV 的分配单位。在线性 LV 中，一个 LE 通常映射一个 PE，因此 $PE \text{ 大小} \times LE \text{ 数量}$ 可以得到 LV 的标称容量。

[概念] 文件系统与挂载点位于 LVM 之上。LV 已创建，只能证明块设备存在；创建 XFS 或 ext4 后，块设备才可以组织文件；完成挂载后，目录才真正指向该文件系统；写入 `/etc/fstab` 并验证后，挂载关系才具备启动持久性。

[操作语义] `pvs`、`vgs`、`lvs` 分别读取 PV、VG、LV 元数据；`pvcreeate`、`vgcreate`、`lvcreate` 则依次建立这三层对象。查询命令回答“现在是什么状态”，创建命令才改变对象。

[操作语义] `mkfs.xfs` 在块设备上新建 XFS，`mount` 建立当前挂载，`/etc/fstab` 描述持久挂载。`lvextend --resizefs` 同时扩展 LV 与受支持的文件系统，因此验收时必须同时查看 LVM 层和文件系统层。

[知识专题] 对象层级、容量单位与查询边界

① [知识点] 六层对象不能互相替代

从一个空闲分区到应用可用目录，完整链路是：设备或分区进入 PV，PV 加入 VG，VG 划出 LV，LV 上创建文件系统，文件系统再挂载到目录。每一层都有独立状态，也有独立的查询入口。



这意味着以下结论都不能成立：设备存在不等于设备空闲；LV 存在不等于文件系统存在；目录存在不等于已经挂载；当前挂载成功不等于重启后仍会挂载。一个完整任务至少要跨越 LVM、文件系统与挂载三个层次进行验收。

② [知识点] PE、LE 与两种容量表达

PE 大小属于 VG。若 `projectvg` 的 PE 为 16 MiB，那么创建 50 个 LE 的线性 LV，其标称容量为 800 MiB：

$$50 \times 16 \text{ MiB} = 800 \text{ MiB}$$

`lvcreate` 和 `lvextend` 中，小写 `-l` 按 extent 数量或百分比表达，大写 `-L` 按容量表达。题目要求“50 个 extent”时应使用 `-l 50`；题目要求“800 MiB”时使用 `-L 800M`。参数并非两种随意替换的写法，而是在精确表达不同题意。

```
lvcreate -l 50 -n projectlv projectvg    # 分配 50 个 extent
lvcreate -L 800M -n projectlv projectvg  # 按容量分配 800 MiB
```

扩容命令还要区分目标总量和增量。`-l 70` 表示最终达到 70 个 extent，`-l +20` 表示在当前基础上增加 20 个 extent；加号不是装饰字符，而是改变操作语义的关键部分。

③ [操作点] 用一组查询建立对象映射

查询时最重要的参数是 `-o`，它让命令只显示本次判断需要的字段。`pvs` 重点看 PV 属于哪个 VG，`vgs` 重点看 PE 和空闲容量，`lvs` 重点看 LV 路径与底层设备；`findmnt` 与 `df` 则进入挂载和文件系统层。

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS      # 块设备、文件系统与挂载点
pvs -o pv_name,vg_name,pv_size,pv_free          # PV 归属与剩余容量
vgs -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count # VG、PE 与空闲 extent
lvs -o lv_name,vg_name,lv_size,lv_path,devices   # LV 路径、容量与底层映射
findmnt /srv/project                             # 目录当前由哪个设备提供
df -hT /srv/project                             # 文件系统类型、容量与使用率
```

`lvs` 与 `df` 的容量略有差异通常是正常的：前者显示块设备标称容量，后者显示文件系统视角的容量与可用空间。真正需要警惕的是扩容后 `lvs` 已明显增长，而 `df` 完全没有变化，这通常意味着只完成了 LV 层。

[Cheatsheet] 设备层看 `lsblk`；PV/VG/LV 分别看 `pvs`、`vgs`、`lvs`；目录挂载源看 `findmnt`；文件系统容量看 `df -hT`。先判断对象层级，再选择命令。

[操作专题] 从空闲设备建立 PV、VG 与 LV

① [操作点] 证明候选设备可以安全使用

`pvccreate` 会写入 LVM 标签和元数据，`mkfs` 会重建文件系统，因此任何写操作之前都必须重新确认设备身份。这里不需要把所有查询命令都执行一遍，但至少要是能回答：设备是否存在、是否已有文件系统、是否挂载、是否已经属于某个 VG。

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS      # 确认设备结构与已有文件系统
pvs -o pv_name,vg_name,pv_size,pv_free          # 确认设备是否已经成为 PV
```

如果 `/dev/vdb1` 已出现文件系统类型、挂载点或现有 VG，操作应停止并重新核对题意。不能用强制选项绕过证据，也不能因为设备名“看起来像数据盘”就直接初始化。

② [操作点] 初始化 PV 并创建指定 PE 的 VG

这一阶段只需抓住 `vgcreate` 的一个重要参数：`-s` (physical extent size) 设置 VG 的 PE 大小。它是 VG 属性，不属于 `pvccreate` 或 `lvcreate`。

```
pvccreate /dev/vdb1                                # 把分区初始化为 PV
vgcreate -s 16M projectvg /dev/vdb1                # 创建 VG，并把 PE 设为 16 MiB
```

创建后立即检查成员关系和关键属性。`vg_extent_size` 应为 16 MiB，`vg_free_count` 表示尚可分配多少个 extent。

```
pvs -o pv_name,vg_name,pv_size,pv_free            # /dev/vdb1 应属于 projectvg
vgs projectvg -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count # 验证 PE 与余量
```

③ [操作点] 按题目要求创建 LV

`lvcreate` 中最常用的三个参数是：`-n` 指定 LV 名称，`-l` 按 extent 数量或百分比分配，`-L` 按容量分配。题目按 extent 描述时，不要先换算成近似容量再使用 `-L`。

```
lvcreate -l 50 -n projectlv projectvg          # 创建 50 个 extent 的 projectlv
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path,devices # 查询真实 LV 路径
```

若题目要求使用 VG 全部剩余空间，典型写法为 `-l 100%FREE`；只有题意明确要求占用全部剩余 extent 时才使用它，不能把它当成通用省事写法。

④ [验证点] 三层对象分别成立

PV、VG、LV 的创建结果应由各自查询命令证明。只看到最后一条 `lvcreate` 返回成功，不足以证明 PE 大小、设备归属和 LV 路径都符合要求。

```
pvs -o pv_name,vg_name,pv_size,pv_free
vgs projectvg -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path,devices
```

[Cheatsheet] `pvccreate <DEV>` 初始化 PV；`vgcreate -s <PE> <VG> <PV>` 创建指定 PE 的 VG；`lvcreate -l <LE> -n <LV> <VG>` 按 extent 创建 LV；按容量则把 `-l` 换成 `-L <SIZE>`。

[操作专题] 创建文件系统并建立持久挂载

① [操作点] 在新 LV 上创建 XFS

`mkfs.xfs` 的语义是“新建文件系统”，不是“让已有文件系统变大”。因此它只能作用于确认无需保留数据的新 LV。设备路径应优先来自 `lvs ... -o lv_path` 的查询结果。

```
mkfs.xfs /dev/projectvg/projectlv          # 在新 LV 上建立 XFS
mkdir -p /srv/project                      # 创建挂载点目录
blkid /dev/projectvg/projectlv             # 读取文件系统 UUID
```

若 LV 上已经存在需要保留的数据，重新执行 `mkfs.xfs` 会破坏原文件系统。扩容现有 XFS 时应使用 `lvextend --resizefs` 或 `xfs_growfs`，不能再次格式化。

② [操作点] 使用 UUID 写入 `/etc/fstab`

`/etc/fstab` 的六个字段依次是：设备标识、挂载点、文件系统类型、挂载选项、dump 标记、fsck 顺序。这里最重要的是：UUID 必须来自当前机器的 `blkid` 或 `lsblk -f`，不能复制示例值。

```
UUID=<真实 UUID> /srv/project xfs defaults 0 0
```

使用 UUID 的目的不是让配置显得更复杂，而是让挂载关系不依赖可能变化的 `/dev/...` 枚举顺序。

③ [操作点] 在重启前验证并执行挂载

`systemctl daemon-reload` 让 `systemd` 重新加载由 `fstab` 生成的 mount units；`findmnt --verify` 检查可解析性；`mount -a` 在当前会话中实际尝试挂载尚未挂载的条目。三者关注点不同，不能只执行其中一个就宣布完成。

```
systemctl daemon-reload      # 重新加载 fstab 对应的 mount units
findmnt --verify             # 检查 fstab 的明显语法与引用问题
mount -a                     # 在当前会话中实际尝试挂载
```

④ [验证点] 目录确实由目标 LV 提供

```
findmnt /srv/project          # 核对挂载源与文件系统类型
df -hT /srv/project           # 核对容量、类型与使用率
lsblk -f                      # 从块设备视角核对 UUID 和挂载点
```

目录存在没有证明力。若 `findmnt /srv/project` 没有结果，向该目录写入数据时，很可能实际写入根文件系统。

[Cheatsheet] 新 LV: `mkfs.xfs <LV_PATH>`；取 UUID: `blkid <LV_PATH>`；持久挂载：写 `fstab` 后执行 `daemon-reload`、`findmnt --verify`、`mount -a`；最终用 `findmnt` 和 `df -hT` 验收。

[知识专题] 扩容的容量来源、增量语义与文件系统边界

① [知识点] 扩容是一条容量链，不是一条命令

目录空间不足时，容量必须沿着 `PV → VG → LV → 文件系统 → 挂载点` 向上提供。VG 没有空闲 extent，LV 就无法继续增长；LV 已增长而文件系统未增长，目录也不会获得新增空间。因此扩容前至少要查看 `vg_free` 或 `vg_free_count`，扩容后至少要比对 `lvs` 与 `df -hT`。

② [知识点] 目标值与增量值必须区分

`lvextend` 沿用 `-l` 和 `-L` 两套表达：小写按 extent，大写按容量；带 `+` 表示增加量，不带 `+` 表示目标总量。

```
-l 70          最终达到 70 个 extent
-l +20         在当前基础上增加 20 个 extent
-L 1120M       最终达到约 1120 MiB
-L +320M       在当前基础上增加约 320 MiB
```

VG 的 PE 为 16 MiB 时，增加 20 个 extent 等于增加 320 MiB。计算题的价值在于帮助检查参数是否表达了题目要求，而不是强迫把所有 extent 题都转换成 MiB。

③ [操作语义] `lvextend` 与 `xfs_growfs`

`lvextend --resizefs` 中, `--extents` 或 `-l` 指定 extent 目标, `--size` 或 `-L` 指定容量目标, `--resizefs` 或 `-r` 在扩大 LV 后继续调整受支持的文件系统。

```
lvextend --extents +20 --resizefs projectvg/projectlv # LV 与文件系统一起增加 20 PE
lvextend -L +320M -r projectvg/projectlv             # 按容量增量完成同一类操作
```

若已经单独扩大 LV, XFS 的文件系统扩容入口是已挂载目录:

```
xfs_growfs /srv/project # 扩展挂载在该目录的 XFS
```

XFS 支持增长, 但不支持原地缩小。因此 `lvreduce` 不能被理解为 `lvextend` 的安全反向命令; 先缩小 LV 会直接截断仍被文件系统使用的空间。

[Cheatsheet] 扩容前看 `vgs`; 按 extent 用 `-l`, 按容量用 `-L`; `+` 表示增量; 优先使用 `lvextend -r` 同步扩展文件系统; XFS 可扩不可缩。

[操作专题] 在保留数据的前提下扩展 LV 与 XFS

① [操作点] 建立扩容前基线

先确认挂载点对应的 LV、文件系统类型和 VG 余量。`vg_free_count` 直接表示还剩多少个 extent, 适合处理“增加 N 个 extent”的题目。

```
findmnt /srv/project # 挂载点与源设备
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path    # 当前 LV 大小
vgs projectvg -o vg_name,vg_free,vg_extent_size,vg_free_count # PE 大小与空闲 PE
df -hT /srv/project    # 当前文件系统容量
```

需要证明数据被保留时, 应在扩容前建立可复核的证据, 而不是扩容后凭感觉判断。

```
printf 'project-data\n' > /srv/project/marker.txt # 建立需要保留的数据
sha256sum /srv/project/marker.txt                 # 记录扩容前校验值
```

② [操作点] VG 余量足够时直接扩展

这一条命令只需抓住两个参数: `--extents +20` 表示增加 20 个 extent, `--resizefs` 表示同步调整文件系统。

```
lvextend --extents +20 --resizefs projectvg/projectlv # 增加 LV, 并让 XFS 使用新增空间
```

③ [操作点] VG 余量不足时先补充容量池

新设备不能直接成为 `lvextend` 的参数。它必须先经过 `pvccreate`，再通过 `vgextend` 加入目标 VG。
`vgextend` 的参数顺序是“目标 VG + 新 PV”。

```
pvccreate /dev/vdc1 # 把新分区初始化为 PV
vgextend projectvg /dev/vdc1 # 把 PV 加入现有 VG
vgs projectvg -o vg_name,vg_size,vg_free,vg_free_count # 重新确认空闲 PE
```

只有 `vg_free_count` 已满足要求后，才继续执行相同的 `lvextend`。反复重试空间不足的扩容命令不会自动发现新磁盘。

④ [验证点] LV、文件系统、挂载与数据同时正确

```
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path # LV 已增长
df -hT /srv/project # XFS 已使用新增空间
findmnt /srv/project # 挂载关系没有改变
sha256sum /srv/project/marker.txt # 原数据校验值保持一致
```

`lvs` 变大只能证明底层块设备增长；`df` 变大才说明文件系统使用了新增空间；校验值一致则证明任务没有通过重新格式化获得结果。

[Cheatsheet] 查余量：`vgs ... vg_free_count`；直接扩容：`lvextend -l +N -r <VG/LV>`；空间不足：`pvccreate <DEV> → vgextend <VG> <PV> → 再扩容`；验收：`lvs + df + findmnt + 数据校验`。

[诊断专题] 根据症状定位 LVM 失败所在层

① [诊断点] `lvs` 已增长，`df` 没变化

这说明 LV 层已经完成，但文件系统层仍停留在旧容量。先确认文件系统类型；XFS 可对已挂载目录执行 `xfs_growfs <MOUNT_POINT>`，也可以在扩容时直接使用 `lvextend -r`。继续重复 `lvextend` 只会让两层差距更大。

② [诊断点] 目录存在，业务仍写入根文件系统

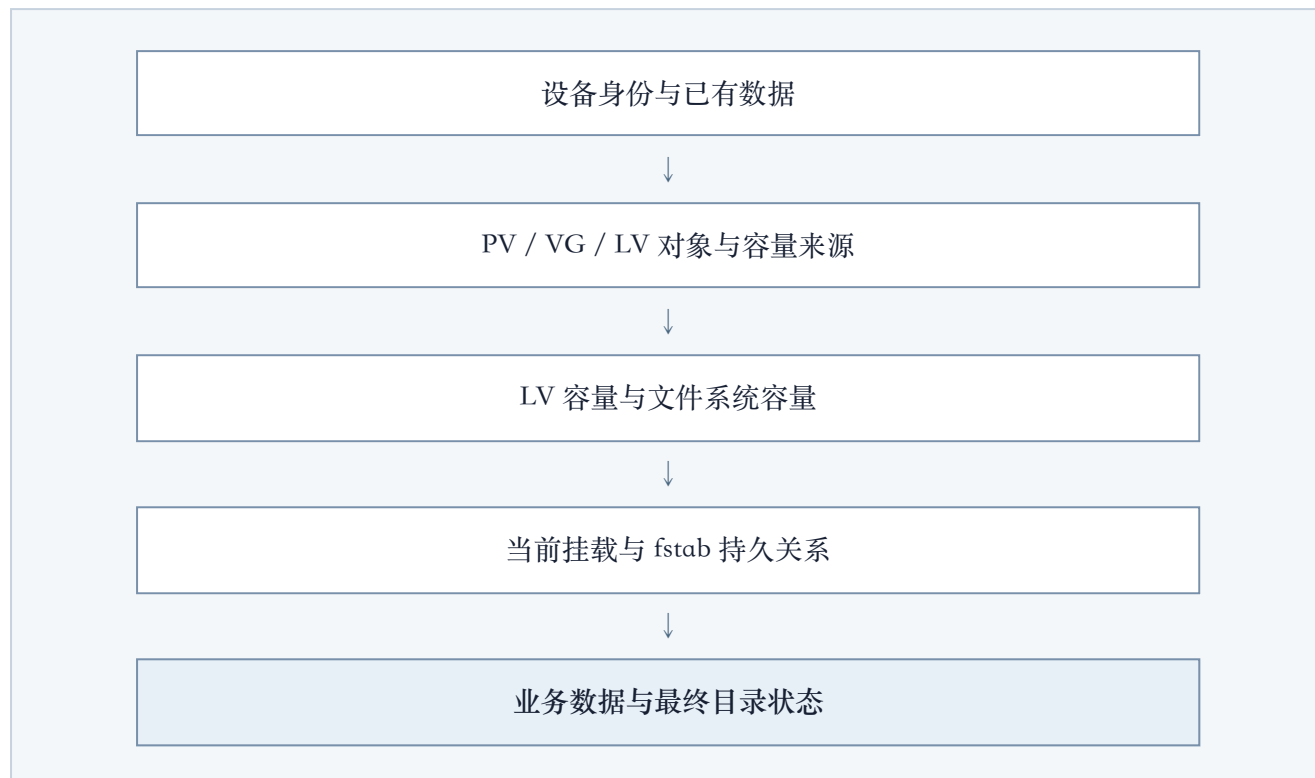
目录本身不代表已经挂载。使用 `findmnt <MOUNT_POINT>` 或 `mountpoint <MOUNT_POINT>` 判断当前挂载；再用 `findmnt --verify` 与 `mount -a` 检查持久配置。问题位于挂载层时，不应删除 LV 或重新格式化。

③ [诊断点] `lvextend` 报容量不足

查询 `vgs` 的 `vg_free` 与 `vg_free_count`。若 VG 余量不足，应寻找可安全使用的新设备，将其转换为 PV 并加入 VG；系统中“还有一块磁盘”并不等于目标 VG 已经拥有这块容量。

④ [诊断点] 候选设备已有文件系统或属于其他 VG

停止写操作，重新确认设备用途。`pvccreate`、`mkfs`、`lvremove`、`vgremove`、`pvremove` 都是可能破坏数据的操作，不能作为“先清掉再重来”的通用排错路径。



[Cheatsheet] `lvs` 与 `df` 不一致查文件系统；目录存在但 `findmnt` 无结果查挂载；`lvextend` 空间不足查 VG 余量；设备已有签名或归属时停止破坏性操作。

[经典任务] 建立并扩展一个可持久挂载的项目卷

服务器提供两个确认空闲的分区 `/dev/vdb1` 与 `/dev/vdc1`。请完成以下目标：

1. 使用 `/dev/vdb1` 创建 `projectvg`，PE 大小为 16 MiB；
2. 创建 `projectlv`，初始大小为 50 个 extent；
3. 在 LV 上创建 XFS，并通过 UUID 持久挂载到 `/srv/project`；
4. 在 `/srv/project/marker.txt` 写入 `keep-this-data`；
5. 将 LV 再增加 20 个 extent，同时扩大文件系统并保留标记文件；
6. 若 VG 余量不足，先使用 `/dev/vdc1` 扩展 VG；
7. 不得重新格式化、删除原 LV 或缩减其他 LV。

验收时要能证明：PV 与 VG 关系正确，VG 的 PE 符合要求，LV 最终为 70 个 extent，XFS 已获得新增空间，挂载来源与 UUID 正确，标记文件仍然存在。

卡住时再想一想

`lvs` 变大，是否已经证明文件系统也变大？

新分区要经过哪两层，才能成为现有 LV 的可用容量？

目录存在，怎样证明它当前确实挂载了目标文件系统？

请先独立完成。参考分析与推荐实现从下一页开始。

[参考解答] 按对象依赖完成创建、挂载与扩容

① [操作点] 建立初始证据

先确认设备、已有 LVM 对象和挂载关系。这里的 `-o` 只保留后续决策真正需要的字段。

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS      # 设备、文件系统与挂载
pvs -o pv_name,vg_name,pv_size,pv_free          # 现有 PV 与归属
vgs -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count # VG 余量与 PE
lvs -o lv_name,vg_name,lv_size,lv_path,devices   # 现有 LV 与设备路径
```

② [操作点] 创建 PV、VG 与初始 LV

`-s 16M` 设置 VG 的 PE; `-l 50` 按 50 个 extent 分配; `-n projectlv` 设置 LV 名称。

```
pvccreate /dev/vdb1      # 初始化 PV
vgcreate -s 16M projectvg /dev/vdb1    # 创建指定 PE 的 VG
lvcreate -l 50 -n projectlv projectvg   # 创建 50 LE 的 LV
```

```
pvs -o pv_name,vg_name,pv_size,pv_free
vgs projectvg -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path,devices
```

③ [操作点] 创建 XFS 并持久挂载

```
mkfs.xfs /dev/projectvg/projectlv      # 新建 XFS
mkdir -p /srv/project                   # 建立挂载点
blkid /dev/projectvg/projectlv          # 获取真实 UUID
```

将真实 UUID 写入 `/etc/fstab` :

```
UUID=<真实 UUID> /srv/project xfs defaults 0 0
```

```
systemctl daemon-reload      # 重新加载 mount units
findmnt --verify              # 检查 fstab
mount -a                      # 当前会话实际挂载
findmnt /srv/project          # 验证挂载源
```

④ [操作点] 建立数据保留证据

```
printf 'keep-this-data\n' > /srv/project/marker.txt # 写入标记数据
sha256sum /srv/project/marker.txt                 # 记录校验值
```

⑤ [操作点] 解决容量来源并扩容

先看 VG 是否至少还有 20 个空闲 extent:

```
vgs projectvg -o vg_name,vg_free,vg_extent_size,vg_free_count
```

若不足, 先让 `/dev/vdc1` 成为 PV, 再加入 `projectvg`:

```
pvccreate /dev/vdc1 # 新设备进入 LVM
vgextend projectvg /dev/vdc1 # 新 PV 加入容量池
vgs projectvg -o vg_name,vg_size,vg_free,vg_free_count # 再次确认余量
```

然后增加 20 个 extent, 并同步扩大 XFS:

```
lvextend --extents +20 --resizefs projectvg/projectlv # LV 与文件系统一起扩展
```

⑥ [验证点] 完成分层验收

```
pvs -o pv_name,vg_name,pv_size,pv_free # PV 与 VG 关系
vgs projectvg -o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count # PE 与余量
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path # LV 最终容量
findmnt /srv/project # 当前挂载源
df -hT /srv/project # XFS 最终容量
blkid /dev/projectvg/projectlv # UUID
findmnt --verify # 持久配置
sha256sum /srv/project/marker.txt # 数据未改变
```

VG 的 PE 应为 16 MiB; 70 个 extent 对应约 1120 MiB; `lvs` 和 `df` 都应体现增长; `findmnt` 应指向目标 LV; `fstab` 的 UUID 与 `blkid` 一致; 校验值保持不变。

[Cheatsheet] 调查 → `pvccreate` → `vgcreate -s` → `lvcreate -l/-L -n` → `mkfs` → UUID 写入 `fstab` → `findmnt --verify` + `mount -a` → 必要时 `pvccreate` + `vgextend` → `lvextend -l +N -r` → 分层验收。

[本章收束] 把命令还原为对象变化

LVM 的操作并不难, 真正容易失分的是把不同层的成功混为一谈。 `pvccreate`、`vgcreate`、`lvcreate` 分别建立三层容量对象; `mkfs`、`mount`、`fstab` 分别处理文件系统、当前挂载和持久关系; `lvextend`

`-r` 同时推进 LV 与文件系统层。

创建题先确认设备身份，再沿对象依赖建立状态；扩容题先确认容量来源，再比较 LV 与文件系统是否同步增长；故障题则根据 `pvs`、`vgs`、`lvs`、`findmnt`、`df` 的证据判断失败位于哪一层。只要每条命令都能对应一个对象和一个验收点，LVM 就不会退化成需要死记的缩写清单。

第九章 网络文件系统与自动挂载

NFS 把远端导出接入本机目录树，autofs 则在访问路径时按需建立挂载。两者的关键不是目录是否存在，而是远端导出、客户端包、当前挂载、持久条目和触发行为是否对应。

[概念] NFS export 是服务端公布的共享路径；客户端 mount 将服务器与导出路径映射到本地目录。网络中断、服务端权限和 root_squash 都会影响最终功能。

[概念] autofs master map 指向子 map；indirect map 在父目录下按 key 创建挂载，direct map 用完整路径作为 key。未访问时 findmnt 无结果可以是正常按需状态。

[操作语义] `showmount -e` 查询传统导出列表，`mount -t nfs` 建立当前挂载，`findmnt -t nfs,nfs4` 读取真实挂载关系。

[操作语义] `automount -m` 展开有效 map，访问 key 触发挂载；`systemctl reload autofs` 重新读取映射。

[知识专题] NFS 路径、版本与身份边界

① [知识点] 客户端看到的是远端文件系统

`server:/export/path` 是挂载源，本地目录只是接入点。目录原有内容会被挂载遮蔽；卸载后重新出现。NFSv4 可能使用服务端伪根，`showmount` 结果不是所有 NFSv4 环境的唯一真相。

② [知识点] root_squash 改变远端 root 身份

服务端默认常把客户端 root 映射为匿名身份，避免远端 root 获得服务端 root 权限。客户端 chmod 失败不应通过关闭 root_squash 解决；按导出策略、远端所有者和实际业务用户判断。

③ [验证点] 分开验证发现、挂载和读写

```
showmount -e servera
findmnt /mnt/share
nfsstat -m
sudo -u alice touch /mnt/share/client-test
```

发现导出不能证明可挂载，当前挂载不能证明题目身份可写。

[Cheatsheet] 导出发现 `showmount -e`；当前源 `findmnt`；选项/版本 `nfsstat -m`；最后用目标身份验证读写。

[操作专题] 建立当前与 fstab NFS 挂载

① [操作点] 查询后建立当前挂载

安装 `nfs-utils`，确认名称解析与导出，再创建空挂载点。`-t nfs` 指定类型，`-o` 只添加题目需要的版本或选项。

```
dnf install nfs-utils
showmount -e servera
mkdir -p /mnt/shared
mount -t nfs servera:/exports/shared /mnt/shared
```

② [操作点] 写入持久条目

```
servera:/exports/shared /mnt/shared nfs defaults,_netdev 0 0
```

`_netdev` 表达依赖网络，但不自动修复服务器不可达。是否使用 `nofail`、`x-systemd.automount` 必须由题意决定。

③ [验证点] 当前、持久与远端功能

```
systemctl daemon-reload
findmnt --verify
umount /mnt/shared
mount -a
findmnt /mnt/shared
```

卸载再 `mount -a` 能证明条目确实建立挂载；仍需读取或写入远端文件。

[Cheatsheet] 当前：`mount -t nfs`；持久：`fstab + _netdev`；重载、verify、卸载后 `mount -a`；最终 `findmnt` 与目标身份功能测试。

[操作专题] 配置 autofs indirect map

① [操作点] 建立 master 与子 map

```
# /etc/auto.master.d/exam.autofs
/shares /etc/auto.shares --timeout=60

# /etc/auto.shares
docs -fstype=nfs,rw servera:/exports/docs
```

访问 `/shares/docs` 才触发挂载。indirect map 的 key 是 `docs`，不是完整路径。

② [操作点] 校验并加载

```
automount -m
systemctl enable --now autofs
systemctl reload autofs
```

③ [验证点] 触发前、触发后与超时

```
findmnt /shares/docs
ls /shares/docs
findmnt /shares/docs
journalctl -u autofs -b
```

触发前无具体 NFS mount 可以正常；访问后必须出现正确源。超时卸载后再次访问应能重新挂载。

[Cheatsheet] master 指父目录和 map；map 的 key 形成子路径；`automount -m` 查有效映射；访问触发后用 `findmnt` 核对源。

[诊断专题] 根据症状区分析、导出、权限与触发层

① [诊断点] No route 或名称失败

先用 `getent hosts`、`ip route get` 和端口连接检查网络，不修改 map 路径掩盖连接问题。

② [诊断点] access denied by server

核对客户端请求的导出路径、来源网络和服务端 export 权限；客户端目录 chmod 无法改变服务端拒绝。

③ [诊断点] autofs 路径存在但未挂载

实际访问 key，查看 `automount -m` 和 autofs journal。indirect key、完整远端路径或 map 语法错误是不同层。

[Cheatsheet] 解析→路由→服务端导出→客户端包/挂载→身份权限；autofs 再加 master/map/触发三层。

[经典任务] 提供一个持久共享和一个按需共享

servera 导出 `/exports/team` 与 `/exports/home/&`。把 team 持久挂载到 `/srv/team`；为用户路径配置 indirect autofs，使访问 `/remote/alice` 时挂载对应远端目录。不得把按需共享写成永久常驻挂载。验收导出、当前/持久 team、map 展开、触发源、用户读写和超时后再触发。

请先独立完成。参考解答从下一页开始。

[参考解答] 分别建立 fstab 与 indirect map

① [操作点] 调查与持久挂载

```
dnf install nfs-utils autofs
getent hosts servera; showmount -e servera
mkdir -p /srv/team
mount -t nfs servera:/exports/team /srv/team
```

写入 fstab 后执行 `daemon-reload`、`findmnt --verify`、卸载与 `mount -a`。

② [操作点] 配置按需用户目录

```
/remote /etc/auto.remote
* -fstype=nfs,rw servera:/exports/home/&
```

* 捕获 key，& 代入远端路径。运行 `automount -m`，启用 autofs。

③ [验证点] 触发和身份功能

```
ls /remote/alice
findmnt /remote/alice
sudo -u alice test -r /remote/alice
```

等待超时后再次访问并检查 journal。

[Cheatsheet] team: fstab 闭环; remote: master + wildcard map + 触发; 两者都核对远端源和目标身份。

[本章收束] 远端源、当前挂载和触发策略必须分开

NFS 题先证明网络与导出，再证明客户端挂载和实际身份功能；autofs 题则把未触发状态视为正常候选，通过访问与 findmnt 证明按需行为。持久 fstab 与 autofs map 是两种不同生命周期，不互相替代。

第十章 管理 SELinux

普通文件权限允许访问，并不代表系统一定会允许这次访问。SELinux 会在传统权限检查之后，再根据进程和目标对象的安全类型、请求的行为以及当前策略作出判断。本章围绕一个使用自定义目录和非标准端口的 Web 服务展开：先从“服务为什么失败”建立证据，再学习怎样配置持久规则，最后把文件类型、端口类型、Boolean 和 AVC 日志串成一条能够用于考试的处理路径。

SELinux 中最重要的不是先背命令，而是先分清“谁在访问、访问什么、请求什么行为、策略为什么允许或拒绝”。下面的概念和操作语义先建立本章的最小地图。

[概念] 自主访问控制（Discretionary Access Control, DAC）与强制访问控制（Mandatory Access Control, MAC）：用户、组和权限位属于传统 DAC；SELinux 在 DAC 允许之后继续执行强制策略判断，因此普通权限正确仍可能被拒绝。

[概念] 安全上下文、类型（type）与进程域（domain）：文件和进程都带有安全上下文。RHCSA 常见问题主要关注第三段 type；进程的 type 常称为 domain，例如 `httpd_t`，文件类型则描述对象用途，例如 `httpd_sys_content_t`。

[概念] 文件上下文规则、端口类型与 Boolean：文件上下文规则定义某类路径应具有持久类型；端口类型定义某类服务可以使用哪些端口；Boolean 是策略预留的行为开关。三者分别处理对象、端口和服务行为，不能互相替代。

[概念] AVC（Access Vector Cache）拒绝记录：它说明哪个进程域对哪个对象请求了什么行为并被策略拒绝，是从现象进入 SELinux 证据链的主要入口。

[操作语义] `getenforce` / `sestatus`：查看当前执行模式以及当前状态与持久配置是否一致。

[操作语义] `semanage fcontext` + `restorecon`：前者建立路径的持久类型规则，后者让现有文件按照规则恢复到预期标签。

[操作语义] `semanage port` / `setsebool -P`：分别管理服务端口类型和持久 Boolean，解决的是不同策略入口。

[操作语义] `ausearch -m AVC -ts recent`：查询近期拒绝证据，本质是缩小到当前失败操作，而不是自动生成修复答案。

[知识专题] 普通权限正确，为什么访问仍会被拒绝

设想一台服务器已经安装并配置了 `httpd`。站点内容放在 `/srv/site`，服务计划监听 TCP 82。目录对 Web 进程具备基本的 Unix 读取权限，防火墙也允许访问，但服务仍可能无法启动，或者启动后访问返回 `403 Forbidden`。

这种现象不能直接归因于 SELinux。正确的起点仍然是查看证据：服务是否在运行，配置是否通过语法检查，目标端口是否真正监听，普通权限是否允许访问，防火墙是否放行。只有这些信息不能解释现象，或

者日志中出现 AVC 拒绝时，才有理由把注意力进一步收缩到 SELinux。

传统的自主访问控制（Discretionary Access Control, DAC）通常根据用户、组和权限位判断访问。SELinux 在 DAC 允许之后继续检查强制访问控制规则。也就是说，普通权限拒绝时，SELinux 不会替它放行；普通权限允许时，SELinux 仍可能拒绝。

当我们查看文件或进程的安全上下文时，会看到类似下面的结构：

```
system_u:object_r:httpd_sys_content_t:s0
```

它依次包含 SELinux 用户、角色、类型和安全级别。在 RHCSA 常见的服务问题中，最需要关注的通常是第三段 `type`。对进程而言，这个类型也常称为 domain，例如 `httpd_t`；对文件而言，它描述对象用途，例如 `httpd_sys_content_t`。策略会结合进程域、目标对象类型和请求的行为，判断是否允许这次访问。



这里最重要的不是背完整的四段字符串，而是能够识别：当前是哪个进程域，正在对哪种对象类型请求什么行为。这三个问题也是后面阅读 AVC 记录时最有价值的线索。

[Cheatsheet] 普通权限允许后仍被拒绝，才进入 SELinux 层；识别 `进程域` → `目标类型` → `请求行为`；文件、端口和 Boolean 是三类独立策略入口。

[操作专题] 识别当前模式与持久配置

SELinux 可以处于 `Enforcing`、`Permissive` 或 `Disabled`。在 `Enforcing` 模式下，策略会真正拒绝不允许的访问；在 `Permissive` 模式下，系统仍然产生标签并记录本应发生的拒绝，但不阻止操作；`Disabled` 表示 SELinux 基础设施没有加载，也不会正常维护标签。

先用下面的命令确认当前模式：

```
getenforce
```

需要同时观察当前状态和配置文件中的启动模式时，`sestatus` 更直接：

```
sestatus
```

代表性输出中，下面两行回答的是两个不同问题：

```
Current mode:                enforcing
Mode from config file:       enforcing
```

前者说明当前内核正在怎样执行策略，后者说明系统下次按配置启动时计划进入什么模式。两者相同才说明当前状态与持久配置一致。

setenforce 只能在已启用 SELinux 的系统中临时切换 Enforcing 与 Permissive：

```
setenforce 0
setenforce 1
```

setenforce 0 可以帮助验证 SELinux 是否参与了故障，但现象在 Permissive 下消失，只能说明 SELinux 可能阻止了某个行为，不能说明问题已经修复。下一步仍要重新触发失败操作，查看 AVC，并找到文件标签、端口类型或 Boolean 等标准配置入口。

持久模式由 `/etc/selinux/config` 管理。例如题目要求系统保持 Enforcing，应确认：

```
SELINUX=enforcing
SELINUXTYPE=targeted
```

修改配置文件不会立即改变当前模式，**setenforce** 也不会修改该文件。因此，考试题若同时要求“当前处于 Enforcing”并且“重启后仍保持 Enforcing”，就必须分别验证这两个状态。

关键提醒

不要通过把 SELinux 切换为 Permissive 或 Disabled 来完成服务题。降级保护只能作为有限的诊断证据，最终解法必须修正具体的策略条件。

[验证]

```
getenforce
sestatus
grep -E '^SELINUX(=|TYPE=)' /etc/selinux/config
```

[Cheatsheet] 当前模式看 **getenforce**；当前与持久配置一起看 **sestatus**；临时诊断用 **setenforce**，最终必须恢复 Enforcing 并核对 `/etc/selinux/config`。

[知识专题] 从一个失败的 Web 服务开始取证

主场景中的 `httpd` 已经配置为使用 `/srv/site` 作为站点目录，并监听 TCP 82。开始修改 SELinux 之前，先确认其他层次提供了什么证据。

```
httpd -t
systemctl status httpd
ss -lntp | grep ':82'
namei -l /srv/site/index.html
firewall-cmd --list-ports
```

这些命令不是要在 SELinux 章节中重讲 `systemd`、`Apache`、文件权限和 `firewalld`，而是为了避免跳过调查直接猜答案。`httpd -t` 检查配置语法，`systemctl status` 说明服务当前状态，`ss` 说明端口是否实际监听，`namei -l` 可以帮助查看路径各层目录的普通权限，防火墙查询则说明网络过滤层是否允许该端口。

假设配置语法正确，但 `httpd` 启动时报告无法绑定 TCP 82；或者端口问题处理后，服务已经运行，本机访问 `/srv/site/index.html` 仍然返回 403。此时应重新触发失败操作，并查询近期 SELinux 拒绝：

```
ausearch -m AVC -ts recent -i
```

代表性的端口绑定拒绝可能包含下面的稳定字段：

```
avc: denied { name_bind }
comm="httpd"
scontext=...:httpd_t:s0
tcontext=...:port_t:s0
tclass=tcp_socket
```

阅读时不必被 PID、时间戳和完整上下文淹没。先提炼四件事：`httpd` 发起了访问；被拒绝的行为是 `name_bind`；源进程类型是 `httpd_t`；目标是一个尚未被策略视为 Web 端口的 TCP socket。若拒绝的是文件读取，则重点会转向目标路径和目标文件类型。

如果查询不到记录，先确认刚刚是否重新触发了失败操作、时间范围是否合适、`auditd` 是否运行。必要时再查看 `journalctl -t setroubleshoot` 等替代入口，但不要把排错变成依次尝试一长串日志命令。

[Cheatsheet] 先证实服务、监听、DAC 与防火墙；重新触发失败后运行 `ausearch -m AVC -ts recent -i`；优先读 `comm`、`scontext`、`tcontext`、`tclass` 与拒绝行为。

[操作专题] 当前标签与持久文件上下文规则

将内容从默认目录移动到 `/srv/site` 后，文件往往继承 `/srv` 的类型，而不是 Web 服务能够读取的 `httpd_sys_content_t`。先查看当前标签：

```
ls -Zd /srv/site
ls -lZ /srv/site
```

`ls -Z` 只能说明对象此刻携带什么标签。即使执行下面的命令后看到类型正确，也不能证明配置具有持久性：

```
chcon -t httpd_sys_content_t /srv/site/index.html
```

`chcon` 直接改变当前标签，适合用来理解“标签改变会影响访问”或进行短暂验证，但它没有建立路径的持久规则。以后执行 `restorecon`、重新标记文件系统或重新创建对象时，标签可能回到策略认为该路径应该具有的值。

稳定的做法是先登记路径规则：

```
semanage fcontext -a \
  -t httpd_sys_content_t \
  '/srv/site(/.*)?'
```

`/srv/site(/.*)?` 同时覆盖目录本身和其下的文件、子目录。这里不需要学习完整正则语法，只要能解释：`/srv/site` 是目录本身，后面的可选部分代表“斜杠以及其后的任意内容”。单引号把表达式原样交给 `semanage`，避免 Shell 对特殊字符产生干扰。

`semanage fcontext` 只记录“这类路径应是什么类型”，不会自动修改现有文件。接着使用 `restorecon` 把规则应用到当前对象：

```
restorecon -Rv /srv/site
```

如果 `semanage` 命令不存在，不要在每个 SELinux 工具旁都展开软件包清单。只需记住通用恢复思路：先检查拼写，再通过仓库反查命令来源，例如：

```
dnf provides '*/semanage'
```

确认并安装对应的软件包后继续。后续其他工具缺失时沿用同一思路。

[验证] 文件上下文配置至少要回答三个不同问题：持久规则是否存在，策略认为路径应该是什么标签，文件当前实际是什么标签。

```
semanage fcontext -l | grep -F '/srv/site'
matchpathcon /srv/site/index.html
ls -lZ /srv/site/index.html
```

若预期标签与当前标签不一致，再执行 `restorecon` 并重新检查。三者一致，才有理由认为当前状态和持久规则形成闭环。

[常见错误] 只执行 `chcon`，看到 `ls -Z` 显示为 `httpd_sys_content_t` 就停止。这里验证到的只是当前标签，尚未证明路径规则存在。

本专题的推荐路径可以重新串成：

```
semanage fcontext -l | grep -F '/srv/site'

semanage fcontext -a \
  -t httpd_sys_content_t \
  '/srv/site(/.*)?'

restorecon -Rv /srv/site

matchpathcon /srv/site/index.html
ls -lZ /srv/site/index.html
```

如果查询发现同一表达式已经存在，不应机械重复 `-a`。先确认原规则的类型，再根据目标决定是否使用修改操作：

```
semanage fcontext -m \
  -t httpd_sys_content_t \
  '/srv/site(/.*)?'
```

[Cheatsheet] 当前标签：`ls -Z`；持久规则：`semanage fcontext -a/-m`；应用规则：`restorecon -Rv`；闭环：规则列表 + `matchpathcon` + `ls -Z`。

[知识专题] 文件类型表达服务可以做什么

静态站点内容通常只需要由 `httpd` 读取，因此使用 `httpd_sys_content_t`。若业务确实要求 Web 服务写入某个目录，例如上传目录或应用生成内容，可以考虑为那一小块目录使用 `httpd_sys_rw_content_t`。

这不是简单的类型名称替换。它表达的是不同的行为边界：一个目录只需读取，就不应因为配置方便而授予服务写入能力；只有明确需要写入的对象，才使用可写类型。例如 `/srv/site` 作为静态站点根目录，而 `/srv/site/uploads` 用于上传，可以分别登记：

```
semanage fcontext -a \
  -t httpd_sys_content_t \
  '/srv/site(/.*)?'

semanage fcontext -a \
  -t httpd_sys_rw_content_t \
  '/srv/site/uploads(/.*)?'

restorecon -Rv /srv/site
```


即使 SELinux 类型允许写入，普通 Unix 所有者、组和权限位仍然必须允许相应进程写入；反过来也一样，普通权限允许不代表 SELinux 一定允许。排错时应明确自己正在检查哪一层，而不是看到“Permission denied”就只反复执行 `chmod`。

[Cheatsheet] 静态只读内容用 `httpd_sys_content_t`；确需 Web 写入的小范围目录用 `httpd_sys_rw_content_t`；SELinux 类型和 Unix 权限必须同时允许。

[操作专题] 允许服务使用非标准端口

Apache 配置中的 `Listen 82`、防火墙中的 `82/tcp` 和 SELinux 中的端口类型是三个独立状态。防火墙放行只能说明网络过滤层允许流量，不能证明 `httpd_t` 进程域被策略允许绑定这个端口。

修改前先查看 `http_port_t` 当前包含哪些 TCP 端口：

```
semanage port -l | grep -w http_port_t
```

若 TCP 82 尚未登记，添加映射：

```
semanage port -a \  
-t http_port_t \  
-p tcp \  
82
```

这里 `-a` 表示新增记录，`-t` 指定 SELinux 类型，`-p` 指定协议。端口映射区分 TCP 与 UDP，不能因为端口号相同就省略协议判断。

如果命令提示端口已经定义，不要反复执行同一条新增命令。先查目标端口当前属于什么类型：

```
semanage port -l | grep -w '82'
```

确认现有映射与任务目标冲突时，再考虑修改：

```
semanage port -m \  
-t http_port_t \  
-p tcp \  
82
```

[验证] 端口规则存在只能证明 SELinux 允许该类服务使用这个端口。完整链路还应分别确认 Apache 配置、进程实际监听、防火墙和最终访问：

```
grep -nE '^Listen[[:space:]]+82$' /etc/httpd/conf/httpd.conf
semanage port -l | grep -w http_port_t
systemctl is-active httpd
ss -lntp | grep ':82'
firewall-cmd --list-ports
curl -I http://localhost:82/
```

此时主场景中的端口限制已经处理。如果站点仍返回 403，而文件上下文尚未修复，就说明端口和目录是两条独立的策略路径；局部成功不能替代综合验收。

[Cheatsheet] 先查 `semanage port -l`；无记录用 `-a`，已有冲突记录才用 `-m`；端口闭环还要检查服务配置、`ss`、`firewalld` 与 `curl`。

[操作专题] Boolean：从业务行为找到已有策略入口

Boolean 用来启用或关闭策略预先提供的可选行为。它不是“SELinux 出错时随便试几个开关”，而是当业务确实需要某类行为时，查找已有策略是否提供了最小入口。

例如，若 Web 应用需要主动连接 MariaDB，可以先搜索与 `httpd` 相关的 Boolean：

```
getsebool -a | grep '^httpd_'
semanage boolean -l | grep httpd
```

找到与业务语义匹配的 `httpd_can_network_connect_db` 后，再持久启用：

```
setsebool -P httpd_can_network_connect_db on
```

不带 `-P` 的 `setsebool` 只改变当前值；`-P` 会将修改写入持久策略配置，执行时可能需要一些时间。修改前后都应使用 `getsebool` 验证：

```
getsebool httpd_can_network_connect_db
```

Boolean 不能替代文件标签、端口类型、普通权限和服务配置。若开关已经启用但访问仍失败，继续根据 AVC 中的对象、行为和上下文调查，而不是盲目打开更多权限。

[验证] 用 `getsebool <BOOLEAN>` 证明当前值；使用 `setsebool -P` 后再次查询，证明持久设置已经进入策略配置。该查询不能证明业务访问成功，仍需重新触发目标功能。

[Cheatsheet] 搜索：`getsebool -a / semanage boolean -l`；持久修改：`setsebool -P <BOOLEAN> on|off`；Boolean 只用于与业务语义匹配的预置行为。

[诊断专题] 从 AVC 记录推进到下一条证据

AVC (Access Vector Cache) 记录的价值在于解释“谁对什么对象做什么时被拒绝”。它不是一条可以直接复制的修复命令。看到拒绝后，应先判断它属于哪一种常见偏差：对象标签不正确，服务使用了非标准端口，业务需要某个已有 Boolean，还是服务本身配置错误。

`audit2why` 可以辅助解释审计拒绝，但它提供的是分析线索，不是绝对结论。`audit2allow` 能依据拒绝生成候选策略规则，却不应成为第一选择。RHEL 官方文档明确建议先检查标签问题，以及服务是否改变了默认目录、端口或行为而没有同步告诉 SELinux。

关键提醒

不要把“工具建议生成自定义策略”理解为“标准修复就是运行 `audit2allow -M`”。优先寻找现有的文件类型、端口类型和 Boolean。只有明确确认标准策略无法表达合法需求时，才进入自定义策略范围；这不属于本章核心操作路径。

主场景完整走过之后，可以把诊断思路收束为一张证据推进图：



这张图不是要求每次机械地从第一步执行所有命令。若题目已经提供了可靠证据，可以从已知状态继续；但每一个结论都应能回答“我凭什么这样判断”。

[Cheatsheet] 重新触发 → 查 AVC → 识别进程、对象与行为 → 优先检查文件类型、端口类型和 Boolean → 采用最小持久修复 → 重新验证业务。

[经典任务] 在 Enforcing 模式下恢复非标准 Web 服务

一台 RHEL 9 服务器已经安装 `httpd`，并完成以下业务配置：

- Apache 使用 `/srv/exam-site` 作为站点目录；
- `/srv/exam-site/index.html` 已存在，内容不得删除或改写；
- Apache 配置为监听 TCP 82；
- `firewalld` 正在运行；
- 当前访问失败，系统可能同时存在多个配置偏差。

请将系统配置到下面的目标终态：

1. SELinux 当前处于 `Enforcing`，重启后的配置也保持 `Enforcing`；
2. `httpd` 当前运行，并配置为开机自动启动；
3. `httpd` 可以在 TCP 82 上监听；
4. 客户端能够通过 `http://servera:82/` 读取现有首页；
5. `/srv/exam-site` 的 SELinux 文件上下文配置是持久的，执行 `restorecon` 后仍保持正确；
6. 不通过关闭 SELinux、删除原文件或安装自定义策略模块绕过问题。

验收时至少应能证明：当前模式与持久模式一致，文件上下文规则与当前标签一致，TCP 82 属于正确的 SELinux 端口类型，服务当前状态与开机启动状态正确，端口实际监听，防火墙允许访问，最终 HTTP 请求成功。

卡住时再想一想

当前标签正确，是否就能证明路径规则已经持久存在？

防火墙允许 TCP 82，是否能证明 SELinux 允许 `httpd` 绑定该端口？

服务显示 `enabled`，是否能证明它现在正在运行？

请先根据验收要求独立完成。参考分析与推荐实现从下一页开始。

[参考解答] 参考分析与推荐实现

先把题目拆成几个彼此独立的状态：SELinux 模式、文件上下文、端口类型、服务当前状态、开机启动、防火墙和最终业务访问。任何一个局部状态正确，都不能单独证明整个任务完成。

[分析思路] 先查询而不是直接修改。下面这组命令用于建立初始证据：

```
getenforce
sestatus
httpd -t
systemctl status httpd
systemctl is-enabled httpd
ss -lntp | grep ':82'
semanage port -l | grep -w http_port_t
semanage fcontext -l | grep -F '/srv/exam-site'
matchpathcon /srv/exam-site/index.html
ls -lZ /srv/exam-site/index.html
firewall-cmd --list-ports
ausearch -m AVC -ts recent -i
```

如果 `httpd` 因 TCP 82 的 `name_bind` 拒绝而无法启动，先处理端口类型。目标端口尚未登记时：

```
semanage port -a \
  -t http_port_t \
  -p tcp \
  82
```

若端口已存在，先确认现有类型，再根据实际状态决定是否使用 `-m`，不要反复执行 `-a`。

接着为自定义站点目录建立持久规则，并应用到现有文件：

```
semanage fcontext -a \
  -t httpd_sys_content_t \
  '/srv/exam-site(/.*)?'
restorecon -Rv /srv/exam-site
```

这里若只执行 `chcon`，短时间内可能看到页面恢复，但持久规则仍然缺失。因此参考路径直接从 `semanage fcontext` 建立规则，不把临时标签当作最终配置。

确认 `/etc/selinux/config` 中的模式为：

```
SELINUX=enforcing
SELINUXTYPE=targeted
```

若当前模式是 Permissive，在修正具体策略条件后切回 Enforcing：

```
setenforce 1
```

然后确保服务当前运行并配置开机启动：

```
systemctl enable --now httpd
```

若 firewalld 尚未允许 TCP 82，添加持久规则并重新加载：

```
firewall-cmd --permanent --add-port=82/tcp  
firewall-cmd --reload
```

[验证] 验证不应只看某条配置命令是否返回成功。依次证明各个目标状态：

```
getenforce  
sestatus  
  
grep -E '^SELINUX(=|TYPE=)' /etc/selinux/config  
  
semanage fcontext -l | grep -F '/srv/exam-site'  
matchpathcon /srv/exam-site/index.html  
ls -lZ /srv/exam-site/index.html  
  
semanage port -l | grep -w http_port_t  
  
systemctl is-active httpd  
systemctl is-enabled httpd  
ss -lntp | grep ':82'  
  
firewall-cmd --list-ports  
curl -I http://localhost:82/
```

理想情况下，当前模式和配置模式均为 Enforcing；路径规则指向 `httpd_sys_content_t`，当前标签与预期一致；`http_port_t` 包含 TCP 82；服务同时为 `active` 和 `enabled`；端口实际监听；防火墙列出 `82/tcp`；HTTP 请求返回成功状态。

最后重新执行：

```
restorecon -Rv /srv/exam-site
```

再检查标签和页面访问。这样可以验证站点不是依赖一次临时 `chcon` 才能工作。

本任务中最容易出现的三个误判是：看到 `enabled` 就认为服务正在运行；看到防火墙放行就认为端口链路已完整；看到 `ls -Z` 当前标签正确就认为文件上下文已经持久。它们都把一个局部证据错误地扩大成了最终结论。

[本章收束] 回到本章的问题

SELinux 故障不应从“关闭保护”开始，而应从证据开始。普通权限、服务状态、防火墙和 SELinux 各自回答不同问题；在确认 SELinux 参与拒绝后，再从进程域、目标对象和被拒绝行为中判断应检查文件类型、端口类型还是 Boolean。

文件上下文题最重要的关系是：`semanage fcontext` 建立持久路径规则，`restorecon` 按规则调整当前标签，`ls -Z` 只观察当前状态，`matchpathcon` 帮助比较预期与实际。非标准端口题则要把 Apache 配置、实际监听、防火墙和 `http_port_t` 四个状态分开验证。Boolean 只有在业务确实需要某类可选行为时才启用，并通过 `-P` 明确持久性。

学完本章后，你应能在不关闭 SELinux 的前提下，处理自定义 Web 目录和非标准端口，读懂一段精简的 AVC 拒绝，并用“查询当前状态—选择最小持久修改—重新触发—综合验证”的方式完成任务。

第十一章 启动过程、Target 与系统恢复

启动恢复题在正常系统之外修改持久状态，任何路径、挂载或 SELinux 判断错误都可能让下一次启动继续失败。本章把 firmware、boot loader、kernel/initramfs、systemd target 与恢复根环境串成可验证链路。

[概念] 启动依次经过 firmware、boot loader、kernel/initramfs 和 systemd；target 是一组 unit 依赖的同步点，不是传统运行级别进程。

[概念] rescue 提供较完整的单用户环境，emergency 更小；`rd.break` 在 initramfs 阶段中断，此时真实根通常挂在 `/sysroot` 且可能只读。

[操作语义] `systemctl get-default/set-default/isolate` 分别查询默认 target、改变下次启动目标和切换当前目标。

[操作语义] 恢复环境中 `mount -o remount,rw /sysroot`、`chroot /sysroot` 后才对真实系统执行 `passwd`、`fstab` 修复和持久配置。

[知识专题] 启动阶段与 target 边界

① [知识点] 每阶段失败证据不同

看不到 boot loader、kernel panic、initramfs 找不到根、systemd 进入 emergency 是不同层。journal 只覆盖内核/systemd 能记录的阶段。

② [知识点] 默认 target 与当前 target 分开

`set-default` 只改变下次启动链接；`isolate` 当前切换并停止无关 unit，可能断开图形或网络会话。

③ [验证点] 查询默认、当前与失败 unit

```
systemctl get-default
systemctl list-units --type=target --state=active
systemctl --failed
journalctl -b -p err
```

[Cheatsheet] 默认 `get-default`；当前 active target；失败 unit 与本次启动 journal；`isolate` 有中断风险。

[操作专题] 使用 rd.break 重置 root 口令

① [操作点] 在 boot loader 临时编辑 kernel 行

在目标启动项按 `e`，在 `linux` 行末增加 `rd.break`，按环境提示继续。这是一次性参数，不修改永久 GRUB 配置。

② [操作点] 进入真实根并修改

```
mount -o remount,rw /sysroot
chroot /sysroot
passwd root
touch /.autorelabel
exit
exit
```

`chroot` 前运行 `passwd` 会修改 `initramfs` 环境而非真实系统。`/.autorelabel` 让下次启动重新标记，过程可能较久。

③ [验证点] 恢复 Enforcing 与登录

启动后确认 `getenforce`、`root` 认证和 `journalctl -b`。不能声称已实机登录；讲义只给静态操作闭环。

[Cheatsheet] `rd.break` → `/sysroot` 重挂 `rw` → `chroot` → `passwd` → `/.autorelabel` → 正常启动后核对 `Enforcing` 与认证。

[操作专题] 在 emergency 中修复错误 fstab

① [诊断点] 先从失败 unit 找条目

`systemctl --failed`、`mount unit status` 和 `journal` 指向无法挂载的路径/UUID。备份 `fstab` 后用 `blkid`、`lsblk -f` 核对真实对象。

② [操作点] 最小修正并验证

根为只读时先重挂 `rw`。修改错误 `UUID`、类型或选项，不用 `nofail` 掩盖必需挂载。

```
findmnt --verify
mount -a
systemctl daemon-reload
```

③ [验证点] 离开恢复环境前证明关键路径

用 `findmnt`、`df -hT` 和数据检查确认，随后 `systemctl default` 或重启。

[Cheatsheet] 失败 unit/journal → blkid/lsblk → 最小修 fstab → verify + mount -a → findmnt/数据 → 返回 default。

[诊断专题] 避免在恢复环境修改错误根

① [诊断点] passwd 成功但口令没变

检查是否忘记 `chroot /sysroot`，以及真实根是否 `rw`。

② [诊断点] 修复后启动卡在 relabel

若创建 `/.autorelabel`，长时间重新标记可能正常；查看控制台而非强制重启。

③ [诊断点] 默认 target 正确但服务未起

target 只表达依赖集合，继续查具体 unit 的 `enabled`、`condition` 和 `journal`。

[Cheatsheet] 先确认当前根与读写状态；再修改真实持久文件；relabel 要等待；target 成功不替代具体服务验证。

[经典任务] 恢复 root 访问并修正启动挂载

系统因错误 `fstab` 进入 `emergency`，且 `root` 口令未知。恢复访问，保留数据并修正目标 `UUID`，使系统进入 `multi-user.target`；SELinux 最终保持 `Enforcing`。不得删除文件系统或用 `nofail` 绕过必需挂载。验收真实根、口令修改、`relabel` 标记、`fstab` 静态/实际挂载和默认 `target`。

请先独立完成。参考解答从下一页开始。

[参考解答] 在 initramfs 中建立真实根修改闭环

① [操作点] rd.break 与 chroot

按启动项临时加 `rd.break`，重挂 `/sysroot` 为 `rw`，`chroot` 后修改 `root` 口令并创建 `/.autorelabel`。

② [操作点] 修复持久配置

在 `chroot` 内运行 `lsblk -f / blkid`，备份并编辑 `/etc/fstab`；执行 `findmnt --verify`。若设备可用，实际 `mount -a`。

③ [操作点] 设置目标并退出

```
systemctl set-default multi-user.target
```

退出两次继续启动。完成 `relabel` 后核对 `get-default`、失败 `units`、目标挂载和 `Enforcing`。

[Cheatsheet] 真实根 `rw/chroot` → `passwd/relabel` → `UUID` 修正 → `verify/mount` → `set-default` → 启动后综合验收。

[本章收束] 恢复操作必须作用于真实持久系统

启动阶段决定证据入口，恢复环境决定路径语义。只要先确认 `/sysroot`、读写状态与 `chroot`，再修改口令、`fstab` 和 `target`，就能避免“命令成功但改错系统”。SELinux `relabel` 和启动后功能验证是恢复闭环的一部分。

第十二章 Podman 容器与持久运行

Podman 以普通用户运行容器时，镜像、容器、宿主目录、端口和用户级 systemd 各有独立状态。本章以 rootless 服务为主线，强调卷标签、登录后之外的持久运行和容器内外双层验证。

[概念] image 是只读模板，container 是镜像的运行实例；rootless 容器属于用户命名空间，宿主 UID、低端端口和存储路径受用户权限约束。

[概念] 端口发布把宿主端口转发到容器端口；bind mount 把宿主路径暴露给容器。SELinux enforcing 下 `:Z` 为单容器私有重标记，`:z` 允许多个容器共享。

[操作语义] `podman pull/images/inspect` 管镜像证据，`podman run/ps/logs/exec` 管容器当前实例。

[操作语义] Quadlet `.container` 或 `podman generate systemd` 生成的用户 unit 把容器目标写成 systemd 声明；`loginctl enable-linger` 允许用户退出后仍由 user manager 运行。

[知识专题] 镜像、容器、端口和存储层

① [知识点] 镜像名称应包含可靠 registry/tag

短名解析依赖 registries 配置；题目给出完整引用时原样使用。digest 可证明不可变内容，tag 可能移动。

② [知识点] 宿主与容器端口方向不能反

`-p 8080:80` 是宿主 8080 到容器 80。容器内监听正确不代表宿主已发布。

③ [验证点] 分别查 inspect、监听和 HTTP

```
podman inspect web
podman port web
ss -lntp | grep ':8080'
curl -I http://localhost:8080/
```

[Cheatsheet] 镜像看 images/inspect；实例看 ps/logs；映射看 podman port/ss；最终用协议请求。

[操作专题] 运行 rootless 容器并持久保存数据

① [操作点] 用目标用户准备目录

```
install -d -m 0750 -o webuser -g webuser /home/webuser/site
sudo -iu webuser podman pull registry.example.com/web:9
```

② [操作点] 创建端口、环境和卷

```
podman run -d --name web -p 8080:80 \  
-v /home/webuser/site:/usr/share/nginx/html:Z \  
-e APP_ENV=exam registry.example.com/web:9
```

③ [验证点] 当前实例和数据

`podman ps`、`logs`、`inspect`、`port` 后从宿主 `curl`；重建容器前后检查宿主文件，证明数据不只存在于可写层。

[Cheatsheet] 目标用户准备目录 → `pull` → `run name/port/volume/env` → `ps/logs/inspect/port` → 宿主 HTTP 与数据检查。

[操作专题] 用用户 systemd 持久运行容器

① [操作点] 选择当前 RHEL 9 可用集成

较新 RHEL 9/Podman 优先 Quadlet：在 `~/.config/containers/systemd/web.container` 描述 Image、PublishPort、Volume，并 `daemon-reload`。题目/环境明确使用生成 unit 时可运行 `podman generate systemd --new --files --name web`。

② [操作点] 启用用户 unit 与 linger

```
loginctl enable-linger webuser  
sudo -iu webuser systemctl --user daemon-reload  
sudo -iu webuser systemctl --user enable --now web.service
```

③ [验证点] user manager、unit 和容器

查询 `loginctl show-user -p Linger`、`systemctl --user is-active/is-enabled`、`podman ps` 与 HTTP。仅容器当前 `running` 不证明注销或重启后运行。

[Cheatsheet] 持久声明 → `user daemon-reload` → `enable --now` → `linger` → `unit` + 容器 + HTTP；当前 RHEL 9 优先按环境支持的 Quadlet。

[诊断专题] 从容器状态定位拉取、端口、权限或 SELinux

① [诊断点] 容器立即退出

看 `podman ps -a`、`logs` 和 `inspect` 的 `exit code/command`，不反复 `run` 同名容器。

② [诊断点] permission denied on volume

检查目标用户路径权限、容器进程 UID 映射和 SELinux AVC；需要私有重标记时使用 `:Z`，不禁用 SELinux。

③ [诊断点] user service 登录后才运行

检查 `linger`、`unit` 所属用户、`XDG/user manager` 和 `enabled` 状态；`root` 的 `systemctl` 与 `systemctl --user` 不是同一管理器。

[Cheatsheet] 退出看 `ps -a/logs/inspect`；卷拒绝查 `DAC+UID 映射+SELinux`；持久失败查 `user unit/linger`；不重建掩盖证据。

[经典任务] 部署一个 rootless 持久 Web 容器

以用户 webuser 使用指定完整镜像，发布宿主 8080 到容器 80，把现有站点目录持久映射并保持 SELinux Enforcing。容器应在用户退出和系统重启后由用户 systemd 管理。不得复制私有认证信息或把目录设为 777。验收镜像、实例、端口、卷标签、数据、unit、linger 和 HTTP。

请先独立完成。参考解答从下一页开始。

[参考解答] 按用户、数据、实例和 systemd 四层部署

- ① [操作点] 以 webuser 准备目录、登录 registry（如题目要求）并 pull 完整镜像。
- ② [操作点] 使用 `podman run -d --name web -p 8080:80 -v <HOST>:<CONTAINER>:Z`，检查 logs/inspect/curl。
- ③ [操作点] 写 Quadlet 或按环境生成 user unit，daemon-reload、enable --now，并由管理员 enable-linger。
- ④ [验证点] 查询 image ID、podman ps、port、宿主文件、user unit active/enabled、Linger=yes 和注销后 HTTP。

[Cheatsheet] 用户身份 → 镜像 → 卷/端口实例 → 当前功能 → user systemd + linger → 注销/重启持久与数据验收。

[本章收束] 容器运行与服务持久性是两套状态

Podman 当前实例、宿主网络、持久目录、SELinux 标签和用户 systemd 必须分别证明。rootless 的关键是始终在目标用户上下文操作，并让 user manager 与 linger 承担生命周期，而不是用 root 重新创建同名容器。

第十三章 RHCSA 综合任务

综合题不引入新的命令体系，而是要求识别多个对象之间的依赖和交叉影响。本章用一台服务器的完整终态训练读题、风险排序、分层验证和最终清单。

[概念] 综合验收把身份、网络、仓库、服务、存储、SELinux、调度和容器视为共享同一主机状态的多个目标，后做的修改可能破坏先完成的题。

[概念] 证据矩阵把每项要求映射到当前状态、持久配置和功能验证，避免以一条成功命令覆盖多个评分点。

[操作语义] 调查命令先建立共享对象图；最小修改只推进一个目标层；最终检查从题目原文逐项反查证据。

[知识专题] 把题目编排为依赖图与风险队列

① [知识点] 基础依赖先于业务

网络/解析影响仓库、NFS 与 SSH；仓库影响软件；身份影响权限和 rootless 容器；存储与 SELinux 影响服务数据。

② [知识点] 高风险写操作必须有证据门

磁盘初始化、网络切换、fstab、启动恢复和权限递归修改先记录基线与恢复路径。

③ [验证点] 每题记录当前、持久、功能三列

例如服务为 active/enabled/HTTP，挂载为 findmnt/fstab/data，防火墙为 runtime/permanent/remote request。

[Cheatsheet] 先画依赖与共享对象；低风险基础先做；高风险操作设证据门；每题维护当前/持久/功能三列。

[操作专题] 执行中保持局部闭环

① [操作点] 完成一层立即验证

仓库刷新后再装包，服务语法通过后再启动，fstab verify/mount 后再继续，SELinux 规则应用后再测业务。

② [诊断点] 卡题保存缺失证据

记录最后错误、已证实层、未证实层和共享影响，不用删除/重建清空现场。

③ [验证点] 每次共享对象变化后回归相关题

修改 httpd 端口后回查 firewalld/SELinux；改变用户组后用新会话回查目录与容器。

[Cheatsheet] 局部闭环 → 保存卡点 → 共享对象变化后回归；不把局部成功扩大。

[操作专题] 完成最终一页检查

① [验证点] 身份、网络 and 软件

id/getent、nmcli/ip/getent、dnf/rpm；确认题目值和持久性。

② [验证点] 服务、安全和调度

active/enabled/listen/function；firewalld 双态；SELinux Enforcing/规则/标签；cron 实际产物。

③ [验证点] 存储和容器

lsblk/pvs/vgs/lvs、findmnt/df/fstab、swapon；Podman image/container/volume/port/user unit/linger。

④ [边界] 重启前先跑静态检查

网络 profile、findmnt --verify、服务语法和恢复入口都成立后才决定是否重启。

[Cheatsheet] 从原题逐项复核；静态配置 → 当前状态 → 功能 → 持久性；共享对象回归；重启不是第一排错。

[经典任务] 把服务器配置为可综合验收的终态

服务器需同时完成项目用户与共享目录、静态网络与仓库、HTTP 服务及非标准内容目录、XFS/LVM/Swap 持久存储、NFS autofs、周期健康检查、rootless 容器和默认 target。题目给定设备与地址必须按现场值使用，已有数据不得删除，SELinux 保持 Enforcing。请自行安排顺序并提供最终证据矩阵。

请先独立完成。参考解答从下一页开始。

[参考解答] 按依赖和风险推进一条推荐路径

- ① [操作点] 先调查网络、设备、身份、现有服务和挂载，建立共享对象表。
- ② [操作点] 完成网络/解析/仓库，再创建身份与目录；随后安装配置服务。
- ③ [操作点] 设备证据充分后完成分区/LVM/文件系统/fstab/Swap；再配置 NFS、SELinux、计划任务和容器。
- ④ [验证点] 每题使用本章三列表；运行服务语法、findmnt verify、Anki 中的各层命令，回查共享端口/路径/用户。重启前确认恢复路径。

[Cheatsheet] 依赖图 → 基础连接 → 身份/软件 → 高风险存储 → 安全/调度/容器 → 证据矩阵 → 回归与持久性。

[本章收束] 最终答案是一组互相一致的系统状态

综合能力不是执行更多命令，而是让用户、路径、端口、设备和服务在当前与重启后保持一致。证据矩阵让每个评分点都有对应查询，也让交叉影响在交卷前被发现。

第一章 Ansible 架构、配置与 Inventory

RHCE 的第一道证据链发生在控制节点：当前项目读取哪个 `ansible.cfg`、Inventory 匹配哪些主机、SSH 与提权使用什么身份。业务任务之前先证明执行边界，才能避免把 `unreachable` 误诊为模块错误。

[概念] 控制节点保存项目、Inventory、变量和内容；受管节点通常通过 SSH 接收临时模块执行，需要可用 Python 与适当提权。Ansible agentless 不等于没有远端前提。

[概念] Inventory 同时定义主机集合、组关系和少量连接变量；业务变量更适合 `group_vars/host_vars`。pattern 决定本次执行集合，错误匹配可能安静地遗漏主机。

[操作语义] `ansible-config dump --only-changed` 观察有效配置，`ansible-inventory --graph/--host` 展开 Inventory，`ansible <PATTERN> -m ping` 验证连接与 Python。

[操作语义] `ansible-doc` 查询模块和参数；ad hoc 适合一次性调查，Playbook 承担可重复目标状态。

[知识专题] 配置发现、项目边界与连接前提

① [知识点] 配置来源有优先顺序

`ANSIBLE_CONFIG` 可显式指定；当前目录、家目录和系统配置按规则搜索。项目中一个拼错或不可读的 `ansible.cfg` 可能导致使用另一份配置。`ansible --version` 会显示 config file。

② [知识点] inventory、remote_user 与 become 是独立参数

SSH 登录用户先连接，become 再在远端切换权限。能 ping 不证明 become 成功；需要 `-b` 的受控命令单独验证。

③ [验证点] 输出有效配置与版本

```
ansible --version
ansible-config dump --only-changed
ansible-inventory --graph
```

[Cheatsheet] 先证配置文件与 inventory；再证主机图、SSH、Python 和 become；连接成功不替代业务终态。

[操作专题] 编写静态 Inventory 并验证 pattern

① [操作点] 定义主机、组与嵌套组

```
[web]
servera ansible_host=192.0.2.11
serverb ansible_host=192.0.2.12

[production:children]
web
```

Inventory 名称是 `inventory_hostname`，连接地址可以不同。

② [操作点] 建立最小项目配置

```
[defaults]
inventory=./inventory
remote_user=devops
host_key_checking=True

[privilege_escalation]
become=True
```

不要把口令明文写进配置；使用题目提供的安全入口。

③ [验证点] 展开、匹配和连接

```
ansible-inventory --graph
ansible-inventory --host servera
ansible web --list-hosts
ansible web -m ansible.builtin.ping
ansible web -b -m ansible.builtin.command -a 'id -u'
```

[Cheatsheet] Inventory graph/host → pattern list-hosts → ping → become 身份；每步回答一个边界。

[诊断专题] 区分 unmatched、unreachable 与 privilege failure

① [诊断点] pattern 匹配 0 主机

先 `--list-hosts` 和 `inventory graph`，检查组名、inventory 路径和 children，不改 SSH。

② [诊断点] UNREACHABLE

使用 `-vvvv` 读取目标地址、用户、密钥和主机密钥错误；从控制节点手工 SSH 验证同一身份。

③ [诊断点] ping 成功但任务权限失败

检查 become、sudo 授权和 `ansible_become_user`，用 `id -u` 最小验证，不先给全局 NOPASSWD。

[Cheatsheet] 0 hosts 查 Inventory/pattern; unreachable 查 SSH; permission 查 become/sudo; 不要在错误层修改业务任务。

[经典任务] 建立可执行的多组主机项目

在指定目录创建 `ansible.cfg` 与静态 Inventory: `web`、`db` 两组归入 `production`, 使用 `devops` 连接并按题意提权。不得关闭主机密钥检查或写明文口令。验收配置来源、组图、每台 `hostvars`、`pattern`、SSH/Python 和 `root` 提权。

请先独立完成。参考解答从下一页开始。

[参考解答] 从配置发现到提权逐层验收

- ① [操作点] 在题目目录写相对 inventory 路径和 remote_user/become 配置。
- ② [操作点] 定义 web/db 与 production:children, 连接地址放 host vars。
- ③ [验证点] 运行 `ansible --version`、`config dump`、`inventory graph/host`、`--list-hosts`、`ping` 和 `command id -u -b`。若失败按 `unmatched/unreachable/become` 分层处理。

[Cheatsheet] `config file` → `inventory graph/host` → `list-hosts` → `ping` → `become id`; 不把一层成功扩大。

[本章收束] 先固定执行边界，再编写业务状态

Inventory 与配置是所有 Playbook 的作用域。只有明确控制节点读取的文件、目标主机集合、连接身份和提权能力，后续模块失败才有可解释的上下文。

第二章 YAML、Playbook、Task 与模块

Playbook 把目标主机、提权和一组任务写成可重复执行的 YAML。结构能通过解析只是第一道门，模块参数、目标主机、远端终态和第二次执行仍需独立验证。

[概念] YAML mapping 表达键值，sequence 表达列表；缩进属于结构，Tab 不用于缩进。一个 play 选择 hosts 并包含 tasks，每个 task 调用一个模块。

[概念] 专用模块表达目标状态并返回 changed；command 不经 Shell，shell 才解释管道/重定向。能用专用模块时不用无条件命令。

[操作语义] `ansible-playbook --syntax-check/--list-hosts/--list-tasks` 分别检查解析、主机和任务边界；`--check --diff` 在模块支持范围内预览。

[操作语义] 模块返回 ok/changed/failed/skipped；recap 总结执行，不证明远端业务功能。

[知识专题] YAML 结构、标量和模块参数

① [知识点] playbook 顶层是 play 列表

```
---
- name: Configure web
  hosts: web
  become: true
  tasks:
    - name: Install httpd
      ansible.builtin.dnf:
        name: httpd
        state: present
```

② [知识点] 布尔与权限模式要避免类型歧义

布尔用 `true/false`；文件 mode 常引用为 `'0644'`，避免 YAML 数值解释。含冒号、`#` 或 Jinja 表达式开头的字符串按需引用。

③ [验证点] 先让解析器和 `ansible-doc` 证明字段

`ansible-doc ansible.builtin.dnf` 查询字段，不从其他模块类推参数名。

[Cheatsheet] 顶层 play 列表；play 含 hosts/tasks；task 只调用一个模块；布尔 `true/false`、mode 引用；字段查 `ansible-doc`。

[操作专题] 安装、配置并启动服务的最小 Playbook

① [操作点] 用目标状态模块

```
- name: Deploy web
  hosts: web
  become: true
  tasks:
    - ansible.builtin.dnf:
        name: httpd
        state: present
    - ansible.builtin.copy:
        content: "exam\\n"
        dest: /var/www/html/index.html
        mode: '0644'
    - ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
```

② [验证点] 静态、限制执行与远端终态

```
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --list-hosts
ansible-playbook site.yml --limit servera
ansible web -b -m command -a 'systemctl is-enabled httpd'
```

再从受管节点或客户端检查文件与 HTTP。

③ [验证点] 第二次执行解释 changed

第二次无 changed 是幂等证据；仍有 changed 时逐 task 查无条件 command、动态内容或错误 changed_when。

[Cheatsheet] syntax/list → limit 执行 → recap → 远端文件/服务/HTTP → 全量 → 第二次执行；recap 不替代终态。

[诊断专题] 区分 YAML、模块、连接和远端失败

① [诊断点] syntax error

根据行列号检查上一层缩进、冒号和列表标记，不只盯报错行。

② [诊断点] unsupported parameters

用 `ansible-doc` 核对 FQCN 与当前 collection 版本字段，不把所有模块参数复制过来。

③ [诊断点] command 与 shell 选错

command 中 `|`、`>` 不会由 Shell 解释；若确需 Shell 使用 `shell` 并确保幂等/引用，优先寻找专用模块。

[Cheatsheet] 解析错查 YAML；参数错查 `ansible-doc`；`unreachable` 回连接层；远端 `failed` 读 module msg；命令行为先选专用模块。

[经典任务] 用 Playbook 部署可重复 Web 终态

在 web 组安装 httpd、部署指定首页并启用服务。第一次先限制到一台，随后扩展全组，第二次执行应无法解释的 changed。不得用 shell 模拟包、文件和服务模块。验收语法、主机范围、recap、远端文件、active/enabled 与 HTTP。

请先独立完成。参考解答从下一页开始。

[参考解答] 用三个专用模块表达终态

① [操作点] 使用 `ansible.builtin.dnf`、`copy`、`service` 分别表达包、文件和服务。

② [验证点] `syntax-check`、`list-hosts` 后 `limit` 执行；远端检查 `rpm`、`stat`、`systemctl` 与 `curl`。

③ [验证点] 全组执行后第二次运行，逐项解释 `changed`；最终再次 HTTP 验收。

[Cheatsheet] 专用模块 → 静态边界 → 小范围 → 远端终态 → 全量 → 第二次执行。

[本章收束] 执行成功必须落到远端目标状态

YAML 和模块把意图结构化，但只有目标主机范围正确、模块字段有效、远端对象符合要求且重复执行稳定，Playbook 才真正完成。

第三章 变量、Facts、注册结果与优先级

变量把同一自动化逻辑映射到不同主机；Facts 描述受管节点，register 保存某个任务的运行结果。正确性取决于变量来源、数据类型、主机作用域和实际返回结构。

[概念] 变量可来自 Inventory、group_vars、host_vars、play vars、vars_files、Facts、register、set_fact 和 extra vars；高优先级覆盖低优先级，但考试重点是减少冲突而非背完整表。

[概念] Facts 是每台主机的数据；magic variables 如 inventory_hostname、groups、hostvars 描述 Inventory 上下文。register 结果是字典，字段随模块与循环改变。

[操作语义] `ansible-inventory --host` 观察 Inventory 变量；`ansible -m setup -a filter=...` 查询 Facts；`debug: var=` 显示对象结构而不套 Jinja 双括号。

[知识专题] 变量来源与作用域

① [知识点] 变量来源与作用域

group_vars/all 作用全体，组文件作用组，host_vars 作用单主机。文件名必须与 Inventory 名称对应；把同一变量在多层重复定义会让覆盖难以审计。

② [知识点] Facts 与 magic variables

`ansible_facts['distribution']` 等来自远端采集；`inventory_hostname` 不要求 DNS 可解析，`ansible_host` 是连接地址。`groups['web']` 是主机名列表，`hostvars[h]` 访问另一主机变量。

③ [验证点] 先看类型和最终值

使用 `debug: var=myvar`、type_debug 过滤器和 `inventory --host`。字符串 `false` 与布尔 false 不同，when 中尤其危险。

[Cheatsheet] 变量来源与作用域；Facts 与 magic variables；先看类型和最终值

[操作专题] 组织 group_vars 与 host_vars

① [操作点] 组织 group_vars 与 host_vars

将共同值放 `group_vars/all.yml`，组差异放 `group_vars/web.yml`，主机例外放 `host_vars/servera.yml`；复杂数据使用 YAML 列表/字典。

② [操作点] 注册并读取结果

command 任务 register: check 后读取 check.rc/stdout/stderr/changed；循环 register 的单项结果位于 results。不要假设所有模块都有 stdout。

③ [验证点] 按主机显示并应用差异

先 debug 代表主机的变量/Facts，再 limit 执行；最终在每组受管节点检查差异化文件和服务。

[Cheatsheet] 组织 group_vars 与 host_vars；注册并读取结果；按主机显示并应用差异

[诊断专题] 变量未定义

① [诊断点] 变量未定义

用 -vvv、debug 和 inventory --host 查拼写、文件名、作用域和条件分支，不用 default 隐藏必需变量缺失。

② [诊断点] 字典字段不存在

先 debug 完整 register/Facts 结构，循环结果尤其查看 results；不要从示例版本推定字段。

③ [诊断点] 值正确但类型错误

用 type_debug 检查引号导致的字符串/布尔/整数差异，修正数据源而非在每个 task 强制转换。

[Cheatsheet] 变量未定义；字典字段不存在；值正确但类型错误

[经典任务] 用组变量和 Facts 部署差异配置

web 与 db 组使用不同包、端口和模板值；RHEL 9 才执行目标任务。不得复制两份 Playbook。验收最终变量、Facts 条件、每组文件和服务。

请先独立完成。参考解答从下一页开始。

[参考解答] 把差异放入数据并验证返回结构

① [操作点] 在 `group_vars` 定义 `service_package/service_port`，用 Facts 条件限制 RHEL 9。

② [操作点] 注册验证命令并根据 `rc` 显示明确结果。

③ [验证点] `inventory --host`、`debug type`、`limit` 与远端文件/监听交叉检查。

[Cheatsheet] 数据源 → 类型 → host 最终值 → limit 执行 → 每组终态。

[本章收束] 变量只有在主机上下文中才有确定值

把共享值、组差异和主机例外放在最小层次；Facts 与 `register` 先观察结构再引用。这样优先级不再是猜测，而是可查询的数据合并结果。

第四章 循环、条件、Handler、Block 与错误控制

控制结构用于对数据集合和主机差异重复表达目标，同时让变更、失败和恢复保持可解释。循环不是复制 task，Handler 不是无条件重启，错误控制也不能把必须失败的状态吞掉。

[概念] loop 为每项设置 item，可用 loop_control 改名；when 对每台主机和每个 item 求值。Handler 只在通知 task 返回 changed 时排队，默认在 play 后部运行。

[概念] block 组织任务并可配 rescue/always；failed_when 和 changed_when 改写结果语义，必须基于可靠返回字段。

[操作语义] notify 触发命名 Handler，meta: flush_handlers 提前运行；assert 把前置条件写成明确失败。

[知识专题] 循环数据应保持结构

① [知识点] 循环数据应保持结构

字典列表让 item.name/item.state 明确。避免平行列表和基于索引拼接。loop_control.label 只改善输出，不改变数据。

② [知识点] when 不使用 Jinja 定界符

when: ansible_facts['os_family'] == 'RedHat' 是表达式；字符串布尔需要显式类型。多个条件列表通常 AND。

③ [知识点] Handler 合并通知

同一 Handler 被多次通知通常只运行一次；通知 task 未 changed 则不运行。配置变化才重启是幂等关键。

[Cheatsheet] 循环数据应保持结构；when 不使用 Jinja 定界符；Handler 合并通知

[操作专题] 循环创建对象

① [操作点] 循环创建对象

使用 user/package 等专用模块遍历字典；给 loop_var 命名避免 include 嵌套 item 冲突。

② [操作点] 根据 register 定义结果

command 探测可用 changed_when: false，按 rc 定义 failed_when；但专用查询模块优先。

③ [验证点] 检查 changed 与 Handler 时点

第一轮配置变化应通知，第二轮无变化不应重启；必要时 flush 后再做依赖 Handler 的功能验证。

[Cheatsheet] 循环创建对象；根据 register 定义结果；检查 changed 与 Handler 时点

[诊断专题] ignore_errors 掩盖失败

① [诊断点] ignore_errors 掩盖失败

它继续执行但主机仍有失败语义，可能让后续使用无效状态。合法替代是明确 failed_when 或 block/rescue。

② [诊断点] Handler 未运行

查通知 task 是否 changed、notify 名称、play 是否在失败前到达 Handler，必要时 flush。

③ [诊断点] 每轮都 changed

查 command/shell、时间戳内容、changed_when 和模块目标值；第二次执行定位具体 task。

[Cheatsheet] ignore_errors 掩盖失败；Handler 未运行；每轮都 changed

[知识专题] 控制结构的输出与边界检查

① [参数点] loop_control 不改变业务数据

label 只缩短终端输出，loop_var 改变循环变量名，index_var 保存索引。调试时仍应检查原始 item 结构，不能因为 label 美观就认为数据完整。

② [验证点] failed_when 与 changed_when 必须可复核

条件应引用稳定的 rc、模块字段或明确输出，不匹配本地化人类文本。执行后用 debug: var=result 抽查被改写的任务结果，确保真实失败没有被标成成功。

③ [边界] rescue 不是事务回滚

block 中前一任务已经改变的远端状态不会自动撤销。rescue 必须显式恢复需要恢复的对象，always 只适合清理、记录和无条件收束。

[Cheatsheet] loop_control 管输出/变量名；结果改写必须基于稳定字段；rescue 需显式恢复，不是自动回滚。

[经典任务] 批量创建服务并只在变化时重启

从字典列表安装多个包并部署配置；仅配置改变时重启对应服务。对不满足内存条件主机明确跳过，并在失败时输出可诊断信息。

请先独立完成。参考解答从下一页开始。

[参考解答] 让数据、条件和 Handler 对齐

- ① [操作点] loop 字典调用 package, 模板 task notify Handler。
- ② [操作点] when 使用 Facts; block/rescue 处理受控异常。
- ③ [验证点] 第一轮观察 changed/handler, 第二轮确认无重启, 远端查服务。

[Cheatsheet] 结构数据 → when → 变更通知 → Handler → 第二次执行。

[本章收束] 控制结构应提高可解释性

循环减少重复, 条件限定适用主机, Handler 绑定真实变更, block 保留失败边界。任何结构都应让 recap 和远端终态更清楚, 而不是隐藏错误。

第五章 文件、模板与 Jinja2

文件自动化首先选择所有权边界：完整内容由 `copy/template` 管，单行由 `lineinfile` 管，受标记区块由 `blockinfile` 管。模板把变量渲染成每台主机的完整文件，错误内容也可能幂等。

[概念] `file` 管路径状态/身份/模式，`copy` 分发静态内容，`template` 渲染 Jinja2，`lineinfile` 维护一行，`blockinfile` 维护带 `marker` 的区块。选择过细工具修改完整受管文件会留下未知旧内容。

[概念] Jinja2 `{{ }}` 输出表达式，`{% %}` 控制结构，`{# #}` 注释。模板在控制节点渲染，Facts/变量来自目标主机上下文。

[操作语义] `template` 的 `validate` 在替换前用临时文件运行校验命令；成功后原子替换并 `notify Handler`。`diff` 与远端语法/功能共同验收。

[知识专题] 按所有权选择文件模块

① [知识点] 按所有权选择文件模块

完整文件归项目所有用 `template/copy`；只拥有一行用 `lineinfile`；只拥有区块用 `blockinfile`。`file state=touch` 每次改变时间，不适合只保证存在。

② [知识点] 模板变量必须处理缺失与类型

用 `mandatory/assert` 处理必需数据；`default` 只用于合法默认。过滤器如 `join`、`sort`、`to_nice_yaml` 改变输出，要检查目标格式。

③ [验证点] 内容、元数据和消费者分层

`stat` / `slurp` / `checksum` 查文件，应用 `-t` 或 `validate` 查语法，服务 `active` / `HTTP` 查功能。

[Cheatsheet] 按所有权选择文件模块；模板变量必须处理缺失与类型；内容、元数据和消费者分层

[操作专题] 渲染差异配置

① [操作点] 渲染差异配置

模板使用 `inventory_hostname`、组变量和 Facts；`for` 循环生成重复行，`if` 只表达真正主机差异。保持缩进与换行符合目标格式。

② [操作点] 安全替换并通知

`template` 设置 `owner/group/mode`、`backup`（按需）、`validate`，并 `notify restart Handler`；只有内容/元数据变化才 `changed`。

③ [验证点] check/diff 与远端语法

先 `--check --diff` 预览支持的变化，再 `limit` 执行；远端运行应用校验并查看渲染内容。

[Cheatsheet] 渲染差异配置；安全替换并通知；check/diff 与远端语法

[诊断专题] 模板变量未定义

① [诊断点] 模板变量未定义

定位变量作用域和拼写，debug 数据结构；不在模板各处 default 空字符串。

② [诊断点] 渲染成功但服务失败

控制端 Jinja 合法不等于目标配置语法合法；使用 `validate` 和远端日志。

③ [诊断点] lineinfile 每次 changed

检查 `regexp` 是否能匹配写入后的 `line`，避免插入重复行；必要时选择 `template`。

[Cheatsheet] 模板变量未定义；渲染成功但服务失败；lineinfile 每次 changed

[知识专题] 模板渲染、校验与换行细节

① [参数点] validate 命令使用占位符

`validate: '/usr/sbin/sshd -t -f %s'` 让模块把临时文件路径代入 `%s`。命令不经 Shell，因此管道与重定向不能直接使用；校验工具必须能读取临时路径。

② [知识点] Jinja 空白会改变真实配置

`trim_blocks`、`lstrip_blocks` 和 `-{%` 等控制空白。多一空行通常无害，但 YAML、sudoers、hosts 或严格配置中的缩进与末尾换行可能改变语义，必须查看 `--diff` 和远端文件。

③ [验证点] 模板依赖清单

模板引用的每个变量都应能从 `defaults`、`group_vars`、`host_vars` 或 `Facts` 追溯；修改模板后回归所有消费主机组，而不只测试当前 `limit` 主机。

[Cheatsheet] `validate` 用 `%s` 临时路径；空白控制可能改语义；diff 后回归所有模板消费者。

[经典任务] 为不同主机生成服务配置并安全重载

使用一个模板按 web 组变量生成监听端口和后端列表，所有者/模式固定；配置校验成功且内容变化时才重载服务。验收 diff、远端内容、语法、Handler、监听和第二次执行。

请先独立完成。参考解答从下一页开始。

[参考解答] 让模板成为完整文件真相

- ① [操作点] 在 j2 中使用明确变量、循环与最小条件。
- ② [操作点] template 设元数据和 validate, notify reload Handler。
- ③ [验证点] check/diff、limit、远端语法/监听, 第二次无 changed。

[Cheatsheet] 数据 → 模板 → validate → 原子替换 → Handler → 远端功能/幂等。

[本章收束] 文件模块的选择决定维护边界

完整文件、单行和区块应由不同模块承担。Jinja 只负责渲染, validate 与远端功能证明内容可用; 第二次执行证明表达稳定。

第六章 外部数据、敏感变量与 Vault

外部变量把用户、服务和环境数据从任务逻辑中分离；Vault 只加密静态内容，不自动隐藏运行时输出。正确流程同时保证数据结构可解析、秘密在磁盘加密、运行时可解密且日志不泄露。

[概念] `vars_files` 显式加载数据文件，`group_vars/host_vars` 按 Inventory 自动加载。YAML 列表适合对象集合，字典适合按名称索引；稳定 schema 比在 task 中兼容多种形状更可靠。

[概念] Vault 可加密整个文件或单个字符串。vault ID 允许多个密码来源；密码文件本身必须受保护且不进入发布内容。

[操作语义] `ansible-vault create/edit/view/encrypt/decrypt/rekey/encrypt_string` 管密文；`--vault-password-file` 或 `--vault-id` 提供运行时秘密。`no_log: true` 只在必要 task 抑制输出。

[知识专题] 外部数据要有明确 schema

① [知识点] 外部数据要有明确 schema

批量用户可定义 `name/groups/password_hash` 等键；Playbook 用 `assert` 验证必需键，避免部分对象执行到中途才失败。

② [知识点] Vault 密文仍是内容源

编辑用 `ansible-vault edit`，不先 `decrypt` 留明文临时文件。`rekey` 更换密码保持数据加密；`decrypt` 是明确导出明文的高风险操作。

③ [验证点] 检查文件头和安全输出

Vault 文件以 `$ANSIBLE_VAULT;` 开头；`view` 能按授权读取。运行后检查目标终态，不打印秘密值。

[Cheatsheet] 外部数据要有明确 schema；Vault 密文仍是内容源；检查文件头和安全输出

[操作专题] 加载外部用户列表

① [操作点] 加载外部用户列表

`vars_files` 加载 YAML，`loop` 遍历用户字典；密码字段应是目标模块所需哈希，不把明文直接传给 `user.password`。

② [操作点] 创建和改密 Vault

使用 `create/edit`；密码轮换用 `rekey old/new vault-id`。密码文件权限收紧并排除版本控制。

③ [验证点] 语法、解密与远端对象

先 vault view 和 playbook syntax-check，再 limit；远端用 getent/id 验证用户，不回显哈希。

[Cheatsheet] 加载外部用户列表；创建和改密 Vault；语法、解密与远端对象

[诊断专题] no vault secrets found

① [诊断点] no vault secrets found

检查当前项目是否收到正确 vault-id/password file，文件是否被错误 decrypt 或 vault ID 不匹配。

② [诊断点] 秘密出现在日志

给最小敏感 task 设置 no_log，并审查 debug、失败消息与注册变量；不能靠删终端历史作为修复。

③ [诊断点] 数据结构与 loop 不匹配

debug 仅显示非敏感键/类型，使用 assert；修正 schema，不在任务中叠加复杂兼容。

[Cheatsheet] no vault secrets found；秘密出现在日志；数据结构与 loop 不匹配

[知识专题] Vault ID、文件权限与日志泄露面

① [参数点] 多 Vault 使用 label

`--vault-id dev@prompt --vault-id prod@/path/key` 将密文 label 与密码来源绑定。运行错误时先确认密文头中的 vault ID 和命令提供的 label，不按文件名猜密码。

② [边界] 密码文件不是 Vault

密码文件通常是解密密钥本身，必须 `0600`、排除版本控制并按题目路径管理；把它再放进同一 Vault 会形成无法启动的循环依赖。

③ [验证点] 失败输出也可能泄密

模块失败的 invocation、register、debug 和 callback 均可能包含参数。对传递秘密的最小 task 使用 `no_log`，同时避免随后 debug 整个注册对象。

[Cheatsheet] 多 Vault 核对 label；密码文件 0600 且不入库；敏感 task 与后续 register 输出一一起审计。

[经典任务] 使用加密变量批量创建账号

下载/提供的用户列表保存在外部 YAML，密码哈希放 Vault；按组变量选择附加组。密码文件路径由题目指定且不得泄露。验收 Vault 保持加密、Playbook 可运行、用户/组正确且输出无秘密。

请先独立完成。参考解答从下一页开始。

[参考解答] 以 schema 和 Vault 边界驱动用户任务

- ① [操作点] 验证用户列表必需键, Vault 保存哈希与敏感值。
- ② [操作点] vars_files 加载, user 模块 loop 创建; 敏感 task no_log。
- ③ [验证点] vault view/syntax/limit, 远端 getent/id, 最终确认源文件仍加密。

[Cheatsheet] schema/assert → Vault → 安全加载 → user loop → 远端身份 → 密文与日志审计。

[本章收束] 加密不等于不会泄露

Vault 保护静态文件, no_log 控制任务输出, 文件权限和版本控制保护密码入口。三层同时成立, 外部数据才既可维护又安全。

第七章 Include、Import、Role 与 Collection

内容拆分必须保留执行边界。import 在解析期静态展开，include 在运行期动态选择；Role 用固定目录把 tasks、handlers、templates、files 和变量组织为可复用接口；Collection 提供 FQCN、Role 和插件版本。

[概念] include_tasks 动态执行并可按 loop/when 选择；import_tasks 静态展开，适合固定结构和 list-tasks。import_playbook 只能在 playbook 顶层。

[概念] Role defaults 是低优先级可覆盖接口，vars 更强不适合作为用户配置入口。Collection requirements 锁定依赖来源/版本，FQCN 避免模块名冲突。

[操作语义] `ansible-galaxy role init` 创建 Role 骨架，`ansible-galaxy collection install -r requirements.yml` 安装依赖；`ansible-doc -t role` / 模块文档查接口。

[知识专题] 动态与静态复用

① [知识点] 动态与静态复用

import 在解析时展开，标签/list-tasks 可见；include 在运行时根据变量与循环决定。不要只背先后，按是否需要运行时选择。

② [知识点] Role 目录和变量接口

tasks/main.yml 是入口，handlers/templates/files 自动按 Role 路径解析；defaults 提供可覆盖默认值，meta 声明依赖。

③ [验证点] 依赖与路径可复现

requirements 文件进入项目；离线/考试材料按给定 tar/路径安装，`ansible-galaxy collection list` 核对实际版本。

[Cheatsheet] 动态与静态复用；Role 目录和变量接口；依赖与路径可复现

[操作专题] 把单文件重构为 Role

① [操作点] 把单文件重构为 Role

移动任务、Handler、模板和文件，不改变 notify 名称与变量语义；把环境差异转为 defaults 或调用时 vars。

② [操作点] 安装并使用 Collection/System Role

requirements 指定 name/source/version；Playbook 用 FQCN 和 `roles:` 调用，阅读 Role README/defaults 获取变量 schema。

③ [验证点] 语法、任务图与远端终态

syntax-check、list-tasks/tags、limit 执行；Role 重构前后远端终态与第二次执行一致。

[Cheatsheet] 把单文件重构为 Role；安装并使用 Collection/System Role；语法、任务图与远端终态

[诊断专题] 找不到 Role/Collection

① [诊断点] 找不到 Role/Collection

检查 roles_path/collections_paths、项目目录、requirements 安装位置和名称空间，不复制内容到随机路径。

② [诊断点] 变量无法覆盖

检查是否错误放在 role vars 而非 defaults，以及调用层变量名/schema。

③ [诊断点] include 标签行为意外

动态 include 的标签不会自动以相同方式传播到内部 task；按文档使用 apply 或给内部任务标签。

[Cheatsheet] 找不到 Role/Collection；变量无法覆盖；include 标签行为意外

[知识专题] Role 接口、依赖与标签传播

① [知识点] defaults 是公开接口，不是所有变量仓库

调用者需要覆盖的值进入 defaults；Role 内部常量可放 vars，但过强优先级会阻止环境差异。变量名加 Role 前缀可减少多 Role 冲突。

② [参数点] requirements 可同时描述 Role 与 Collection

实际项目可分 roles/requirements.yml 与 collections/requirements.yml，也可按工具支持格式组织。离线 tarball 的 source 与版本要保持题目给定 lineage，不得擅自换最新版。

③ [验证点] 重构前后比较任务与 Handler

`--list-tasks`、tags、远端 diff 和第二次执行共同证明移动内容没有丢失 Handler、模板路径或变量覆盖。

[Cheatsheet] defaults 暴露接口；requirements 固定真实依赖；重构用任务图、远端 diff 与幂等证明等价。

[经典任务] 把 Web Playbook 重构为可复用 Role

创建 `web_role`，包含包、模板、Handler 和服务；默认端口可由组变量覆盖。通过 `requirements` 安装题目 Collection，并用新 Playbook 调用。验收依赖、任务图、两组差异、Handler 和幂等性。

请先独立完成。参考解答从下一页开始。

[参考解答] 保留行为地移动内容

① [操作点] role init, 移动 tasks/handlers/templates/files, 接口值放 defaults。

② [操作点] requirements 安装 Collection, 以 FQCN 调用。

③ [验证点] syntax/list-tasks、limit、全量、远端终态、第二次执行。

[Cheatsheet] 依赖安装 → Role 接口 → 静态边界 → 远端等价 → 幂等。

[本章收束] 复用的价值是稳定接口而非更多目录

选择 include/import 的时点语义, Role 用 defaults 暴露最小接口, Collection 用 requirements 固定依赖。重构完成必须证明远端行为未改变。

第八章 自动化用户、软件包、仓库、服务与计划任务

本章把 RHCSA 手工状态映射为专用模块参数。自动化重点是声明最终用户、仓库、包、服务和 cron，而不是远程执行对应命令；变量数据驱动多对象，Handler 只响应配置变化。

[概念] user/group/authorized_key 管身份，dnf/package/yum_repository 管软件，service/systemd_service 管当前与启动状态，cron 管周期声明。每个模块的 state 对应目标终态。

[概念] 模块幂等依赖稳定输入：随机密码盐、动态时间戳、无 regexp 的追加文本都会制造持续 changed。

[操作语义] 使用 FQCN；password 传哈希，groups 配合 append；enabled 与 state 分别表达启动和当前服务。

[知识专题] 身份模块参数边界

① [知识点] 身份模块参数边界

user.groups 默认可替换附加组，append: true 才追加；remove 要明确是否删除家目录。authorized_key 的 exclusive 对循环逐项使用可能互相删除。

② [知识点] 仓库与包分层

yum_repository 创建 repo 配置，rpm_key 管 key，dnf 安装包。仓库文件存在后仍要 makecache/包事务证据。

③ [验证点] 服务双态与 cron 身份

service 的 started/enabled 分别验证；cron 的 user、时间字段和 job 必须与题意一致，远端检查 crontab/产物。

[Cheatsheet] 身份模块参数边界；仓库与包分层；服务双态与 cron 身份

[操作专题] 用外部列表创建用户

① [操作点] 用外部列表创建用户

loop 字典调用 group/user/authorized_key；密码哈希从 Vault，no_log 限制敏感 task。

② [操作点] 配置仓库、包和服务

先 key/repo，再 dnf；模板 notify Handler，service 保证 started+enabled。

③ [验证点] 模块结果与远端对象

第二次执行无 `changed`；`getent/id`、`dnf repolist/rpm`、`systemctl`、`crontab` 和实际服务功能。

[Cheatsheet] 用外部列表创建用户；配置仓库、包和服务；模块结果与远端对象

[诊断专题] 用户附加组丢失

① [诊断点] 用户附加组丢失

检查 `groups` 是否在未 `append` 情况下替换全部列表；修正数据为完整目标或 `append`。

② [诊断点] 服务每次重启

检查 `Handler` 是否被动态模板/无条件 `changed task` 每次通知。

③ [诊断点] `cron` 存在但不执行

远端查 `user`、绝对路径、环境、`crond` 日志和产物；模块 `changed=0` 只证明条目稳定。

[Cheatsheet] 用户附加组丢失；服务每次重启；`cron` 存在但不执行

[知识专题] 模块参数的替换、追加与删除语义

① [参数点] `user` 模块表达最终账号而非 `useradd` 命令

`state`、`uid`、`group`、`groups`、`append`、`shell`、`password_expire_max` 等共同描述目标。删除账号时 `remove: true` 会删除家目录，必须由题意明确。

② [参数点] `dnf state` 的范围

`present` 保证安装，`latest` 会随仓库元数据升级并可能持续改变发布结果，`absent` 删除。只有题目要求最新时使用 `latest`，并阅读依赖事务。

③ [验证点] `authorized_key` 与 `sudoers` 仍需功能测试

公钥文件存在后从控制节点实际 SSH；`sudoers` 由 `template/copy` 部署时用 `visudo -cf %s validate`，再以目标用户执行允许与拒绝命令。

[Cheatsheet] `user` 删除/组语义要显式；`dnf latest` 只按题意；公钥与 `sudoers` 最终做真实认证/授权测试。

[经典任务] 用变量为多组主机配置基础服务

从外部用户数据创建组/用户/公钥，配置指定仓库和包，部署服务并只在配置变化时重启，创建周期任务。验收每组数据差异、秘密安全、服务双态、cron 产物和第二次执行。

请先独立完成。参考解答从下一页开始。

[参考解答] 把手工命令映射为专用模块

- ① [操作点] `group/user/authorized_key loop`; Vault 哈希。
- ② [操作点] `rpm_key/yum_repository/dnf/template/Handler/service`。
- ③ [操作点] `cron` 声明; 远端 `getent/rpm/systemctl/crontab/功能`, 第二次执行。

[Cheatsheet] 数据 → 身份 → 仓库/包 → 配置/Handler/服务 → `cron` → 远端/幂等。

[本章收束] 自动化对象要以 state 而非命令表达

专用模块让 Ansible 比较当前和目标; 变量提供差异, Handler 绑定真实变化。recap 之外仍需在受管节点证明身份、包、服务和调度功能。

第九章 自动化网络、防火墙与 SELinux

网络、安全与 SELinux 自动化要求同时表达当前和持久状态，并避免批量断连。模块或 RHEL System Role 描述目标连接；firewalld immediate/permanent、SELinux fcontext/port/boolean 分别对应不同策略对象。

[概念] 网络变更可能切断 Ansible 控制通道，应先 limit、保留旧连接和控制台恢复路径。RHEL network System Role 以结构化变量表达连接集合。

[概念] firewalld 的 permanent 与 immediate 分别写持久和 runtime；SELinux fcontext 建规则后仍要 restorecon，端口和 Boolean 是独立入口。

[操作语义] `ansible.posix.firewalld`、`community.general.sefcontext/seport` 与 `ansible.posix.seboolean`（以安装 collection 文档为准）表达状态；command 只补没有模块的恢复动作。

[知识专题] 网络 role 输入必须按节点变量化

① [知识点] 网络 role 输入必须按节点变量化

每台主机地址/网关/DNS 放 host/group vars，不在 task 中按主机名堆 when。应用连接前验证设备名与现有连接。

② [知识点] firewalld 双态

`permanent: true` 写持久；`immediate: true` 同时应用 runtime，前提 firewalld 运行。zone/service/port 必须匹配。

③ [知识点] SELinux 三类状态

sefcontext 管路径规则，restorecon 应用当前标签；seport 管协议/端口类型；seboolean 的 persistent 管持久值。

[Cheatsheet] 网络 role 输入必须按节点变量化；firewalld 双态；SELinux 三类状态

[操作专题] 小范围应用网络

① [操作点] 小范围应用网络

syntax/check（role 支持范围）、limit 单主机，等待连接恢复并重新 gather facts；再扩全组。

② [操作点] 部署非标准 Web 安全状态

template 配置端口，firewalld service/port 双态，sefcontext+restorecon，seport，服务 Handler。

③ [验证点] 远端终态和外部访问

nmcli/ip、firewall-cmd 双查询、semanage/lsc -Z/getsebool、ss 与外部 curl；第二次执行。

[Cheatsheet] 小范围应用网络；部署非标准 Web 安全状态；远端终态和外部访问

[诊断专题] 网络 task 后 unreachable

① [诊断点] 网络 task 后 unreachable

从控制台/旧地址确认 profile 与 route，检查 Ansible 是否等待重连；不要在全组重复错误连接。

② [诊断点] firewalld 每次 changed

检查 immediate/permanent、zone 和 module 版本字段，避免 reload command 每次执行。

③ [诊断点] 标签规则存在但访问仍拒绝

确认 restorecon 已应用、目标类型/端口/Boolean 与 AVC；规则存在不等于当前标签。

[Cheatsheet] 网络 task 后 unreachable；firewalld 每次 changed；标签规则存在但访问仍拒绝

[知识专题] 网络批量变更的串行与恢复策略

① [边界] serial 降低影响但不修复错误

play 设置 serial: 1 可逐台应用网络，配合 max_fail_percentage 控制停止条件。第一台失败时立即停止并调查，不能依赖 serial 自动回滚连接。

② [验证点] 等待重连要验证新地址

网络 Role 完成后可用 wait_for_connection，但成功只证明 Ansible 重新连接；继续运行 ip route get、getent 和目标协议测试，确认连接不是经错误备用路径恢复。

③ [边界] SELinux module 与 restorecon 分工

规则模块只保证 policy mapping，当前对象仍需恢复标签。restorecon command 可先用 -n 预览差异，再在规则确定后应用并用 matchpathcon / ls -Z 对比。

[Cheatsheet] 网络用 serial/停止条件降低批量风险；重连后验地址/路由；fcontext 规则与 restorecon 当前标签分开。

[经典任务] 自动部署自定义目录与端口 Web 服务

按主机变量配置网络；部署 httpd 自定义目录和 8080，配置 firewalld 当前/持久、SELinux 文件规则和端口类型。不得 Permissive。先一台再全组，验收连接恢复、安全三层、监听、外部 HTTP 和幂等性。

请先独立完成。参考解答从下一页开始。

[参考解答] 按风险顺序应用网络与安全

① [操作点] network role limit, 一台恢复连接并验证路由/DNS。

② [操作点] 模板/Handler、firewalld 双态、sefcontext+restorecon、seport。

③ [验证点] 远端命令与外部 curl, 全组后第二次执行。

[Cheatsheet] 网络小范围/重连 → 服务 → 防火墙双态 → SELinux 规则/当前标签/端口 → 外部功能/幂等。

[本章收束] 自动化仍要尊重系统状态分层

模块把目标状态写成参数, 但连接风险、runtime/permanent 和规则/当前标签的区别没有消失。先小范围、再远端证据、最后全量和幂等。

第十章 自动化存储、文件系统与挂载

存储自动化必须面对每台主机设备条件不同和写操作不可逆。模块能保证对象 state，却不能判断题目中的设备是否安全；设备证据、容量来源、文件系统和 mount 双态仍需逐层验证。

[概念] community.general.lvg/lvol 管 VG/LV，community.general.filesystem 管签名，ansible.posix.mount 管 fstab 与当前挂载；实际 FQCN/字段以已安装 Collection 为准。

[概念] 容量可表达绝对值、增量或百分比。自动化应描述目标总量，避免每次运行都增加；扩容文件系统需要 resizefs 或专用步骤，XFS 不缩小。

[操作语义] 先 gather facts/lsblk 调查，但不从设备名模式推定空闲。模块 state present 只推进对应层；远端 pvs/vgs/lvs/blkid/findmnt/df 形成验收。

[知识专题] 设备安全不是模块自动判断

① [知识点] 设备安全不是模块自动判断

lvg 的 pvs 参数会初始化/加入设备；题目设备不确定时先 assert 事实和人工证据门。不要自动 wipefs/force。

② [知识点] 目标大小避免增量漂移

lvol size 写目标容量或百分比；使用扩容参数时确认模块幂等语义。每次 +1G 的命令式 shell 会持续增长。

③ [验证点] LVM、文件系统和挂载三层

lvs 变大不代表 df 变大；mount state mounted/ephemeral/present 的当前与 fstab 语义不同，按模块文档选择。

[Cheatsheet] 设备安全不是模块自动判断；目标大小避免增量漂移；LVM、文件系统和挂载三层

[操作专题] 用变量描述每主机布局

① [操作点] 用变量描述每主机布局

host_vars 定义 devices、vg、lv、size、fstype、mount；assert 设备键存在和尺寸条件。

② [操作点] 按依赖创建

lvg → lvol → filesystem → file mountpoint → mount。已有 filesystem 不重建；扩容使用 resizefs 并保留数据。

③ [验证点] 执行前后与幂等

limit 单主机，远端查对象/UUID/fstab/findmnt/df/数据；第二次无 changed，再扩全组。

[Cheatsheet] 用变量描述每主机布局；按依赖创建；执行前后与幂等

[诊断专题] VG 不存在或设备不存在

① [诊断点] VG 不存在或设备不存在

用 hostvars/Facts 与远端 lsblk 证明条件，fail/assert 给明确消息，不用 ignore_errors。

② [诊断点] LV 变大 df 不变

文件系统 resize 未完成；按类型执行模块 resize 或 xfs_growfs，不继续增加 LV。

③ [诊断点] mount 每次 changed

检查 src 是否稳定（UUID/设备路径）、opts 顺序、state 与 fstab 现状；不要用 shell mount。

[Cheatsheet] VG 不存在或设备不存在；LV 变大 df 不变；mount 每次 changed

[知识专题] 设备事实、容量单位与失败保护

① [知识点] ansible_devices 不是安全空闲证明

Facts 可给设备大小、分区和型号，但未必包含所有签名/LVM/挂载关系。真正初始化前仍需受控 `lsblk/blkid/pvs` 证据或题目明确保证。

② [参数点] 容量字符串保持题意单位

`size: 2g`、百分比和 `resizefs` 行为由当前 collection 模块定义。`extent` 题若模块不能精确表达，应计算并 `assert` 结果，不用近似容量悄悄改变目标。

③ [失败边界] 先 fail 再做破坏性任务

用 `assert` 检查设备变量存在、VG 条件和目标大小；`block/rescue` 可输出明确错误，但不能在 `rescue` 中格式化另一块猜测设备。

[Cheatsheet] Facts 只做初筛；容量保持题意单位并 `assert`；破坏性 task 前显式 `fail`，`rescue` 不猜替代设备。

[经典任务] 按主机变量创建并挂载逻辑卷

在指定主机使用给定空闲设备创建 VG/LV/ext4 或 XFS 并持久挂载；空间不足时输出明确失败，不得格式化未知设备。随后扩容并保留标记数据。验收模块结果、三层容量、fstab、挂载和幂等。

请先独立完成。参考解答从下一页开始。

[参考解答] 把设备差异放入变量并设置证据门

- ① [操作点] `host_vars` 描述布局, `assert` 设备/容量前提。
- ② [操作点] `lvg/lvol/filesystem/mount` 按依赖; 扩容 `resizefs`。
- ③ [验证点] `pvs/vgs/lvs`、`blkid`、`findmnt/df`、标记校验, 第二次执行。

[Cheatsheet] 变量/设备证据 → LVM → filesystem → mount → 数据 → 扩容 → 幂等。

[本章收束] 幂等模块不能替代设备风险判断

模块擅长比较已知对象状态, 但初始化哪个设备仍由数据和证据决定。按依赖创建并分层验收, 才能避免自动化放大破坏性错误。

第十一章 Ansible 故障排除、验证与幂等性

Ansible 故障首先归类为控制端解析、Inventory 匹配、SSH/提权、模块执行或远端终态。verbosity 和 register 提供证据；ignore_errors、无条件 changed_when 或 recap 绿色不能替代根因修复。

[概念] FAILED 表示主机可达但任务失败，UNREACHABLE 表示连接层未建立，SKIPPED 是条件未满足，CHANGED 是模块认为状态改变。它们是执行分类，不是业务结论。

[概念] 幂等性要求相同目标状态重复执行不再改变；正确性要求目标本身符合题意。错误配置也可能稳定幂等，因此两者必须组合。

[操作语义] 从 `--syntax-check/--list-hosts` 开始，使用 `-v/-vvv` 增加证据，debug 注册对象，`--step/--start-at-task` 谨慎缩小；远端协议与系统查询最终验收。

[知识专题] 按失败层分类

① [知识点] 按失败层分类

YAML/模块字段在控制端；0 hosts 在 Inventory；unreachable 在 SSH；become 在 sudo；failed msg 在模块/远端；绿色 recap 后仍查终态。

② [知识点] changed 语义可被模块与规则改写

changed_when 应基于可靠输出，不为追求全绿无条件 false；failed_when 不能吞掉真实失败。

③ [验证点] 第二次执行与远端状态并行

比较两次 recap/任务 diff，同时从受管节点查文件、服务、端口、挂载和数据。

[Cheatsheet] 按失败层分类；changed 语义可被模块与规则改写；第二次执行与远端状态并行

[操作专题] 最小化复现

① [操作点] 最小化复现

syntax/list/limit 单主机，使用 tags/start-at-task 前确认依赖；verbosity 只提升到能回答问题的层。

② [操作点] 读取 register 结构

debug var 完整结果，区分 rc/stdout/msg/results；用 assert 明确前置和验收条件。

③ [验证点] 修复后做全链回归

从失败层向下到远端功能，再全组执行和第二次执行；检查 Handler 是否按变化运行。

[Cheatsheet] 最小化复现；读取 register 结构；修复后做全链回归

[诊断专题] 变量未定义/模板错误

① [诊断点] 变量未定义/模板错误

查最终 hostvars、数据类型、模板路径和 scope；不要用 default 空值掩盖。

② [诊断点] unreachable 被当作 task 失败

业务模块尚未执行，先修 SSH/host key/remote_user/Python。

③ [诊断点] 每次 changed 但终态相同

查 command/shell、mtime、随机值、模板动态内容与 module 参数；不要全局 changed_when false。

[Cheatsheet] 变量未定义/模板错误；unreachable 被当作 task 失败；每次 changed 但终态相同

[知识专题] check mode、diff 与可观测性边界

① [知识点] check mode 支持度取决于模块

支持 check_mode 的模块预测 changed，不支持的任务可能跳过或仍需特殊处理。不能把一次 `--check` 全绿当作真实执行成功。

② [验证点] diff 只显示可公开差异

`--diff` 有助于模板/文件审查，但可能暴露敏感内容；秘密任务应 `no_log`，并用非敏感结构证据验收。二进制或某些模块没有可读 diff。

③ [诊断点] start-at-task 会跳过前置状态

从中间开始可能缺少 Facts、变量设置、文件下载或 Handler 通知，只适合前置状态已明确成立的复现。最终仍从完整 Playbook 回归。

[Cheatsheet] check 支持度要核对；diff 注意秘密；start-at-task 不替代完整回归。

[经典任务] 修复一个多层故障项目

项目同时存在错误 Inventory 组、变量类型、模板字段、Handler 名称和无条件 changed。要求逐层修复，不能 ignore_errors；最终全组终态正确且第二次无无法解释 changed。

请先独立完成。参考解答从下一页开始。

[参考解答] 按层收缩并回归

① [诊断点] syntax/list-hosts/inventory 先修控制与范围；ping/become 修连接。

② [诊断点] limit 运行，debug register/变量，修模板与 Handler。

③ [验证点] 远端文件/服务/端口，全组与第二次执行。

[Cheatsheet] 控制端 → Inventory → 连接/become → task/Handler → 远端终态 → 全量/幂等。

[本章收束] 绿色执行不是最终判据

可靠排错让每个错误回到所属层，可靠验收同时看执行语义和远端事实。幂等性是正确终态的附加条件，不是替代条件。

第十二章 RHCE 综合任务

RHCE 综合任务把项目配置、变量、Vault、Role 和系统状态组合为一套多节点声明。完成标准不是 Playbook 数量，而是所有目标主机被正确覆盖、秘密安全、远端终态正确且第二次执行稳定。

[概念] 控制面包含 `ansible.cfg`、Inventory、requirements、group_vars/host_vars、Vault、templates 和 roles；数据面是受管节点上的用户、包、服务、网络、安全和存储。

[概念] 执行应从静态边界到单主机，再到主机组和全量；每个阶段保留 recap 与远端证据。

[操作语义] 发布入口是可重复的构建/执行命令；`--syntax-check`、inventory graph、requirements install、vault access、limit/full/idempotence 和远端验收构成主链。

[知识专题] 先审计项目完整性

① [知识点] 先审计项目完整性

确认题目要求的文件路径、名称、权限、Vault 密文、Role/Collection 依赖和 Inventory 组均存在；不从任意工作目录执行。

② [知识点] 桥接 RHCSA 目标为模块 state

用户/包/服务/防火墙/SELinux/存储使用专用模块或 System Roles；shell 仅补明确缺口并定义 changed/failed。

③ [验证点] 每组主机有独立终态矩阵

web、db、balancer 等分别核对变量差异、文件、服务、端口和安全；inventory 漏主机不会在 recap 中自动报错。

[Cheatsheet] 先审计项目完整性；桥接 RHCSA 目标为模块 state；每组主机有独立终态矩阵

[操作专题] 执行前门

① [操作点] 执行前门

安装 requirements, vault view, syntax/list-hosts/list-tasks; ping/become; check/diff（支持范围）。

② [操作点] 从一台到全量

limit 代表主机，修复控制/连接/task 问题并验远端；扩组、全量，观察 Handler 与失败分类。

③ [验证点] 第二次执行和外部功能

再次全量，解释 changed；从客户端验证 HTTP/网络，从节点验证身份、包、服务、SELinux、mount 和数据。

[Cheatsheet] 执行前门；从一台到全量；第二次执行和外部功能

[诊断专题] 全绿但漏主机

① [诊断点] 全绿但漏主机

对照题目与 `--list-hosts` /Inventory graph，不能只看 recap 中出现的主机。

② [诊断点] 秘密或生成物泄露

检查 debug/no_log、明文密码文件权限、构建输出和版本控制；保持 Vault 加密。

③ [诊断点] 修一组破坏另一组

把差异移到变量/模板条件，回归所有共享 Role 消费者；不要复制分叉 Role。

[Cheatsheet] 全绿但漏主机；秘密或生成物泄露；修一组破坏另一组

[知识专题] 交付目录、命名与可重复执行入口

① [验证点] 题目要求的路径本身可能评分

Inventory、ansible.cfg、playbook、Vault、templates 和 roles 必须位于指定绝对路径并具备正确权限。内容等价但文件名或入口错误仍可能无法被评分命令发现。

② [知识点] 控制节点产物与受管节点产物分开

template 源、requirements 和 Vault 留在控制端；目标配置、用户和挂载在远端。验证脚本要明确在哪一端执行，不能看到控制端文件就判定远端完成。

③ [验证点] 最终执行记录可重现

从项目目录运行固定命令，记录 inventory graph、syntax、首次/第二次 recap 和远端矩阵。清理临时明文与 debug 任务，但不删除题目要求的内容源。

[Cheatsheet] 路径/命名是接口；控制端与远端对象分开；固定入口重现首次、二次与远端矩阵。

[经典任务] 完成一个多组主机自动化项目

建立指定 Inventory/配置、Vault 与 Role，配置用户、仓库、包、模板/Handler、网络、防火墙/SELinux、LVM/mount，并使用 requirements。不得泄露秘密或掩盖错误。验收文件路径、主机覆盖、各组终态、外部功能与第二次执行。

请先独立完成。参考解答从下一页开始。

[参考解答] 按发布门连续推进

- ① [操作点] 审计项目/依赖/Vault/Inventory, syntax/list/ping/become。
- ② [操作点] limit 代表主机，远端验收；扩组与全量。
- ③ [验证点] 全量第二次执行；按主机组矩阵查身份/服务/网络/安全/存储与外部功能。

[Cheatsheet] 项目完整性 → 静态/连接 → 单机 → 分组/全量 → 远端矩阵 → 幂等/安全审计。

[本章收束] 自动化交付是可复现的正确终态

项目源、依赖、秘密和执行边界必须可重建；受管节点的目标状态必须由专用模块表达并由外部证据验证；第二次执行则证明这种正确状态可以稳定维持。